

从具体操作理解排序

排序算法 学习手册

从几个数字如何移动
到不同条件下如何选择方法

90 个目录条目 · 25 幅过程图

操作步骤、数字示例、效率差异、适用前提与参考来源。
不要求已有编程或算法基础。

中文版 | 2026 年 10 月 5 日

先读这里

排序不是只知道 1 应该在 2 前面，而是按照一套可执行的步骤，把原来的全部记录重新安排好。本手册沿着具体动作讲解：比较了什么，什么被暂存，接着移动什么，以及哪一部分已经能保证正确。

下一节先列完整目录，再展开课程。90 个条目包括核心算法、重要变体、相关实现家族、系统方法、受限模型，以及少量经常被冠以排序之名的非排序过程。这不代表 90 项独立发明。同义名称放在同一条目，做法确实不同的变体才可能单列。

不存在不看条件的“从最快到最慢”。A—E 组按明确前提与保证组织普通串行工作；F—I 组则在新的数据或机器条件下重新比较。同档中相邻条目的先后，服务于理解，不冒充测速排名。后面的效率导读会说明这些边界。

覆盖范围广，但不是穷尽名单。所有增量序列、枢轴采样方式、自适应归并策略、研究型整数排序、数据无关电路、并行映射、混合方案与未来发明，不可能由这份有限目录全部代表。阈值大小等参数选择，也不单算一种算法。

多数条目给出可直接手算的配方与完整小例子。少数高级构造会明确标识：C23 解释稳定块归并的动作，但不展开 WikiSort／GrailSort 的每次标签交换；D25 解释漏斗构造；D26 说明 AKS 的组件，但不提供完整扩展图与比较器调度；D31 解释 IPS4o 的阶段，但不代替完整并发实现。它们讲清构造和限度，不把短概述伪装成工业代码。

初学时，可先读基础，再按 C01 → C03 → C04 → C09 → C13 → C16 → D01 → D03 体验几种基本动作。目录本身仍按效率条件组织，这条可选路线只是先从容易手算的动作入手。每看完一步，都试着遮住结果，亲手完成下一步。

导航：目录链接可跳到条目，方括号资料编号可跳到参考来源。C、D、X 是固定标识，不是速度等级；中英文版编号一致。

完整目录

基础与符号

有条件的效率导读

A 键有结构：有条件的线性方法

X09 直接寻址循环排序

D29 位图排序：有界互异整数

D01 计数排序

D02 鸽巢排序

D03 最低有效位优先基数排序

D04 最高有效位优先基数排序

D05 美国旗排序

D06 二进制 MSD 交换排序

D27 字典树排序 (Trie sorting)

D23 Spreadsort 混合分布排序

D07 桶排序

D08 闪电排序

D09 邻近映射排序

B 一般比较：最坏 $n \log n$ 级保证

C09 自顶向下归并排序

C10 自底向上归并排序

C11 自然归并排序 成对归并轮次

C12 多路归并排序

C18 TimSort 经典段长策略

C19 Powersort 幂次归并排序

C20 Peeksort 中点探查归并排序

C24 自适应 ShiversSort

C17 内省排序

C21 模式破坏快速排序 pdqsort

C22 块快速排序 带内省后备版

C23 稳定块归并排序 WikiSort / GrailSort 家族 [高级构造 / 家族]

C16 堆排序

X14 弱堆排序

X04 平滑排序

X02 平衡搜索树排序

X03 锦标赛排序

X05 耐心排序

X15 笛卡尔树排序

X16 伸展树排序

C 期望表现较好：不保护时最坏较弱

C13 随机化双路快速排序

C26 采用 Lomuto 分区的快速排序

C27 采用 Hoare 分界式分区的快速排序

C14 三路快速排序

C15 双枢轴快速排序

C32 QuickMergesort：借用内部缓冲区的混合排序

X01 非平衡二叉搜索树排序

X06 图书馆排序

D12 采样排序

D 介于 $n \log n$ 与平方级之间

C31 稳定的二分旋转归并排序

C08 希尔排序 Knuth 增量

E 一般为平方级：小输入或特殊成本目标

C01 插入排序

C02 折半插入排序

C03 选择排序

C28 稳定选择排序：移动插入版

C29 双向选择排序

C30 简单互换排序：依次比较所有后续位置

- C04 冒泡排序 提前结束版
- C05 鸡尾酒排序 双向冒泡排序
- C06 侏儒排序
- C07 梳排序
- X07 串排序
- X08 循环排序 比较版本
- C25 Ford Johnson 归并插入排序
- D28 秩排序／枚举排序 (Rank / enumeration sort)

F 额外结构：位移有限与字符串

- D30 偏移不超过 k 的小堆排序
- D10 爆发式字典树排序
- D11 多键快速排序

G 外部存储与分布式数据

- D14 外部多路归并排序
- D15 置换选择 有序段生成 [部件]
- D16 多阶段归并排序
- D17 分布式范围排序／TeraSort [系统]
- D13 带采样的直方图排序
- D25 漏斗排序 惰性缓冲分支 [高级构造／家族]

H 并行执行与排序网络

- D26 AKS 排序网络 高级理论构造 [高级构造／家族]
- D21 采用 Merge Path 的并行归并排序
- D18 双调排序网络
- D19 Batcher 奇偶归并排序
- D22 列排序
- D24 剪切排序
- D20 奇偶换位排序
- D31 IPS4o：原地并行超标量抽样排序 [高级构造／家族]

I 受限移动与物理模型

X10 煎饼排序

X11 焦面煎饼排序

X12 珠排序／重力排序

X13 意大利面排序

J 刻意低效但可定义的过程

X17 Stoooge 排序 三段递归排序

X18 慢速排序

X22 排列穷举排序

X19 Bogo 排序／猴子排序

X20 Bozo 排序 随机异位交换

X21 Bogobogo 排序

K 不是真正排序的名称与相关任务

X23 奇迹排序〔不保证完成真正排序〕

X24 Stalin “排序” 过滤器〔不保证完成真正排序〕

X25 睡眠排序〔不保证完成真正排序〕

X26 量子 Bogo 排序 笑话〔不保证完成真正排序〕

X27 拓扑排序：另一种“排序”任务〔另一类排序任务〕

学习附录

关键伪代码

按任务选择

练习与参考答案

术语表

别名与变体辨析

核验范围与局限

参考来源与延伸阅读

基础：动作与保证

有序还不够

把 [5, 2, 4, 1, 3] 变成 [1, 2, 3, 4, 5]，可以通过两项检查：第一，相邻两个数字都满足前者不大于后者；第二，原来的每个数字都还在，出现次数也完全相同。第二项常用“多重集合不变”来表达。

例如，把 [3, 1, 2, 2] 变成 [1, 2, 3]，虽然结果从小到大，却丢掉了两个 2，因此不算正确排序。排序不是筛选，不应偷偷删除数据或补造数据。

空数组和只有一个元素的数组天然有序。允许重复值的升序，更准确地叫“非递减”，例如 [1, 2, 2, 4]。要改成降序，通常只需改变比较方向。

排序的是记录 比较的是键

一条学生记录可以包含姓名、学号和分数。如果按分数排序，分数就是“键”。交换位置时应搬动整条记录，而不是只移动分数，否则张三的名字可能配上李四的成绩。

如果先按日期、再按姓名排序，就先比较日期；日期相同才比较姓名。比较规则必须前后一致。不能一会儿说甲应该在乙前面，一会儿又反过来；也不能出现甲在乙前、乙在丙前、丙又在甲前的循环。

稳定性到底保留什么

给相同的数字加上身份标签：[2 甲, 1 乙, 2 丙]。稳定排序得到 [1 乙, 2 甲, 2 丙]，两个 2 的先后关系没有改变。若得到 [1 乙, 2 丙, 2 甲]，数值顺序仍然正确，但不稳定。

因此，“不稳定”不等于“排序会出错”。它只说明相等键对应的记录可能换位。若确实需要保留原顺序，可以把“原始位置”加入比较键，但这会增加信息存储或比较成本。

原地和自适应不是同一回事

本手册把严格原地定义为：除输入和输出本身以外，只需要 $O(1)$ 的额外存储。某些算法在原数组上交换，却还使用 $O(\log n)$ 的递归栈；条目会明确写出，避免用一个“是”掩盖实际空间开销。

自适应表示算法能利用输入里已经存在的顺序。这里的“接近有序”可能指逆序对少、自然有序段少，或者每个元素离最终位置都不远。不同算法利用的结构不一样。

学会看效率

先说清楚数的是什么

n 表示记录条数。一次比较是判断两个键谁在前；一次移动是复制或交换记录；申请内存又是另一类成本。比较很长的字符串可能要逐字符查看，而搬动一个大对象也可能比搬动它的地址贵得多。

复杂度关注的是“数据变多后，工作量怎样增长”，不是直接预测几秒钟。常数开销、缓存、编程语言、记录大小和比较函数，都会改变实测速度。因此，不能仅凭两个大 O 符号给同一档算法排出永远正确的先后。

未特别注明时，条目讨论顺序执行、基本操作按常数计费的 RAM 模型。涉及字符检查、磁盘块传输、比较次数、并行总工作量或并行深度时，会另外说明。单位不同的数字不能硬放在一起比。

大 O 是上界 不是精确秒数

$O(f(n))$ 表示一种渐近上界：当 n 足够大时，工作量不超过 $f(n)$ 的某个常数倍。 Ω 表示下界， Θ 表示上下界同阶。一个 $\Theta(n)$ 的算法当然也符合 $O(n^2)$ ，只是后者很不精确，所以不要把宽松上界误认为实际增长率。

$\log_2 n$ 可以理解为“不断对半分，大约分几次才能只剩一个”。1024 对半分 10 次变成 1，所以 $\log_2 1024 = 10$ 。大 O 通常省略对数底数，因为换底只差常数倍。

$n!$ 读作 n 的阶乘，即 $n \times (n-1) \times \dots \times 1$ 。它也是 n 个互不相同元素的排列总数。10! 已经是 3,628,800；20! 是 2,432,902,008,176,640,000。因此，盲目猜测整个正确排列，可能需要极多次尝试。

最好 平均 期望 最坏

最好情况看同样规模中最容易的输入，最坏情况看最困难的输入。平均情况必须有输入分布：是所有不同元素的排列等可能，还是某个实际数据来源？如果分布没说清，“平均”就可能让人误解。

期望时间还可能是针对算法自身的随机选择求平均。例如，固定一份输入，每次独立地随机选择枢轴，快速排序可有 $O(n \log n)$ 的期望工作量；但某次特别倒霉的选择序列仍可能退化到平方级。

“最坏无有限上界”并不是“最慢要花一个非常大的固定数”。随机洗牌可能连续失败任意多次。几乎必然终止也不代表有截止时间。如果一个过程完全不改变无序输入，就根本不会逐步接近答案。

为什么比较排序很难再突破

三个互不相同的数字有六种先后排列。只允许用比较来了解顺序时，算法必须能分辨这六种情况。推广到 n 个不同元素，就要区分 $n!$ 种排列；把比较过程画成决策树，至少有一条路径需要 $\log_2(n!)$ 级别的比较。

由此得到一般比较排序的最坏情况 $\Omega(n \log n)$ 下界；在均匀排列分布下，平均比较次数也有同阶下界。这不是说每一种输入都必须花这么久。它也不限制计数和基数排序，因为后两者直接利用了键的取值或位表示。[F1]

多处理器更快完成也不违反这个结论。并行耗时、总操作数是两种指标。很多人一起做同样多的工作，可以更早结束，但未必节省总工作量。

别把公式里的其他字母扔掉

k 常表示整数范围或计数槽数量； b 表示基数或桶数； d 表示处理多少位； r 表示自然有序段数； p 表示处理器数量。例如 $O(n + k)$ 只有在 k 没有远大于 n 时，才可近似看成对 n 的线性工作。

外部排序还会使用 M 和 B ： M 是内存能容纳的记录数， B 是一个磁盘块能放的记录数。外存模型按整块传输计费。字符串算法还需要考虑长度与公共前缀。各条目如使用专门符号，会就地说明。

后面会用到的工具箱

二分查找只适用于已排序的搜索区间。比如把 4 插入 [1, 3, 6, 8]，查看中间值 6，知道目标在它前面；再看 3，就知道应放在 3 后面的空隙。每次排除大约一半位置。半开区间 $[lo, hi)$ 包含 lo 、不包含 hi ，长度就是 $hi - lo$ 。

栈是“后进先出”：新东西从栈顶放入，也先从栈顶取出。队列是“先进先出”：从队尾加入，从队头取出。优先队列则不按加入先后，而是优先取最小或最大的键；堆可以实现优先队列。

二叉树的节点最多有左右两个孩子。往下是子孙，朝根部走遇到的是祖先。二叉搜索树要求左子树中的键都在当前键之前，右子树中的键都在当前键之后。“中序遍历”就是先读左子树、再读节点、最后读右子树。堆的约束不同，它只要求父子之间满足大小关系。

树旋转用来改变形状而保留搜索顺序。以右旋为例： x 是根， y 是它的左孩子；让 y 升到 x 上面， x 成为 y 的右孩子，再把 y 原来的右子树交给 x 作左子树。左旋则完全对称。这是连接关系的调整，不是重新把所有键排序。

数组中的“稳定旋转”是交换相邻两块的位置，并保留每块内部顺序。例如 $[a, b \mid c, d, e]$ 变成 $[c, d, e \mid a, b]$ 。一种实现先反转左块、再反转右块、最后反转整体。中间过程暂时颠倒顺序，但最终每块内部恢复原先后。

floor 表示向下取整， ceil 表示向上取整。位只有 0 和 1；翻转位就是 0 变 1、1 变 0，与 1 做 XOR 异或可实现翻转。哨兵是为了处理边界而加入的特殊值，如负无穷或正无穷；最终输出真实数据时必须去掉。

有序段熵 H 用来概括总长度怎样分散到各个有序段。对每一段，取它占总记录的比例，再乘以“该比例倒数的以 2 为底对数”，最后求和。只有一段时 $H=0$ ； r 段一样长时 $H=\log_2 r$ 。只有原论文支持相应分析时，条目才会使用这个指标。

把基础结构说清楚

数组是一排有编号的位置。 $A[0]$ 是第一格， $A[1]$ 是第二格。交换是两条记录对调；移动可以把一条记录复制到相邻位置。移动时覆盖一个格子，必须保证被覆盖的原记录已经保存在别处。

有序段是连续且已经排好的区间，例如 $[1, 3, 6, 2, 5]$ 中的 $[1, 3, 6]$ 。逆序对是前后次序错误的一对，例如 $[3, 1, 2]$ 中有 $(3, 1)$ 、 $(3, 2)$ 。不变式就是重复执行每一步后，仍然成立的那条保证。

递归就是先解决更小的同类问题，再使用它的结果。把 $[4, 1, 3, 2]$ 按两半排序，会产生 $[4, 1]$ 、 $[3, 2]$ 两个小任务。零条或一条记录等基本情况不再拆分。递归调用栈用来记住尚未结束的大任务。

分区按规则把记录分到不同区域，例如小于某个枢轴的一侧和大于它的一侧。桶是接收某类记录的容器。分区或入桶，并不一定把区域内部排好。归并则不同：它把原本已经有序的几段合成一段。

最小堆把最小键之一暴露在根部；最大堆把最大键之一暴露在根部。它并没有让整个数组从头到尾有序。下沉就是让不合要求的根向下移动，直到恢复父子之间的大小关系。C16、D30 会展示具体动作。

参数与记号

符号	含义
n	输入记录数量
k 或 R	整数键范围或计数槽数量；k 在局部也可能表示归并路数
b 与 d	基数或桶数，以及处理的位数或趟数
w	按位分析时键的二进制宽度
r	自然有序段数；矩阵算法中也可能专指行数
H	上文解释的有序段熵
M 与 B	以记录数计的内存容量与磁盘块容量
p	处理器数量；Powersort 条目中另指边界幂值
q	局部定义的量，例如网格边长或抽样与桶参数
I	逆序对数量

上标不便显示时会用 ^ 表示乘方： $n^{3/2}$ 就是 n 的 1.5 次方； $\log^2 n$ 是 $(\log n)^2$ 。公式明确写出对数底数时，不应省去。某条目重新定义了字母含义，应以该处说明为准。

怎样读过程图

标记	含义
C	正在比较或检查的键。
M	正在移动的记录或块；具体动作以该步文字为准。
S	只保证区域内部有序，不保证已到全局最终位置。
F	相应位置或已完成的输出，在本次任务中已经确定。

下划线表示尚未写入的输出格，或原记录已暂存到数组外的空位。竖线分隔区域，不是数据。箭头前进到下一项标明的步骤或阶段；一个箭头可能概括多次比较或移动。计数行会特别标明，不把次数冒充原始记录。图结构和矩阵图按各自图注阅读。

有条件的效率导读

先明确问题：多少条记录、什么样的键、多少内存、已有多少顺序、多少处理器，以及计量哪种成本？只有明确这些，“更快”才有意义。下表比较同一种串行单位成本模型中的增长，不是测速结果。
[F1] [F2] [F3]

增长趋势：从低到高	具体意味着什么
n	在规定键条件下，每条记录只访问有限几次，或每个位置只做常数工作。
$n \log n$	大约 $\log n$ 层，每层合计约 n 工作；均衡合并或分区常形成这种结构。
$n \log^2 n$	又多一层对数因子，例如一次合并内部还要反复二分和旋转。
$n^{3/2}$	处在 $n \log n$ 与平方级之间；这里的界对应特定希尔增量。
n^2	每条记录可能扫描或跨越很长一段数组； n 翻倍，工作量可能约变为四倍。
$n^{(\log_{3/2} 3)} \approx n^{2.7095}$	Stooge 排序反复访问彼此重叠的三分之二区间。
$n^{(\Theta(\log n))}$	指数本身还在增大；所述 Slowsort 递推超出固定次多项式，但低于通常的指数增长。
$n! \times \text{多项式因子}$	枚举或反复猜测整个排列，可能情况的数量成为主要负担。

这里排序的是可比较的函数，不是任何条件下的所有算法。宽松上界不一定是精确运行时间。随机搜索的期望成本，与确定性最坏界是不同陈述；随机尝试可能根本没有有限的最坏截止时间。

五种会改变结论的条件

整数范围很小

对 0 到 100 的分数，计数只要扫描 n 条记录并处理 101 个槽。只有两个键、却相差一万亿时，给每个可能值开槽就完全不同。基数排序不用给整个数值范围开槽，但要为每一位处理付费。

已经接近有序

插入可以用接近线性的工作修复少量逆序；自然段归并可以利用几条很长的有序段。“接近有序”必须说明是逆序少、段数少，还是离最终位置近。

比较或写入特别贵

Ford-Johnson 优先减少比较；循环排序优先减少记录写入。它们可能在总处理时间上更慢，却节省了应用最在乎的资源。

数据装不进内存

顺序读写大块数据，生成有序段，再多路合并。反复跳到分散位置读记录，可能比在内存里多比较一些次还贵。必须单独计算输入输出传输。

有很多处理器

深度是必须依次等待的并行轮数；总工作量是所有处理器合起来做了多少。网络深度很短，也可能仍做 $n \log^2 n$ 次比较。硬件、同步和内存带宽还会增加限制。

只有条件与实际任务一致时，测速才有参考意义。本手册提供原理与渐近界，不编造运行秒数。

A 键有结构：有条件的线性方法

这些方法直接查看数值、位或数字，不只靠两两比较。必须保留取值范围、位数和分布等条件。直接放到目标位置的线性结论不是一般比较排序的保证；本组后面的分布方法还可能退化到平方级。

编号只标识条目，不表示名次。

X09 直接寻址循环排序

名称／关系：已知排列的循环放置；直接寻址循环排序

键值若直接代表最终位置，就能沿置换环搬运，无须再比较求排名。

具体怎么做

- 1. 要求键恰好是 $0,1,\dots,n-1$ 的一个排列，或已知等价的常数时间目标位置函数。
- 2. 依次检查位置 s ；如果该记录的键已等于 s ，就跳过。
- 3. 否则暂存记录 x ，把 s 视为一个待填的空位。
- 4. 用 x 的键作为目标 d ，把 x 写到 d ，并暂存被挤出的记录。
- 5. 继续按键寻址，直到 d 等于 s ，完成这一环，再继续向后扫描。

小例子：[2,0,1] 中，2 去位置 2，挤出的 1 去位置 1，最后 0 回位置 0，得到 [0,1,2]。

效率与存储

最好	$\Theta(n)$
平均或期望	$\Theta(n)$
最坏	满足排列前提时 $\Theta(n)$
额外空间	$O(1)$
稳定性	键必须唯一，因此没有同键稳定性问题
严格原地	是

为什么快，为什么慢

值本身就告诉最终下标，每次真正的交换至少让一条归位，因此只需线性交换量。这种直接寻址能力来自“恰为指定排列”的前提。

适合场景：适合把带原始编号的记录复原，或高效执行一个已知置换。

容易误解：任意整数、缺失键或重复目标都不满足前提，可能导致越界或死循环。

依据：[X-CYCLE-PAPER]

D29 位图排序：有界互异整数

名称／关系：位图排序；限定互异整数的位向量排序

为每个可能的数值准备一个二进制位，记录它是否出现。再按数值顺序扫描这些位，就能依次输出结果。这是一种受限的存在性表示，不能默认保留重复记录。

具体做法

- 1. 要求输入是已知区间 min..max 内的互异整数。令 $k = \text{max} - \text{min} + 1$ ，把 k 个二进制位全部清零。
 - 2. 读到 x，就将 $\text{bit}[x - \text{min}]$ 设为 1。一个机器字能打包很多位，但直观上只是给“这个数存在”做标记。
 - 3. 从偏移量 0 扫到 k-1，每遇到一个 1，就输出 min+偏移量。
- 跟着做：对 [5,1,4,0]，取范围 0..5。初始位图 [0,0,0,0,0,0]；标记 5 后为 [0,0,0,0,0,1]；标记 1 后为 [0,1,0,0,0,1]；标记 4 后为 [0,1,0,0,1,1]；标记 0 后为 [1,1,0,0,1,1]。
- 跟着做：从左往右读取有标记的位置，输出 [0,1,4,5]。但 [1,1] 只会设置同一个位，输出一个 1。需要保留重复次数时，必须把位改成计数器，转为计数排序。

效率与存储

时间	逐位扫描的简单版本为 $\Theta(n+k)$
额外空间	k 位，加上需要实际存储的输出；机器字打包主要影响常数
稳定性	互异裸数值无需讨论；附带记录需要其他存储结构
严格原地	否

为什么快，为什么慢

- 每个输入值做一次标记，再扫描一次可能值范围即可。位打包省内存，却不能消除对整个取值范围大小的依赖。
- 适合：取值范围紧凑的互异编号。一个机器字容纳多个标记，通常比完整计数器省空间。
- 注意：只有在互异值前提下，它才完成了排序。对一般输入偷偷去重，是另一项任务；取值范围巨大且稀疏时，位图也可能很浪费。
- 来源：[N09]

D01 计数排序

名称／关系：键索引计数；分布计数排序

统计每个整数键的次数，把次数转换成位置，再把记录放进对应的最终区间。

具体怎么做

- 1. 找出最小、最大整数键，令 $R = \text{最大值} - \text{最小值} + 1$ ；建立 R 个零计数器和长度为 n 的输出数组。
- 2. 逐个读取记录，把“键-最小值”对应的计数器加一。
- 3. 累加前面各键的次数，得到每个键的输出起点： $\text{start}[0]=0$ ， $\text{start}[v]=\text{start}[v-1]+\text{count}[v-1]$ ；复制一份起点表，用于记录下一可写位置。
- 4. 从左到右扫描输入，把完整记录写到该键的下一输出位置，再把该位置加一。最小键不是零时，用 $\text{key}-\text{minimum}$ 作为表下标。
- 5. 如有需要，把输出复制回原数组；键之外的其他字段始终跟随记录移动。

完整小例子：输入 [2 甲,0,2 乙,1]，键 0、1、2 的出现次数为 [1,1,2]。把前面各键的次数累加，得到每种键的起始位置 [0,1,2]：0 从下标 0 开始，1 从下标 1 开始，2 从下标 2 开始。

按输入从左到右放置：2 甲 写到下标 2，并把键 2 的下一可用位置改为 3；0 写到下标 0；2 乙 写到下标 3；1 写到下标 1。结果为 [0,1,2 甲,2 乙]。相等记录依次占用同一段中的下一个空位，所以原始先后没有改变。

效率与存储

最好	$\Theta(n+R)$
平均或期望	$\Theta(n+R)$ ，无须假设数据均匀
最坏	$\Theta(n+R)$ ； R 很大时并非 $\Theta(n)$
额外空间	辅助空间 $\Theta(n+R)$
稳定性	稳定，从左到右依次占用同键区间的空位
严格原地	该稳定记录版本不是原地排序

为什么快，为什么慢

键可以直接指向计数器和输出区域，不必反复比较任意两个键。即使记录很少，初始化与扫描整个键范围也要花时间，这正是 $+k$ 的来源。

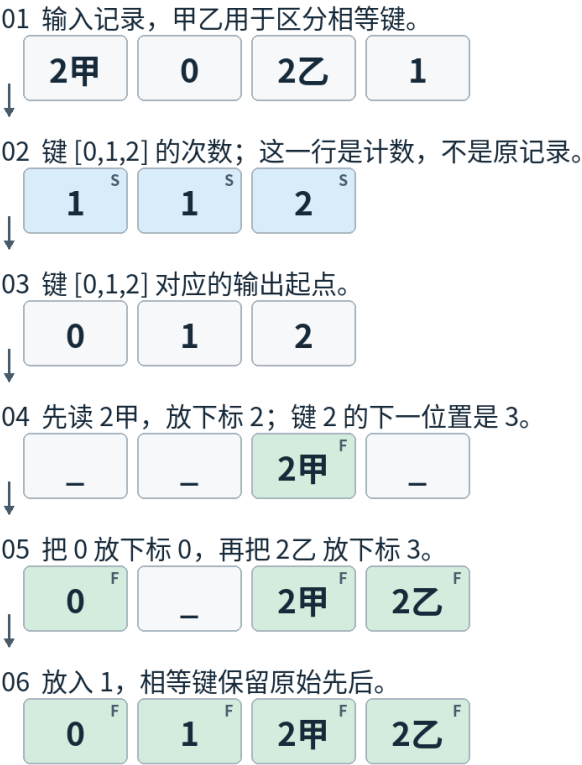
适合场景：适用于小范围整数、成绩、字节值，以及基数排序中的稳定单趟排序。

容易误解：键值范围稀疏时会浪费大量内存；只根据次数重建数字会丢失记录的其他字段。

依据：[DIST01]

过程图 D01

计数：用出现次数确定稳定输出位置



C 正在比较 M 正在移动 S 区域内有序 F 最终归位

D01 计数：用出现次数确定稳定输出位置。与本条目的示例对应；标记含义见“基础”。

D02 鸽巢排序

为每个可能的键建立一个容器，放入完整记录，再按键的顺序连接容器。

具体怎么做

- 1. 找出整数键的最小值和最大值，计算包含两端的范围 R。
- 2. 建立 R 个空列表，第 i 个列表代表“最小值+i”。
- 3. 按原顺序读取记录，把整条记录追加到对应列表末尾。
- 4. 从最小键的列表走到最大键的列表；每个列表均从头到尾输出。
- 5. 检查输出仍有 n 条记录；空列表直接跳过。

小例子：[3a,1,3b,2] 分入 1:[1]、2:[2]、3:[3a,3b]，连接后为 [1,2,3a,3b]。

效率与存储

最好	$\Theta(n+R)$
平均或期望	$\Theta(n+R)$

最坏	$\Theta(n+R)$ ，取决于键值范围
额外空间	列表桶需要 $\Theta(n+R)$
稳定性	按到达顺序追加和读取时稳定
严格原地	上述版本不是原地排序

为什么快，为什么慢

每条记录进入已知的巢一次，再离开一次；但空巢也要访问，范围巨大而稀疏时，反而可能不如比较排序。

适合场景：适用于整数范围与记录数量接近、需要保留完整记录的场景。

容易误解：此处使用真正的逐键容器；只累计次数的“鸽巢排序”实质上是计数排序，不应重复计数。

依据：[DIST02]

D03 最低有效位优先基数排序

名称／关系：最低有效位优先基数排序；LSD 基数排序

从最低位开始，逐位稳定排序，让后续排序保留此前已经排好的低位顺序。

具体怎么做

- 1. 选择基数 b 和固定的 d 位键；初学时先用非负整数，不足位数在左侧补零。
- 2. 从最右一位开始，用稳定的计数排序把所有记录分到 b 个桶。
- 3. 按数字大小收集各桶；同一数字的记录必须保留上一趟的相对顺序。
- 4. 向左移动一位，再对整个数组执行相同的稳定排序。
- 5. 完成 d 趟后直接读取数组；有符号数需要先采用保持数值顺序的编码。

小例子：[21,13,12] → 按个位 [21,12,13] → 按十位 [12,13,21]。

效率与存储

最好	固定执行 d 趟时为 $\Theta(d(n+b))$
平均或期望	$\Theta(d(n+b))$
最坏	$\Theta(d(n+b))$ ，假设取一位是 $O(1)$
额外空间	重复使用输出缓冲区，空间 $\Theta(n+b)$
稳定性	每趟数字排序稳定时，整体稳定
严格原地	使用计数缓冲区的版本不是原地排序

为什么快，为什么慢

每轮稳定处理多确定一位顺序，共需 d 轮，每轮扫描记录与数字桶。位数很长或桶的字符集很大时，都不能把这部分当成免费。

适合场景：适用于定长整数、定长字符串和数值键紧凑的记录。

容易误解：单趟不稳定会破坏之前的结果；变长文本和浮点数位模式都需要明确的排序规则。

依据：[DIST03]

过程图 D03

LSD 基数排序：稳定轮次保留先前低位顺序



C 正在比较 M 正在移动 S 区域内有序 F 最终归位

D03 LSD 基数排序：稳定轮次保留先前低位顺序。与本条目的示例对应；标记含义见“基础”。

D04 最高有效位优先基数排序

名称／关系：最高有效位优先基数排序；MSD 基数排序

先按第一位分组，再在每组中按下一位继续排序。

具体怎么做

- 1. 使用大小为 b 的有序字符集，并设置一个小于所有真实字符的字符串结束标记。
- 2. 在当前位置 d 统计当前子数组中的字符，并计算桶的边界。
- 3. 把完整字符串的引用稳定分配到辅助数组，再复制回这个子区间。
- 4. 对于至少含两个字符串、且不是结束标记的桶，在 $d+1$ 位置递归处理。
- 5. 单个字符串或结束标记即可停止；小桶若改用插入排序，只需比较剩余后缀。

完整小例子：用定长两位数 [21,13,12]。先检查十位：1 号桶收到 [13,12]，2 号桶收到 [21]。此时已经知道，第一桶的每个数都小于第二桶的每个数。

接下来只在 [13,12] 内检查个位：12 放入个位 2 号桶，13 放入 3 号桶，得到 [12,13]；单独的 [21] 不需再排。依桶序连接，得到 [12,13,21]。与 LSD 不同，后续低位排序只在先前划定的前缀组内部进行。

字符串 [bat,ant,bar] 可先分成 a:[ant] 与 b:[bat,bar]。b 组第二个字符同为 a，要继续看第三个字符，r 在 t 前，于是得到 [ant,bar,bat]。字符串结束标记应排在其任何后续字符之前。

效率与存储

最好	第一位已区分所有键时为 $\Theta(n+b)$
平均或期望	取决于共同前缀；此计数实现为 $\Theta(C+bq)$
最坏	固定 b 时 $O(nw)$ ； $C \leq nw$ ，w 为最大长度
额外空间	递归保留各层桶计数时 $O(n+bw)$
稳定性	分配和小区间排序均稳定时稳定
严格原地	不是，需要辅助记录数组

为什么快，为什么慢

高位一旦分开，各组便永久有序地隔离，后续只需处理组内；公共前缀越长，真正分开的时间越晚，要检查的位也越多。

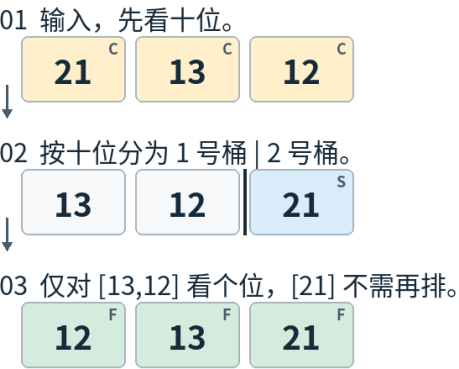
适合场景：适用于变长字符串，尤其是短前缀即可区分大量键时；C 表示检查字符数，q 表示处理子数组数。

容易误解：不能越过结束标记继续递归；即使 n 很小，长公共前缀也会带来许多趟处理。

依据：[DIST04]

过程图 D04

MSD 基数排序：低位处理只在前缀组内进行



C 正在比较 M 正在移动 S 区域内有序 F 最终归位

D04 MSD 基数排序：低位处理只在前缀组内进行。与本条目的示例对应；标记含义见“基础”。

D05 美国旗排序

通过循环置换把记录送入桶区间，实现无须 n 条记录输出数组的 MSD 分配。

具体怎么做

- 1. 在当前位统计各桶数量，用前缀和计算固定的开始与结束位置。
 - 2. 每个桶保存一个“下一个空位”指针，初始指向桶的开头。
 - 3. 扫描桶中尚未完成的位置；若记录属于别的桶，就与目标桶下一个空位交换，并推进目标指针。
 - 4. 继续处理被换出的记录，直到当前位置放入正确记录，再推进本桶指针。
 - 5. 所有桶填满后，对含多个元素的桶按下一位继续处理；结束标记桶已完成。
- 完整小例子：对 [21,12,20]，十位计数为 1 号桶一条、2 号桶两条。它们在数组中的区域分别是 [0,1)、[1,3)，下一可写位置分别为 0、1。下标 0 的 21 属于 2 号桶，于是与该桶下一空位下标 1 交换，得到 [12,21,20]，并将 2 号桶指针推进到 2。

下标 0 现在是正确的 1 号桶记录，该区域完成。下标 1、2 的十位都为 2，继续只对 [21,20] 的个位分区：个位 1 应排在 0 后，交换得 [12,20,21]。这种循环交换不像稳定计数那样保留相等记录的先后。

效率与存储

最好	只需一位时 $\Theta(n+b)$
平均或期望	$\Theta(C+bq)$ ：C 为检查数字位次数，q 为处理子数组数，取决于前缀
最坏	固定基数 b 时 $O(nw)$
额外空间	栈与桶信息保守界 $O(bw)$ ，无 n 条记录缓冲区
稳定性	不稳定，循环置换可能改变相等键的顺序
严格原地	记录原地置换，但辅助空间并非严格 $O(1)$

为什么快，为什么慢

目的区域边界允许循环交换在原数组中完成分配，省下一份输出数组；计数和递归检查各位的时间成本仍然存在。

适合场景：适用于不希望建立第二份完整记录数组的大型整数或字符串排序。

容易误解：原始桶边界与移动指针不能混淆；“原地基数排序”仍然需要计数器和遍历状态。

依据：[DIST05] [DIST06]

D06 二进制 MSD 交换排序

名称／关系：基数交换排序；二进制快速排序

用双指针按一个二进制位划分，再递归处理下一位。

具体怎么做

- 1. 把键表示为无符号 w 位数；有符号键先转换为保持大小关系的无符号表示。

- 2. 从最高位开始，左指针越过该位为 0 的键，右指针越过该位为 1 的键。
 - 3. 两指针未交叉且都停下时，交换这两个放错位置的记录，再继续扫描。
 - 4. 划分后左区间该位为 0、右区间该位为 1，分别处理下一位。
 - 5. 区间少于两个记录，或 w 位全部处理完毕时停止。
- 小例子：[101,001,110,010] 先分为 [001,010 | 101,110]，再细分得到有序结果。

效率与存储

最好	一位键为 $\Theta(n)$ ，一般取决于区分键所需深度
平均或期望	上界 $O(nw)$ ；独立宽随机键通常约需 $n \log n$ 次位检查
最坏	$\Theta(nw)$ ，例如没有全相等捷径时的相同键
额外空间	递归栈 $O(w)$
稳定性	不稳定
严格原地	记录原地存储，另需 $O(w)$ 栈

为什么快，为什么慢

每次只按一个二进制位分成两组，省下大桶表；但键很宽、前导位大量相同时，就会需要更多轮分区。

适合场景：适合作为低缓冲区基数方法，也有助于理解快速排序划分与数字排序的联系。

容易误解：即使一个分区为空，也必须推进到下一位，否则共同前缀可能导致无限递归。

依据：[DIST07]

D27 字典树排序 (Trie sorting)

名称／关系：Trie 排序；前缀树排序；字典树排序

字典树的每条边代表一位数字或一个字符，相同前缀共用一段路径。按照分支从小到大的顺序读取，就能依次得到有序记录。

具体做法

- 1. 对定长非负整数，把所有键写成相同位数，不足时在前面补零。从根出发，按每一位数字走到对应分支；分支不存在就建立。
- 2. 一个键读完后，把整条记录追加到终点的先进先出列表。相同键会到达同一终点，但不能用后一条覆盖前一条。
- 3. 按 0 到 9 的顺序遍历各分支，到终点就按到达顺序输出列表。若处理变长字符串，应先输出当前节点的终止记录，再访问孩子，使前缀排在它的扩展字符串之前。

跟着做：依次插入 [21 甲,13,12,21 乙]。21 甲 建立“根→2→1”；13 建立“根→1→3”；12 共用第一位 1，再建立第二位 2；21 乙 追加到 21 甲 所在的终点列表。

跟着做：输出时先走根的 1 分支，再走 2 分支。在 1 下面先读 2 再读 3，因此先输出 12、13；在 2 下面的 1 终点，依次输出 21 甲、21 乙。结果为 [12,13,21 甲,21 乙]，不必让所有完整数字两两比较。

效率与存储

时间	固定孩子数组实现为 $O(L+bV)$ ， L 为读入总位数， V 为节点数， b 为字符集大小
指标	字符集固定且较小时为 $O(L+n)$ ；每键 d 位时可写 $O(nd)$
额外空间	本简单表示为 $O(bV+n)$
稳定性	终点用先进先出列表时稳定；严格原地：否

为什么快，为什么慢

每一位加入共享路径，最后有序遍历一次输出。省掉的整键比较，换成了节点存储与访问孩子槽位的成本。

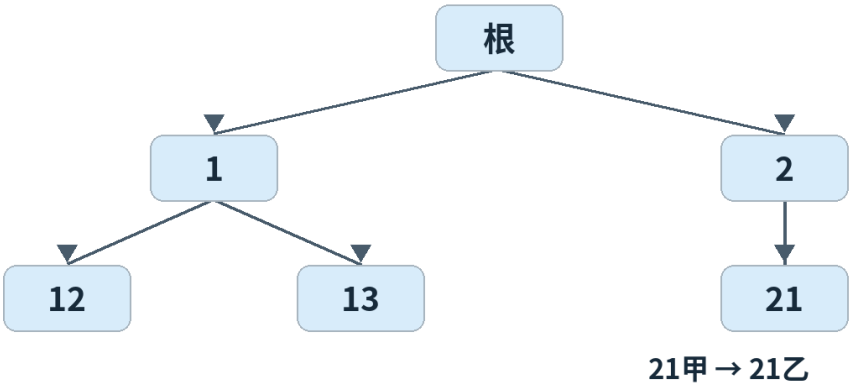
适合：键有大量公共前缀，或者还要做前缀查询时。稀疏孩子结构省空间，但必须保证输出时按键序访问孩子。

注意：不补齐位数的十进制文本按字典顺序排时，“10”会排在“2”前面，并非数值大小顺序。负数还需要保持大小关系的编码或单独处理。

来源：[N07]

过程图 D27

字典树共用前缀，按分支顺序输出



输出：[12,13,21甲,21乙]

同键终点列表保留到达先后。

D27 字典树与前缀路径。与本条目的示例对应；标记含义见“基础”。

D23 Spreadsort 混合分布排序

名称／关系：Boost.Sort 的 spreadsort 家族

根据当前键值范围调整基数划分大小，当继续按位划分不划算时改用比较排序。

具体怎么做

1. 整数版本先扫描当前区间，找出最小值、最大值，并检查是否已有序。
 2. 记录数量少时改用比较排序；否则选择有限数量的有效位进行下一次划分。
 3. 统计各有序桶内的记录数，并计算桶边界。
 4. 利用循环置换，在原数组中把记录送入对应桶，同时保留桶信息。
 5. 对每个未完成的桶，再选择基数划分或有复杂度保证的比较排序；所有桶有序或键全相等时停止。
- 小例子：[257,3,256,2] 可按高字节分为 [3,2 | 257,256]，再对小桶比较排序得到 [2,3,256,257]。

效率与存储

最好	已有序检查成功时 $\Theta(n)$
平均或期望	依赖键宽、范围及实现，不能把实测加速推广到所有分布
最坏	有保证回退的有界基数整数版本，保守界 $O(n \min(\log n, w))$
额外空间	每层桶信息有界；桶数上限固定时 $O(w)$
稳定性	不稳定
严格原地	记录原地移动，仍需计数器和递归信息

为什么快，为什么慢

一次检查多个键位，有利于迅速分开分布较散的数值；小范围或集中数据由比较后备处理，上界必须结合所用键宽模型理解。

适合场景：适用于可提取数字键的生产级混合排序；Boost 提供整数、浮点数及字符串专用实现。

容易误解：取位函数与比较函数必须定义同一顺序；不能把整数复杂度直接套到任意长度字符串。

依据：[DIST36] [DIST37]

D07 桶排序

名称／关系：桶分布排序；是分布排序中的一个家族

把数值分配到有序区间，在区间内排序，然后依次连接。

具体怎么做

1. 对于 [0,1) 内的数，选择 m 个桶，把 x 放入 $\text{floor}(m \times x)$ 号桶；其他范围先定义保持顺序的映射。

- 2. 建立空桶列表，并按输入顺序追加记录。
 - 3. 每个桶使用插入排序，只移动键严格大于当前键的记录。
 - 4. 按桶编号递增的顺序连接所有桶。
 - 5. 实现映射前，检查最大端点、负值和空输入等边界情况。
- 小例子：m=4 时，[.72,.13,.68,.21] 分到 0 号桶 [.13,.21] 和 2 号桶 [.72,.68]，排序连接后得到结果。

效率与存储

最好	$\Theta(n+m)$
平均或期望	独立均匀输入且 $m=\Theta(n)$ 时期望 $\Theta(n)$
最坏	桶内用插入排序时最坏 $\Theta(n^2+m)$
额外空间	$\Theta(n+m)$
稳定性	按顺序追加且桶内稳定排序时稳定
严格原地	列表桶版本不是原地排序

为什么快，为什么慢

桶小且分布均匀时，局部排序很便宜；若全部记录进入同一个采用插入排序的桶，快速分配之后仍留下一个平方级大问题。

适合场景：适用于分布已知且较分散的数值；一般情况下，桶内工作量取决于各桶大小平方之和。

容易误解：建立桶并不会自动使分布均匀；“插值排序”有时指这一类方法，此处不重复计算。

依据：[DIST08]

D08 闪电排序

名称／关系：Neubert 闪电排序

根据数值范围估计有序类别，把记录置换到对应类别，再完成局部排序。

具体怎么做

- 1. 找出最小值和最大值；两者相等时所有键都相同，可以停止。
- 2. 选择 m 类，计算 $class(x)=\text{floor}((m-1)(x-\text{最小值})/(\text{最大值}-\text{最小值}))$ ，并统计各类数量。
- 3. 对数量求前缀和，得到各类边界和下一个可用位置。
- 4. 采用循环置换把每条记录送入对应类别；每放入一条就更新位置。
- 5. 最后对每类内部进行插入排序；分到正确类别并不意味着类内已经有序。

小例子：[8,2,7,3] 使用 m=3，分为 [2,3 | 7 | 8]，类内收尾后得到 [2,3,7,8]。

效率与存储

最好	$\Theta(n+m)$ ；全相等检查可为 $\Theta(n)$
平均或期望	均匀数值输入且 $m=\Theta(n)$ 时期望 $\Theta(n)$
最坏	使用插入排序收尾时 $O(n^2+m)$
额外空间	类别信息 $\Theta(m)$ ， m 通常与 n 成正比
稳定性	不稳定
严格原地	置换 ([DIST09]) ([DIST10]) 地进行，但 m 增长时不是 $O(1)$ 空间

为什么快，为什么慢

数值到类别的映射先把记录送到接近最终的位置；若类别划分与实际分布不匹配，仍会留下长而乱的组，最终清理就会主导耗时。

适合场景：适用于类别估计能让局部组较小的数值分布排序。

容易误解：线性时间的分配阶段不保证整个排序线性；必须注意偏斜分布、溢出和舍入误差。

依据： ([DIST09]) ([DIST10])

D09 邻近映射排序

名称／关系：ProxmapSort；邻近映射排序

先计算粗粒度键类别的准确目标区间，再把记录插入该区间的有序位置。

具体怎么做

- 1. 选择把键映射到 m 类的单调函数；较前类别中的键不能大于较后类别中的键。
- 2. 统计映射到各类的记录数量。
- 3. 计算计数的前缀和，确定各类在新 n 元素数组中的起点和预留容量。
- 4. 按输入顺序读取记录；在相应预留区间内右移较大记录，再插入新记录。
- 5. 从头到尾读取目标数组，即相当于连接所有预留区间。

小例子：按 $\text{floor}(x)$ 映射：[2.8,1.3,2.1] 为 1 类预留一格、2 类预留两格，得到 [1.3,2.1,2.8]。

效率与存储

最好	插入工作量线性时 $\Theta(n+m)$
平均或期望	$m=O(n)$ 且桶大小平方和的期望为 $O(n)$ 时 $\Theta(n)$
最坏	$\Theta(n^2+m)$

额外空间	$\Theta(n+m)$
稳定性	按上述仅移动严格较大键的规则时稳定
严格原地	使用目标数组的版本不是原地排序

为什么快，为什么慢

命中计数为每桶预留精确输出区间，真正的收益来自较短的局部插入范围；若映射区域很拥挤，移动量仍会回到平方级。

适合场景：适用于已知数值分布；保留邻近映射还能缩小后续查找范围。

容易误解：打乱大小顺序的普通哈希函数不能用于此处；平均性能良好需要桶足够小。

依据：[DIST11] [DIST12]

B 一般比较：最坏 $n \log n$ 级保证

这里明确指定的实现具有 $O(n \log n)$ 的最坏串行工作量保证；部分分析对整次排序做摊还计费。同一档内部采用便于理解的顺序，不冒充实测快慢。原有顺序、临时内存、比较成本和缓存行为都会改变实际结果。

编号只标识条目，不表示名次。

c09 自顶向下归并排序

名称／关系：递归归并排序；自顶向下归并排序

递归排好左右两半，再不断取两个队首中较小的元素；相等时先取左边，便能保持稳定性。

具体怎么做

- 1. 排序左闭右开区间 $[lo, hi)$ 。不足两个元素就返回，否则取 $mid = lo + \text{floor}((hi - lo) / 2)$ 。
- 2. 分别递归排好 $[lo, mid)$ 和 $[mid, hi)$ ，两个递归都结束后才归并。
- 3. 把当前区间复制到辅助数组。左右各设一个指针，分别指向各自第一项，输出指针从 lo 开始。
- 4. 比较两个尚未取出的队首，把较小者写回原数组，并只移动它所属的指针；相等时取左侧。
- 5. 一侧耗尽后，复制另一侧的剩余元素。后续归并复用同一个辅助数组，不必每次重新分配。
- 6. 基础版每层都归并。可额外检查 $A[mid - 1] \leq A[mid]$ ，成立时跳过归并，使已排序输入达到线性时间。

小例子： $[4, 1, 3, 2] \rightarrow [4, 1] \mid [3, 2] \rightarrow [1, 4] \mid [2, 3] \rightarrow$ 依次取 1、2、3、4 $\rightarrow [1, 2, 3, 4]$ 。

效率与存储

最好	$\Theta(n \log n)$ ，每层均归并的基础版
平均或期望	$\Theta(n \log n)$
最坏	$\Theta(n \log n)$
额外空间	$\Theta(n)$ 缓冲区，加 $O(\log n)$ 递归栈
稳定性	稳定，相等时先取左侧
严格原地	否，此处为缓冲数组版

为什么快，为什么慢

同一归并层中，每条记录只需读写常数次；反复对半拆分或将段长翻倍，大约产生 $\log_2 n$ 层，因此总量是 n 乘以 $\log n$ 。

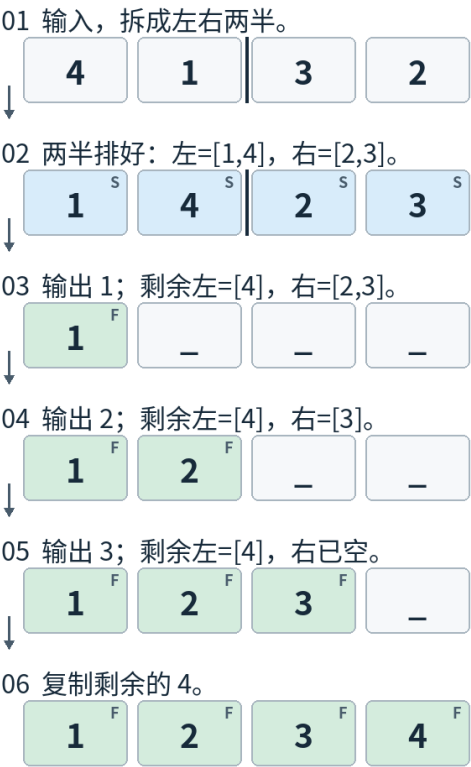
适合场景：适合要求时间可预测和稳定多关键字排序的场景，也是分治法的典型例子；链表版可重连节点，空间分析不同。

容易误解：若直接覆盖尚未读取的数据，会丢失元素；还要区分基础版与跳过归并优化版的最好情况。

依据：[C09] [C11]

过程图 C09

归并：每次只输出两段段首中较小的记录



C 正在比较 M 正在移动 S 区域内有序 F 最终归位

C09 归并：每次只输出两段段首中较小的记录。与本条目的示例对应；标记含义见“基础”。

c10 自底向上归并排序

名称／关系：迭代归并排序；自底向上归并排序

从单元素有序段开始，每轮合并相邻的等目标长度有序段；归并动作与自顶向下版本相同。

具体怎么做

- 1. 准备一个可复用的输出缓冲区，令 width=1；单个元素天然有序。
- 2. 起点依次取 0、2×width、4×width……，每次配对两个相邻的 width 长区间，末端超过 n 时截断。
- 3. 用两个队首指针归并每一对，相等时先取左边。若整轮交替使用源数组和目标数组，末尾孤立段也要原样复制过去。
- 4. 一轮完成后，各段长度至多为 2×width 且内部有序。把 width 加倍，开始下一轮。

5. $\text{width} \geq n$ 时结束；若最终结果在辅助数组中，按接口要求复制或交换回原数组。整个过程无需递归。

小例子：[4,1,3,2,5] → 宽度 1：[1,4|2,3|5] → 宽度 2：[1,2,3,4|5] → 宽度 4：[1,2,3,4,5]。

效率与存储

最好	$\Theta(n \log n)$ ，每轮均归并的基础版
平均或期望	$\Theta(n \log n)$
最坏	$\Theta(n \log n)$
额外空间	$\Theta(n)$ 数组缓冲， $O(1)$ 控制变量
稳定性	稳定，相等时先取左侧
严格原地	否，此处为缓冲数组版

为什么快，为什么慢

同一归并层中，每条记录只需读写常数次；反复对半拆分或将段长翻倍，大约产生 $\log_2 n$ 层，因此总量是 n 乘以 $\log n$ 。

适合场景：适合偏好迭代控制、避免递归，或希望按整轮顺序处理数据的场景。

容易误解：右段可能不足 width ，甚至不存在；数组长度不必是 2 的幂，两个端点都必须正确截断。

依据：[C10]

C11 自然归并排序 成对归并轮次

名称／关系：自然归并排序；本文指定成对轮次策略

把输入中原本有序连续片段直接当作初始有序段，再按轮次成对归并，让段数逐轮大致减半。

具体怎么做

1. 从左向右识别极大的非降连续段；也可识别严格下降段，并将其反转。
2. 若扫描后只有一段覆盖整个数组，说明已经有序，可直接结束。
3. 稳定归并第 1、2 段，再归并第 3、4 段，依次继续；若末尾剩单独一段，保留到下一轮。
4. 用归并后的区间替换每对原有区间，重复成对归并，直到只剩一段。
5. 这里必须按轮次配对；若总是把下一小段追加到不断变长的前缀，元素搬移可能退化为平方级。

小例子：[1,4|2,5|3,6] → [1,2,4,5|3,6] → [1,2,3,4,5,6]；三个初始有序段只需两轮归并。

效率与存储

最好	$\Theta(n)$ ，只有一个自然有序段
----	------------------------

平均或期望	$\Theta(n \log n)$ ，随机互异键排列
最坏	$O(n(1+\log r))$ ， r 为初始段数；总体最坏 $\Theta(n \log n)$
额外空间	$O(n)$ 缓冲，加至多 $O(r)$ 段描述信息
稳定性	稳定，采用稳定归并且只反转严格下降段
严格原地	否，此处为缓冲数组版

为什么快，为什么慢

已有的有序段直接拿来用，不再拆成单个元素。每轮大约把段数减半，初始段越少，需要的归并轮次越少。

适合场景：适合由多个已排序批次拼接的数据、有少数断点的有序导出数据，也便于理解 TimSort 与 Powersort。

容易误解：把含相等键的非升段直接反转，会颠倒相等元素身份；简单实现只能反转严格下降段。

依据：[C11] [C20]

C12 多路归并排序

名称／关系： k 路归并排序；多路归并排序

先排好 k 份数据，再用最小堆维护各段当前队首，每次快速选出下一项，避免反复扫描所有有序段。

具体怎么做

1. 选定路数 $k \geq 2$ ，把输入分成 k 个大致等长的连续区间，递归排好；空区间和单元素区间直接结束。
2. 归并时，把每个非空有序段的第一项加入最小堆，并同时保存段编号与当前位置。
3. 弹出堆中最小项，追加到输出缓冲区；只推进这一项来自的有序段。
4. 若该段还有下一项，就把它放入堆。重复到堆空，再把缓冲结果作为当前区间的排序结果。
5. 要保持稳定，堆的比较先看键，再按原始段顺序处理相等键；各段内部按原顺序取出。也可用败者树选择队首最小项。

完整小例子：把 $[7,1,4,2,6,3]$ 分为 $[7,1]$ 、 $[4,2]$ 、 $[6,3]$ ，先递归排好各段，得到 $[1,7]$ 、 $[2,4]$ 、 $[3,6]$ 。把三个段首 1、2、3 放进最小堆，并记下各自来自哪一段。

取出 1，只从它所在段补入 7，候选变成 $\{2,3,7\}$ ；取出 2，补入 4，变成 $\{3,4,7\}$ ；取出 3，补入 6，变成 $\{4,6,7\}$ 。随后依次取出 4、6、7，对应段读完就退出。输出为 $[1,2,3,4,6,7]$ 。每次只有刚刚被取走段首的那一段向前推进。

效率与存储

最好	$\Theta(n \log n)$ ，固定 k 、平衡递归且每层完整处理
----	---

平均或期望	$\Theta(n \log n)$, k 固定
最坏	$\Theta(n \log n)$, k 固定; 单次 k 路归并为 $O(n \log k)$
额外空间	$O(n)$ 输出缓冲, 加当前归并的 $O(k)$ 堆
稳定性	使用原段顺序打破平局时稳定
严格原地	否, 此处为缓冲数组版

为什么快，为什么慢

堆避免每输出一条都完整扫描 k 个段首; 更新获胜那一路只花 $\log k$, 而一次合并更多路, 又能减少外层归并层数。

适合场景: 适合减少整轮归并次数、合并多个有序流, 也是外部排序的核心归并步骤; 磁盘 I/O 需另建成本模型。

容易误解: 合并 k 个已排序段, 不等于直接排序 n 个任意元素; 每次线性扫描 k 个队首会花 $O(nk)$, 而非 $O(n \log k)$ 。

依据: [C12]

C18 TimSort 经典段长策略

名称/关系: Tim sort; 本文为修正后的经典段长调度

结合自然有序段、折半插入和稳定归并, 用有序段长度栈决定归并次序; 这里采用修正后的经典 CPython 3.10 策略。

具体怎么做

1. 计算 minrun: 令整数 $m=n$, 当 $m \geq 64$ 时不断减半, 并记录被移除的二进制位中是否出现过 1; 最后取 m 加这个标记。
2. 扫描下一个非降段或严格下降段, 后者先反转。若长度不足 minrun, 用稳定折半插入向后扩展, 但不超过数组末尾。
3. 把段起点与长度压栈。栈顶最多四段从旧到新记作 W 、 X 、 Y 、 Z ; 不存在的段不参与条件判断。
4. 若 $X \leq Y+Z$ 或 $W \leq X+Y$, 且 $X < Z$, 则归并 X 、 Y , 否则归并 Y 、 Z ; 若前两个条件均不满足但 $Y \leq Z$, 也归并 Y 、 Z 。每次归并后重新检查, 否则继续读取新段。
5. 复制较短的一段作为临时缓冲, 进行稳定归并。某侧连续胜出时, 可用倍增探测加二分查找找到批量复制边界, 这叫疾驰模式; 相等键的边界处理必须保持原顺序。
6. 读完输入后继续收拢栈: 通常归并最后两段; 若倒数第三段比最后一段短, 就改归并倒数第三与倒数第二段, 直到仅剩一段。

完整小例子：对真实的小输入 [5,2,4,1,3]，经典 minrun 的计算结果是 5。先发现递减段 [5,2]，反转成 [2,5]；再用折半插入把短段补长：插入 4，得到 [2,4,5 | 1,3]；插入 1，得到 [1,2,4,5 | 3]；最后插入 3，成为 [1,2,3,4,5]。整份数据已是一条段，不需要归并。

要观察段长策略，可看已识别的三段长度 [80,50,40]。80>50+40 不成立，于是先合并相邻的 50 与 40，成为 [80,90]，再合成 170。相同的数据搬动模式可用小数字说明：[1,7 | 2,6 | 3,4,5] 先变成 [1,7 | 2,3,4,5,6]，再合成 [1,2,3,4,5,6,7]。这里的小段只演示两次合并，不冒充这些长度会触发上述经典策略。

效率与存储

最好	$\Theta(n)$ ，识别到单一有序段
平均或期望	$\Theta(n \log n)$ ，互异键均匀随机排列
最坏	$\Theta(n \log n)$ ，正确修正的策略与实现
额外空间	$O(n)$ 归并缓冲，加 $O(\log n)$ 段栈
稳定性	稳定
严格原地	按辅助空间定义并非原地，但可修改输入本身

为什么快，为什么慢

识别自然段利用已有顺序，小段插入减少调度开销，连续获胜时可批量搬动。归并策略则避免长期采用严重失衡的合并顺序。

适合场景：适合带有原有顺序的现实记录数据的稳定排序。本文描述的是修正后的经典 CPython 3.10 策略。编写时查阅的 CPython 主分支改用 Powersort 调度，minrun 的工程细节也已有变化，因此不能把本条目当作新版代码的逐行说明。

容易误解：只检查栈顶三段并不等于修正后的完整策略；接口所谓“原地修改”，也可能分配线性大小的归并缓冲。

依据：[C18] [C19] [C30]

C19 Powersort 幂次归并排序

名称／关系：Powersort 归并调度策略

为相邻自然有序段的边界计算一个整数“幂次”，先完成较深边界对应的归并，从而获得近最优的归并次序。

具体怎么做

1. 先读出一个有序段 R，再读相邻的下一段 S。识别方式同自然归并；严格下降段可安全反转。

- 2. 设两段的零基起点为 s 、 t ，长度为 a 、 b ，总长为 n 。计算中心 $x=(s+a/2)/n$ 、 $y=(t+b/2)/n$ ；最小满足 $\text{floor}(2^p x)=\text{floor}(2^p y)$ 的正整数 p 就是边界幂次。
- 3. 若栈顶已保存边界的幂次大于 p ，就弹出其左段，与当前 R 稳定归并。连续合并时，刚算出的 p 保持不变。
- 4. 把合并后的 R 连同 p 压栈，再令当前段 $R=S$ 。继续读下一段并重复；栈中保存的幂次由底到顶递增。
- 5. 最后一段读完后，逐个弹出左段，与当前段归并，直到栈空。可以使用普通缓冲归并，也可配合疾驰优化。

完整小例子：输入为 $[1,5|2,6|0,3,4,7]$ ，自然段长度是 $[2,2,4]$ ，总长 8。三段中心除以总长后，分别为 $1/8$ 、 $3/8$ 、 $3/4$ 。第一条边界两侧中心乘 2 后向下取整均为 0；乘 4 后分别为 0、1，所以边界幂次是 2。

第二条边界两侧的 $3/8$ 与 $3/4$ ，乘 2 后取整已分别为 0、1，因此幂次是 1。更浅的新边界出现，要求先结清前面较深的边界：把 $[1,5]$ 与 $[2,6]$ 合成 $[1,2,5,6]$ ，再与 $[0,3,4,7]$ 合并。依次选两段段首中较小的值，输出 0、1、2、3、4、5、6、7。按参与合并的段长计费，共为 $4+8=12$ 。

效率与存储

最好	$\Theta(n)$ ，只有一个自然有序段
平均或期望	$\Theta(n \log n)$ ，互异键均匀随机排列
最坏	$O(n(1+H))$ ，且为 $O(n \log n)$ ； H 是段长熵
额外空间	$O(n)$ 归并缓冲， $O(\log n)$ 段栈
稳定性	稳定，采用相邻段稳定归并
严格原地	普通缓冲实现不是原地

为什么快，为什么慢

不能只因旁边有许多短段，就让长段中的记录被反复复制。边界幂次利用段的位置与长度，安排接近理想代价的归并树。

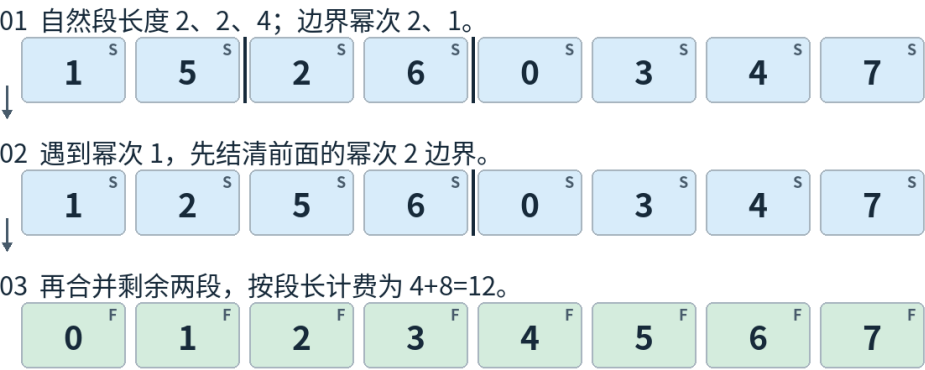
适合场景：适合段长差异明显的数据的稳定自适应排序。令 $p_i=L_i/n$ ，段长熵 $H=\sum p_i \log_2(1/p_i)$ ；归并代价距最优仅差 $O(n)$ 的加法项。

容易误解：幂次取决于有序段在整个数组中的相对位置，并非简单的段长对数；工程实现应使用精确整数或位运算，避免浮点舍入。

依据：[C20] [C19] [C28]

过程图 C19

Powersort：先合并较深的边界



C 正在比较 M 正在移动 S 区域内有序 F 最终归位

C19 Powersort：先合并较深的边界。与本条目的示例对应；标记含义见“基础”。

C20 Peeksort 中点探查归并排序

名称／关系：Peeksort 归并调度策略

先探查区间中点所在的自然有序段，再在附近的段边界分割，而不是把已有的有序段硬切成两半。

具体怎么做

1. 处理区间 [lo,hi] 时，同时记录已经知道的有序前缀和有序后缀。开始时，两端单个元素各算一个已知段；整个区间已知有序时无需处理。
2. 查看 $mid = \text{floor}((lo+hi)/2)$ 。若中点落在已知有序前缀中，只递归处理其后的部分，再归并；落在已知后缀中时对称处理。
3. 否则从 mid 向两侧扫描，找出包含中点的极大非降段 [i,j]，不得越出当前区间；若覆盖全部区间就返回。
4. 在更靠近中点的段边界切开：左边界近就切在 i 前，否则切在 j 后；会产生空半区的边界不用。
5. 递归排好两侧，并把新发现的段边界作为相应子问题的前缀或后缀信息传下去，避免重复扫描；最后稳定归并两边。

完整小例子：对 [1,4|2,3,7|5,6]，中间下标 3 上的数字是 3。向两侧检查，找到下标 2..4 的有序段 [2,3,7]。左右边界距离相同，这里选右边界，即在下标 4 之后切开。

左问题 [1,4|2,3,7] 只有两条有序段，归并时依次取 1、2、3、4、7；右问题 [5,6] 本来就有序。最后先从左边取 1、2、3、4，再从右边取 5、6，最后取 7，得到 [1,2,3,4,5,6,7]。原来的中心有序段被利用起来，而不是硬拆成单元素。

效率与存储

最好	$\Theta(n)$ ，输入非降有序
----	---------------------

平均或期望	$\Theta(n \log n)$ ，互异键均匀随机排列
最坏	$O(n(1+H)) \leq O(n(1+\log r))$ ；总体 $O(n \log n)$
额外空间	$O(n)$ 归并缓冲，加 $O(\log n)$ 控制栈
稳定性	稳定
严格原地	否，缓冲数组实现

为什么快，为什么慢

沿真实有序段边界拆分，可以保留已有成果；记住已发现的端点，则避免后续切分时反复扫描同一长段。

适合场景：它是与 Powersort 对应的自顶向下方法，展示了保留完整有序段如何在段长不均时仍取得近最优归并代价。

容易误解：传递已知前后缀边界是算法的一部分，不是可有可无的记录；反复扫描同一个长有序段会破坏预期的开销界。

依据：[C20] [C21] [C28]

C24 自适应 ShiversSort

名称／关系：Adaptive ShiversSort；不能与所有 ShiversSort 版本混同

只根据最新三个有序段的长度级别决定归并；级别定义为 $\text{floor}(\log_2 \text{段长})$ ，即把长度按 2 的幂划档。

具体怎么做

1. 从左向右识别自然有序段，只反转严格下降段。按原顺序维护一个栈；栈中保存区间端点，不复制整段数据。
2. 长度为 L 的段，其级别为 $\text{level}(L) = \text{floor}(\log_2 L)$ 。本条采用所引 2024 作者稿算法 1 中 $c=1$ 的版本。
3. 读取下一段之前，查看栈顶三个段 X 、 Y 、 Z ， Z 最新。若 $\text{level}(X) \leq \max(\text{level}(Y), \text{level}(Z))$ ，就稳定归并 X 与 Y ， Z 仍放在合并结果上方。
4. 更新合并段长度，再次检查同一条件；若条件不成立，或不足三段，就在还有输入时压入下一段。
5. 输入耗尽且上述条件不再触发后，反复归并栈顶两段，直到只剩一段。数据搬移可由普通稳定缓冲归并完成。

完整小例子：自然段为 $[1,4,7,10 \mid 2,5,8 \mid 0,3,6,9,11]$ ，长度 $[4,3,5]$ 对应层级 $[2,1,2]$ 。较旧一段的层级 2 不大于 $\max(1,2)$ ，因此规则要求合并较旧的两段，而不是默认合并最新两段。

第一次合并比较两段段首，输出 $[1,2,4,5,7,8,10]$ ，留下 $[1,2,4,5,7,8,10 \mid 0,3,6,9,11]$ 。输入读完，合并最后两段：先取 0，再依次取 1、2、3、4、5、6、7、8、9、10、11。长度变化正是 $[4,3,5] \rightarrow [7,5] \rightarrow [12]$ 。

效率与存储

最好	$\Theta(n)$ ，只有一个有序段
平均或期望	$\Theta(n \log n)$ ，互异键均匀随机排列
最坏	$O(n(1+H)) \leq O(n \log n)$ ，H 为段长熵
额外空间	$O(n)$ 归并缓冲，加 $O(\log n)$ 段栈
稳定性	稳定
严格原地	普通缓冲实现不是原地

为什么快，为什么慢

只利用少量段长信息，就能避免长段被浪费地合并许多次。每次合并后都要重算层级，因为下一次判断依据的是新长度。

适合场景：适合学习简单且有理论保证的段自适应归并策略；与 Powersort 一样，归并代价为 $nH+O(n)$ ，但决策只看最新三段的长度。

容易误解：自适应 ShiversSort 不是最初的 ShiversSort；上述规则要求合并三个段中较旧的两段，而非总是合并栈顶两段。

依据：[C27]

C17 内省排序

名称／关系：内省式排序

先使用快速排序，一旦划分深度过大，就把当前子问题改用堆排序，防止退化为平方级时间。

具体怎么做

- 1. 设置深度预算，例如 $2 \times \text{floor}(\log_2 n)$ ，再选一个固定的小区间阈值供插入排序使用。
 - 2. 处理较大区间前先检查预算。若为 0，就对整个当前区间执行堆排序，然后返回。
 - 3. 否则用采样选枢轴，执行正确的双路或三路划分；给两个子问题的预算都减一。
 - 4. 使用继承的预算继续处理子区间。递归较短区、循环处理较长区，可控制额外栈空间。
 - 5. 足够小的区间用插入排序收尾。另一种实现会暂时留下小区间，最后统一做一遍插入整理。
 - 6. 最坏保证来自限制快速划分层数并采用有保证的后备算法，不能只依赖枢轴采样碰巧选得好。
- 完整小例子：用 [3,5,1,4,2] 演示，特意把深度预算设为 1，并关闭“小区间改用插入”的阈值。这里只为在五个数字上展示兜底；正常实现的预算通常接近 $2 \text{ floor}(\log_2 n)$ ，小区间阈值也更大。

以 3 分区，先交换 5 与 2，再将枢轴归位，得到 [1,2|3|4,5]。预算已耗尽，剩下两侧不再继续快速分区，而改用堆排序。[1,2] 先建成 [2,1]，取出最大值 2 后恢复 [1,2]；右侧同理，最终为 [1,2,3,4,5]。若使用常见的小区间阈值，这份小输入会直接由插入排序完成。

效率与存储

最好	基础互异键双路版为 $\Theta(n \log n)$ ，自适应变体另论
平均或期望	$\Theta(n \log n)$ ，随机互异键
最坏	$\Theta(n \log n)$ ，保留堆排序后备
额外空间	$O(\log n)$ 栈
稳定性	不稳定
严格原地	在数组内重排，栈空间并非常数

为什么快，为什么慢

快速分区负责通常的高效路径，深度上限限制坏分区能重复多久；剩余部分交给堆排序，以明确的最坏上界收尾。

适合场景：适合通用不稳定排序，兼顾快速排序的常见表现与确定性的最坏时间上界。

容易误解：预算衡量的是一条分支上的划分深度，不是总调用次数；仅加上插入排序收尾，并不构成内省排序。

依据：[C17]

C21 模式破坏快速排序 pdqsort

名称／关系：pdqsort；模式破坏快速排序

在快速排序中加入低开销的模式检测、确定性的打散交换和坏划分预算，同时保留堆排序兜底。

具体怎么做

- 小区间直接插入排序；较大区间用三数取中选枢轴，足够大时按参考实现使用九数取样的中位数方案。
- 把元素分成小于枢轴和不小于枢轴两区，并记录划分是否发生交换，保留相应的枢轴边界信息。
- 处理右侧区间时，若新枢轴等于已知下界，就把相等元素集中并跳过，因为这里不可能再有更小的元素。
- 若任一侧不足当前区间的八分之一，就记作坏划分，并减少初始约为 $\text{floor}(\log_2 n)$ 的预算；预算耗尽后，对剩余区间执行堆排序。
- 坏划分后，在两侧内部交换选定的端点与四分之一位置元素以打散模式。若划分平衡且无需交换，就尝试带少量搬移预算的插入排序，超过预算立即放弃尝试。

6. 继续处理尚未完成的两侧，小区间由插入排序收尾。若比较器本身适合无分支计算，还可采用块缓冲的无分支分区。

完整小例子：按常见的小区间规则， $[3,1,2]$ 足够短，会直接交给插入排序：右移 3，为 1 腾出位置，成为 $[1,3,2]$ ；再右移 3，插入 2，得到 $[1,2,3]$ 。这确实是混合算法的一条执行路径，不是另拿一种排序冒充 pdqsort。

更大区间才会用到模式处理。单独演示一次坏分区： $[1,5,4,3,2]$ 若取枢轴 1，会分为 $[\]|1|[5,4,3,2]$ 。一侧为空，因此消耗一次坏分区预算；若这就是最后一次预算，便用堆排序把右侧整理成 $[2,3,4,5]$ ，完成整个数组。正式实现的三数取中未必会在这五个数中选到 1，第二个例子只展示预算规则，不声称是某个库逐行执行的日志。

效率与存储

最好	识别到特定模式时 $\Theta(n)$ ：全相等、严格升序或严格降序
平均或期望	参考设计在随机互异键上为 $\Theta(n \log n)$
最坏	$O(n \log n)$ ，限制坏划分次数并保留堆排序后备
额外空间	$O(\log n)$ 栈，加固定大小缓冲
稳定性	不稳定
严格原地	数组内重排，控制空间为对数级

为什么快，为什么慢

识别出的特定模式可能只需很少移动；坏分区预算限制反复猜错的次数，固定大小的块缓冲还可减少分支开销，但不会改变排序本身的目标。

适合场景：适合高性能通用不稳定数组排序；线性特殊情况针对明确识别的模式，并不涵盖所有模糊意义上的“近乎有序”。

容易误解：仅靠打散交换不能证明最坏界，坏划分预算与后备算法不能省略；无分支比较也并非对所有数据类型都更快。

依据：[C22] [C31]

C22 块快速排序 带内省后备版

名称／关系：BlockQuicksort；本文带内省保护的块分区排序

按小块进行分区：先记录哪些位置的元素放错了枢轴一侧，再集中搬移，把比较与移动分开。

具体怎么做

- 1. 选择枢轴和固定块大小 B，例如论文中的 128；维护左右两块及两个“错位下标”缓冲区。
- 2. 扫描左块，记录应到右分区的位置；扫描右块，记录应到左分区的位置。

- 3. 按两个缓冲区中的下标成对交换。若只有一边的缓冲耗尽，保留另一边尚未处理的下标，只让对应已耗尽侧读取新块。
- 4. 某侧错位元素全部处理完后，才把该侧块边界向中间推进。最后正确处理不足整块的中央余项，再让枢轴就位。
- 5. 继续快速排序两侧，小区间插入排序，并像内省排序一样设置深度限制，超限改用堆排序；较短侧递归可控制栈深。

完整小例子：用 [1,8,2,7|5|6,3,9,4] 演示分区，枢轴为 5。左块中偏移 1、3 的 8、7 太大，右块中偏移 1、3 的 3、4 太小。先记录这些错位下标，再按配对执行交换。

交换 8 与 3，得到 [1,3,2,7|5|6,8,9,4]；交换 7 与 4，得到 [1,3,2,4|5|6,8,9,7]。左侧全小于 5，右侧全大于 5。继续排好两侧，得到 [1,2,3,4|5|6,7,8,9]，小侧可以用插入排序。缓存下标改变的是交换调度，不改变分区应满足的条件。

效率与存储

最好	互异键平衡划分且无模式捷径时 $\Theta(n \log n)$
平均或期望	$\Theta(n \log n)$ ，互异键均匀随机排列
最坏	带所述后备为 $O(n \log n)$ ；无保护版本可达 $\Theta(n^2)$
额外空间	$O(\log n)$ 栈加 $O(B)$ ，B 为固定常数
稳定性	不稳定
严格原地	数组内重排，另有下标缓冲与栈

为什么快，为什么慢

先记录比较结果、再集中交换，让循环对处理器更规律。它主要改善常数与分支表现；最坏保证来自内省后备，而不是缓存下标本身。

适合场景：比较便宜时，可降低分支预测失败的代价；它优化机器层面的分区过程，并没有突破比较排序的渐近下界。

容易误解：换块时不能丢弃尚未消费完的错位下标；仅使用块缓冲，并不能阻止划分树退化到平方级。

依据：[C23]

过程图 C22

块分区：先收集下标，再集中交换



C 正在比较 M 正在移动 S 区域内有序 F 最终归位

C22 块分区：先收集下标，再集中交换。与本条目的示例对应；标记含义见“基础”。

C23 稳定块归并排序 WikiSort / GrailSort 家族

名称 / 关系：WikiSort、GrailSort 是相关构造，不是同一份代码的两个名字

阅读层级：高级构造或实现家族；具体展开范围见下文。

借用数组内部的不同键元素作为工作区和块次序标签，归并后再归还；通过精细的内部重排避免线性外部缓冲。

具体怎么做

1. 先把小区间稳定排好，再像自底向上归并一样扩大归并规模。以下给出 WikiSort 风格的相邻两段 A、B 的归并蓝图。
2. 取接近 $\sqrt{|A|}$ 的块长。用保持其余元素顺序的旋转，从 A 或 B 中预留不同键元素：一组作块大小的内部缓冲，一组作标签；这些元素仍属于待排序输入。
3. 把剩余数据分块，将每个 A 块的首元素与一个不同标签交换；原首元素暂存在标签区。即使块被调换，标签仍能标识 A 块原来的次序。
4. 通过整块交换，让 A 块向 B 块中推进。选当前最小标签的 A 块，用其保存的首键在 B 中二分定位，再旋转到该位置。局部归并前恢复原首元素，把前一 A 块和夹在其后的 B 元素稳定合并。
5. 局部归并借用内部缓冲。不同键不足时，减少标签并增大数据块，改用该算法的二分查找加旋转后备归并；两端不足整块的部分要单独处理。
6. 数据块完成后恢复标签，把被打乱的不同键工作区排好，再稳定地把全部预留元素归还，继续更大规模。GrailSort 使用相近思路，但具体构造不同。

完整小例子：先跟踪两条有序段中的全部记录：A=[0,2,4,6,8,9,10,11]，B=[1,3,5,7]。教学上可把互异记录 [0,6,10,11] 预留为内部工作区，剩余数据段为 [2,4,8,9] 与 [1,3,5,7]。预留的四条并没有删除，最后必须全部回到输出。

按每块两条记录划分，[2,4] 与 [8,9] 在块键相同时仍须保留原始块序。可把互异标签暂放到块首，原来的块首键则存回标签区。经过有序块移动和局部稳定合并，数据部分成为 [1,2,3,4,5,7,8,9]。恢复并重新分配预留的 [0,6,10,11] 后，完整输出才是 [0,1,2,3,4,5,6,7,8,9,10,11]。

这是构造层面的示例，说明标签、局部缓冲与恢复为何缺一不可，并追踪记录没有丢失；它没有逐项展开 WikiSort 或 GrailSort 的全部标签交换，两者的少量互异键与边角块处理也并不相同。需要较短、完整明确的无线性缓冲配方时，可读 C31 及其伪代码，代价是接受较弱的 $O(n \log^2 n)$ 时间上界。

效率与存储

最好	依实现而定，不对整个家族宣称统一的紧确最好界
平均或期望	所引实现对随机互异键为 $\Theta(n \log n)$
最坏	正确实现的所引块归并算法为 $O(n \log n)$
额外空间	无动态缓冲版本的外部辅助空间为 $O(1)$
稳定性	完整遵循抽取和归还的相等键规则时稳定
严格原地	是，限无动态缓冲版本

为什么快，为什么慢

按块处理可以集中搬动记录，内部标签又省下独立的线性工作区。恢复步骤及互异键太少时的处理必须足够高效，否则可能多出额外的对数因子。

适合场景：适合无法接受线性临时内存、又要求稳定性的高级场景。固定 512 项缓存仍是 $O(1)$ ；随 n 增长的 $\sqrt{n}$ 或 $n/2$ 缓冲则不是。

容易误解：这是实现蓝图，不能当成可直接替代完整代码的短算法；省略少量不同键时的后备路径或标签恢复，会破坏正确性、稳定性和复杂度。

依据：[C24] [C25] [C26]

C16 堆排序

先建立大顶堆，让最大元素始终位于根部；反复把根换到末尾，再修复缩小后的堆。

具体怎么做

1. 采用从 0 开始的下标，节点 i 的孩子是 $2i+1$ 和 $2i+2$ 。大顶堆要求父节点不小于孩子。
2. 自底向上建堆：令 i 从 $\text{floor}(n/2)-1$ 递减到 0，依次下沉。先选较大的孩子，若孩子更大就交换，并沿这条路径继续。

- 3. 交换 $A[0]$ 和当前堆的最后一项；移到末尾的最大元素已经到达最终位置。
 - 4. 堆大小减一，再在缩小后的堆内下沉新根。已经排好的末尾不能参与堆操作。
 - 5. 重复取最大值与修复，直到堆只剩一个元素。下沉用循环实现，只需少量下标和一个临时元素。
- 小例子： $[3,1,4,2] \rightarrow$ 建堆 $[4,2,3,1] \rightarrow [3,2,1|4] \rightarrow [2,1|3,4] \rightarrow [1,2,3,4]$ 。

效率与存储

最好	全相等且下沉遇序即停时可为 $\Theta(n)$ ；互异键表格常列 $\Theta(n \log n)$
平均或期望	$\Theta(n \log n)$ ，随机互异键
最坏	$\Theta(n \log n)$
额外空间	$\Theta(1)$ ，迭代下沉
稳定性	不稳定
严格原地	是，严格常数额外空间

为什么快，为什么慢

建堆时大部分节点靠近叶子，总成本为线性。之后每取出一个最大值，只修复一条最长约 $\log_2 n$ 的路径，不必重新扫描所有剩余记录。

适合场景：适合同时需要确定性最坏界和常数额外空间的场景，也常作为 introsort、pdqsort 的安全后备。

容易误解：自底向上建堆为 $\Theta(n)$ ，不是 $\Theta(n \log n)$ ；逐个插入是另一种建堆方法。已排序后缀必须排除在堆外。

依据：[C16]

过程图 C16

堆排序：反复取最大值，扩大已归位后缀



C 正在比较 M 正在移动 S 区域内有序 F 最终归位

C16 堆排序：反复取最大值，扩大已归位后缀。与本条目的示例对应；标记含义见“基础”。

X14 弱堆排序

弱堆只保留足以让最大值位于位置 0 的部分父子约束；每项附带一个方向位，决定哪条孩子分支受该约束。

具体怎么做

1. 下标从 0 开始，位置 0 是根。将方向位 r 全部设为 0；这些位描述树的连接方向，不是数据值。
“特殊祖先”指必须不小于当前项的上层节点，它不一定是直接父节点。
 2. 定义 $\text{join}(a,b)$ ：比较 $A[a]$ 与 $A[b]$ 。若 b 处较大，就交换两值，并把 $r[b]$ 在 0、1 间翻转。较大值升到 a ，同时通过变更方向位保持弱堆结构。
 3. 计算 $d(j)$ ：令 $t=j$ ；当 t 的奇偶性等于 $r[\text{floor}(t/2)]$ 时，向上移动到 $\text{floor}(t/2)$ ；最后返回 $\text{floor}(t/2)$ 。
对 $j=n-1$ 到 1 依次执行 $\text{join}(d(j),j)$ ，建堆并让最大值到达 0。
 4. 对 $m=n-1$ 到 2，交换位置 0 与 m 。最大值在 m 就位；剩余前缀需要恢复根的最大值性质。
 5. 令 $j=1$ ；当新下标仍小于 m 时，沿 $j=2j+r[j]$ 向下走到底。随后逐层令 $j=\text{floor}(j/2)$ 向上，并在每个非零 j 执行 $\text{join}(0,j)$ ，通过这些比较恢复根。
 6. 最后按需交换位置 0、1。少于两个元素时直接返回；整个过程中，方向位与数据数组一起保留。
- 完整小例子：从 $A=[2,4,1,3,9]$ 、反向位 $[0,0,0,0,0]$ 开始。建堆中的 $\text{join}(0,4)$ 交换 2、9，得到 $[9,4,1,3,2]$ ，位数组变成 $[0,0,0,0,1]$ 。剩余建堆合并都不需交换，这就是完成后的弱堆。

将根与下标 4 交换，得到 [2,4,1,3|9]；恢复根时与下标 1 交换，成为 [4,2,1,3|9]，反向位为 [0,1,0,0,1]。再把 4 取到下标 3，得到 [3,2,1|4,9]，本次恢复不需交换。

把 3 取到下标 2，成为 [1,2|3,4,9]；根合并交换 1、2，并把反向位 1 改回零，得到 [2,1|3,4,9]。最后交换剩余一对，输出 [1,2,3,4,9]。这些位只控制恢复路径，并不是第二份待排序数据。

效率与存储

最好	标准完整路径版本为 $\Theta(n \log n)$
平均或期望	$\Theta(n \log n)$
最坏	$\Theta(n \log n)$
额外空间	额外 $\Theta(n)$ 位，外加 $O(1)$ 个机器字
稳定性	不稳定
严格原地	数据在原数组内，但额外位数组使其不属于严格常数空间原地排序

为什么快，为什么慢

反向位放宽普通堆的结构，使合并能少做一些比较；取出时仍要走对数长度的路径，而每条记录一位依然是随输入增长的额外空间。

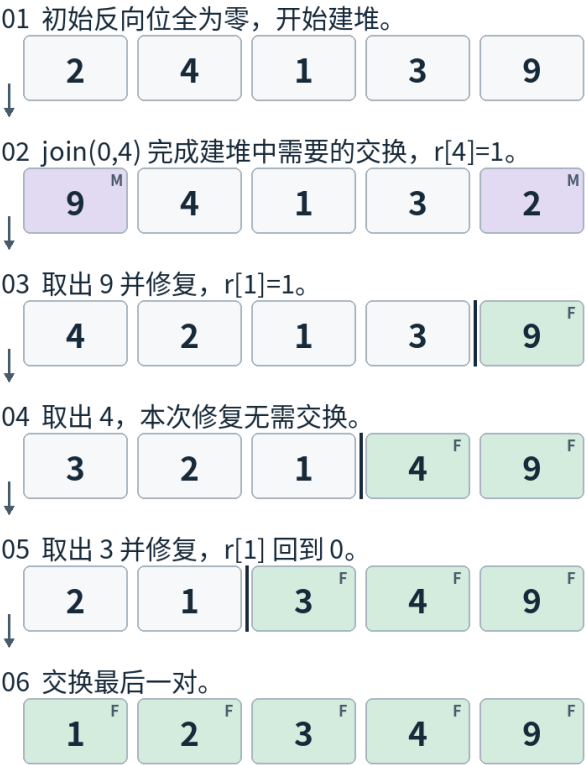
适合场景：适合研究减少比较次数的堆结构，尤其可用来分析比较本身很昂贵的情况。

容易误解：每项一位仍然是随输入增长的额外空间，不能称为 $O(1)$ ；也不能用普通二叉堆规则代替方向位规则。

依据：[X-WEAK] [X-WEAK-CODE]

过程图 X14

弱堆：反向位改变修复路径，后缀保持归位



C 正在比较 M 正在移动 S 区域内有序 F 最终归位

X14 弱堆：反向位改变修复路径，后缀保持归位。与本条目的示例对应；标记含义见“基础”。

X04 平滑排序

平滑排序把数组组织成“森林”，也就是几棵小树，并让剩余最大值始终位于最右边的树根。

具体怎么做

- 1. 最大堆中，每个父节点都不小于它的孩子。树的大小使用 Leonardo 数 1、1、3、5、9.....：后项等于前两项之和再加 1；数组中先放两棵子树，最后放根。
- 2. 从左到右加入元素。若末尾两棵树的阶数为 k+1、k，就以新元素为父节点合并；否则加入单元素树：上一棵为 1 阶时追加 0 阶，其余情况追加 1 阶。
- 3. 各树根应从左到右递增。将新根与前一棵树的根比较；只有前一根比当前根及当前两个孩子都大时，才交换两根，并继续向左比较。
- 4. 不满足上述条件或已无前一根时，停止跨树移动。在当前树内，把移来的元素与较大的孩子交换并继续下沉，直到没有孩子比它大，恢复最大堆规则。
- 5. 森林建好后，最右根就是最大值，将它留在最终位置。取走单元素根就删去该树；取走较大树的根，则留下它的两棵子树。

6. 对暴露出的根继续执行上述跨根比较与向下交换，先处理左根，再处理右根。不断取最大值直到结束；紧凑实现只编码树的大小，并不另建节点。

小例子：处理 [3,1,2]：前两个单元素根 [3] [1] 交换为 [1] [3]。加入 2 后合成 [1,3,2]，末尾的 2 是根；孩子 3 更大，于是交换得到 [1,2,3]。取走根 3 后，留下的单元素根 1、2 也已按顺序排列。

效率与存储

最好	已有序时 $\Theta(n)$
平均或期望	$O(n \log n)$ ；实际适应性取决于输入结构
最坏	$\Theta(n \log n)$
额外空间	Dijkstra 紧凑版本为 $O(1)$ 个机器字
稳定性	不稳定
严格原地	紧凑实现为原地排序

为什么快，为什么慢

Leonardo 森林能识别根与孩子已经满足要求的情况，在有序输入上少做许多堆修复；困难输入仍通过受控长度的路径处理。

适合场景：展示如何在保留最坏 $n \log n$ 保证的同时利用已有顺序，适合进阶学习。

容易误解：这里展示可手算的 Leonardo 森林规则；Dijkstra 的具体实现还包含紧凑状态记录和修复时机优化。仅照搬普通二叉堆下沉并不够；显式保存堆大小列表则需要 $O(\log n)$ 额外空间。

依据：[X-SMOOTH] [X-SMOOTH-IMPL]

X02 平衡搜索树排序

名称／关系：平衡搜索树排序；本文用 AVL 实现

用高度受控的搜索树保存记录，避免插入路径退化成链。

具体怎么做

1. 建立空 AVL 树；相同键用先进先出队列保存，空子树高度记为 0。
2. 按二叉搜索规则插入，并向上更新祖先高度：节点高度等于较高孩子的高度加 1。
3. 在 x 处右旋：把左孩子 y 提到 x 的位置，让 x 成为 y 的右孩子，并把 y 原来的右子树接到 x 的左侧。左旋完全镜像。这样重接指针不会改变中序次序。
4. 若节点左边比右边高两层，就右旋；若其左孩子偏右，先对左孩子左旋，再对当前节点右旋。
5. 右边过高时对称处理：左旋；若右孩子偏左，先对右孩子右旋。继续向根返回，并更新经过节点的高度。

6. 每次插入都恢复平衡；最后依次输出左子树、节点队列、右子树。

小例子：插入 [1,2,3]：根 1 的右孩子是 2，2 的右孩子是 3。在 1 处左旋，将 2 提到根，让 1 成为它的左孩子，3 保持为右孩子。中序输出为 1,2,3。

效率与存储

最好	普通 AVL 逐次插入不同键时为 $\Theta(n \log n)$
平均或期望	不同键时 $\Theta(n \log n)$
最坏	$\Theta(n \log n)$ ；若只有 d 个不同键，上界为 $O(n \log d + n)$
额外空间	记录与节点 $\Theta(n)$ ；栈 $O(\log n)$
稳定性	同键使用先进先出队列时稳定
严格原地	否

为什么快，为什么慢

旋转阻止搜索树长成一条长链，让每次插入只走对数数量的链接；最后中序遍历把每条记录输出一遍。

适合场景：需要确定的性能上界，以及后续有序查找、前驱或后继查询时可采用。

容易误解：“树排序是 $n \log n$ ” 必须附带平衡机制或随机输入假设；旋转也必须保持中序顺序不变。

依据：[X-BALANCED] [X-AVL]

X03 锦标赛排序

名称／关系：选择树排序；锦标赛排序

每轮对决让较小者晋级；输出冠军后，只重算冠军经过的路径。

具体怎么做

- 1. 把记录放在完全二叉比赛树的叶子上，多余叶子标为“无参赛者”。
- 2. 每对中选择较小的有效记录；键相同时让原位置更靠前者晋级。
- 3. 逐层向上比较，根节点最终保存全局最小记录及其叶子编号。
- 4. 把冠军追加到输出，并将其叶子标为无效。
- 5. 只重新计算该叶子的祖先节点；重复直到全部 n 条记录输出完毕。

小例子：[4,1,3,2] 首轮胜者为 1 与 2，根输出 1；更新相关路径后依次输出 2、3、4。

效率与存储

最好	每次都回放固定高度路径时为 $\Theta(n \log n)$
----	----------------------------------

平均或期望	$\Theta(n \log n)$
最坏	$\Theta(n \log n)$
额外空间	比赛树及输出需要 $\Theta(n)$ 空间
稳定性	按原位置处理相等键时稳定
严格原地	否

为什么快，为什么慢

取走一个获胜者后，只需重赛它经过的那条路径。其余比赛结果继续使用，不用为下一个最小值重新扫描所有记录。

适合场景：适理解重复取最小值，以及多路归并中使用的胜者树和败者树。

容易误解：哨兵必须排在所有有效键之后，包括真正的无穷大；显式使用有效标记更稳妥。

依据：[X-TOURNAMENT]

X05 耐心排序

先把数字放成自下而上递减的牌堆，再不断取出各堆顶中最小的牌。

具体怎么做

1. 从左到右读入牌 x ，始终让各堆顶从左至右递增。
2. 二分查找第一个堆顶不小于 x 的牌堆，把 x 压到它上面。
3. 若所有堆顶都小于 x ，就在最右边新建牌堆。
4. 把每个牌堆的堆顶加入最小优先队列，并保留所属牌堆编号。
5. 取出最小堆顶追加到输出，再用该牌堆新露出的牌补入队列。
6. 直到队列和全部牌堆都为空时结束。

小例子：[3,1,4,2] 形成 [3,1]、[4,2] 两堆，右侧是堆顶；依次取出的最小值为 1、2、3、4。

效率与存储

最好	降序输入只形成一堆时 $\Theta(n)$
平均或期望	$O(n \log(p+1))$ ， p 为牌堆数量
最坏	$\Theta(n \log n)$
额外空间	$\Theta(n)$
稳定性	按“堆顶 $\geq x$ ”放牌时不稳定，同键会倒叠

严格原地	否
------	---

为什么快，为什么慢

二分决定放入哪堆，小堆决定下一次取哪个堆顶。形成的堆数少时，两种结构都很小；直接把牌堆连接起来并不能完成排序。

适合场景：可把排序与最长严格递增子序列联系起来：按照上述放牌规则，牌堆数就是该子序列长度。

容易误解：只建牌堆还没有完成排序，最后的归并输出不可省略。

依据：[X-PATIENCE] [X-PATIENCE-CODE]

X15 笛卡尔树排序

建立中序顺序与输入一致的最小堆树，再用优先队列选择已暴露节点中的最小值。

具体怎么做

- 1. 扫描数组，维护一个从栈底到栈顶键值递增的节点栈。
- 2. 遇到新节点 x ，弹出所有比它大的栈顶；最后弹出的节点成为 x 的左孩子。
- 3. 若栈非空，让 x 成为当前栈顶的右孩子，再把 x 压栈；扫描结束后栈底即根。
- 4. 先只把根加入最小优先队列，并准备空输出。
- 5. 取出队列中最小节点并输出其值，再把该节点存在的左右孩子加入队列。
- 6. 队列为空时结束；不能只做中序输出，那样得到的仍是原来的未排序输入。

小例子：[3,1,4,2] 建树后根为 1，左孩子 3，右孩子 2，且 2 的左孩子为 4；候选集合依次为 {1}、{3, 2}、{3,4}、{4}。

效率与存储

最好	单调输入的候选集合大小为 1，时间 $\Theta(n)$
平均或期望	$O(n \log n)$ ；更细为 $O(n+\sum \log(q_i+1))$ ， q_i 为各步候选数
最坏	候选队列使用二叉堆时，最坏 $\Theta(n \log n)$
额外空间	$\Theta(n)$
稳定性	不附加原位置处理相等键时，不保证稳定
严格原地	否

为什么快，为什么慢

建笛卡尔树时，每个节点至多进出构建栈一次，因此为线性；真正排序还取决于前沿优先队列的大小，不能只看建树很便宜。

适合场景：展示通过候选集合大小实现的适应性；初始树本身可在线性时间内建立。

容易误解：笛卡尔树按值满足堆序、按原位置满足中序顺序，并不是按值建立的二叉搜索树。

依据：[X-CARTESIAN] [X-CARTESIAN-CODE]

X16 伸展树排序

名称／关系：Splaysort；伸展树排序

伸展树排序逐项插入搜索树，再用不改变中序次序的旋转，把刚插入的键提升到根。

具体怎么做

1. 建立空二叉搜索树，相同键的记录用先进先出队列保存。
2. 按较小走左、较大走右的规则查找；新键建节点，同键追加队列。
3. 旋转会把 x 提到父节点上方： x 是左孩子时右旋，是右孩子时左旋；键值夹在两者之间的子树转接到空出的孩子位置。若父节点就是根，只需旋转一次，称为 zig。
4. 若 x 与父节点同为左孩子或同为右孩子，先把父节点转到祖父上方，再把 x 转到父节点上方，称为 zig-zig。
5. 若方向相反，先把 x 转到父节点上方，再转到祖父上方，称为 zig-zag；重复直到 x 成为根。
6. 全部插入后按左子树、节点队列、右子树的顺序输出。

小例子：插入 [1,3,2]：前两项形成根 3、左孩子 1；再把 2 插到 1 的右侧。先在 1 处左旋，再在 3 处右旋，2 就成为根，左右孩子为 1、3。这就是 zig-zag；中序输出为 1,2,3。

效率与存储

最好	例如不同键单调输入时 $\Theta(n)$
平均或期望	不同键等概率随机排列时 $\Theta(n \log n)$
最坏	总最坏时间 $\Theta(n \log n)$ ；单次插入仍可能 $\Theta(n)$
额外空间	$\Theta(n)$ ；遍历栈最高可达 $O(n)$
稳定性	本例按同键先进先出策略，因此稳定
严格原地	否

为什么快，为什么慢

一次伸展操作可能很长，保证针对的是整串旋转的总账；昂贵操作会改变树形，后续操作再利用这些变化分摊成本。

适合场景：适合学习均摊分析和自适应数据结构；连续插入的键在排名上相近时可能很省时。

容易误解：伸展树并非始终高度平衡；对数保证来自一系列操作的均摊分析，而非每次操作的最坏时间。

依据：[X-SPLAY] [X-SPLAY-BASE]

C 期望表现较好：不保护时最坏较弱

均衡分区或随机化构造可以带来接近 $n \log n$ 的工作量，但这里不带保护的版本并非每次都保证如此。加入内省后备机制后，某个实现可能进入 B 组。随机输入的平均情况，也不同于固定输入下对随机选择求期望。

编号只标识条目，不表示名次。

C13 随机化双路快速排序

名称／关系：快速排序；本文为随机枢轴、枢轴归位的双路版本

选一个枢轴，把较小值移到左侧、较大值移到右侧，再分别处理两边；随机化降低持续遇到坏划分的概率。

具体怎么做

1. 先随机打乱数组，或每次在当前区间均匀随机选枢轴。把枢轴移到首位 lo ，保存其键 p 。
2. 令 $i=lo+1$ 、 $j=hi$ ；只要 $A[i]<p$ 就右移 i ，只要 $A[j]>p$ 就左移 j ，并检查边界。
3. 若 $i \geq j$ ，结束扫描；否则交换 $A[i]$ 、 $A[j]$ ，再令 i 加一、 j 减一，继续。遇到相等键时，两侧扫描都应停下。
4. 交换 $A[lo]$ 与 $A[j]$ ，枢轴位置 j 已确定。后续只处理 $[lo, j-1]$ 和 $[j+1, hi]$ ，不再包含 j 。
5. 递归处理较短的一侧，较长一侧用循环继续，可控制栈深度。区间不足两个元素时结束。

小例子：以 3 为枢轴的一次划分： $[3,5,1,4,2] \rightarrow$ 交换 5 与 2 得 $[3,2,1,4,5] \rightarrow$ 枢轴就位得 $[1,2|3|4,5]$ 。

效率与存储

最好	$\Theta(n \log n)$ ，互异键且划分平衡
平均或期望	期望 $\Theta(n \log n)$ ，随机枢轴或互异键均匀随机排列
最坏	$\Theta(n^2)$ ，随机化只降低连续坏划分的概率
额外空间	较短侧递归为 $O(\log n)$ 栈；朴素递归最坏 $O(n)$
稳定性	不稳定
严格原地	数组内部重排，但总额外空间并非严格 $O(1)$

为什么快，为什么慢

一次分区扫描当前区间一遍。若不断均衡拆分，只有约 $\log n$ 层；若每次只分出一条，就会剩下 $n-1$ 、 $n-2$条，工作量累积成平方级。

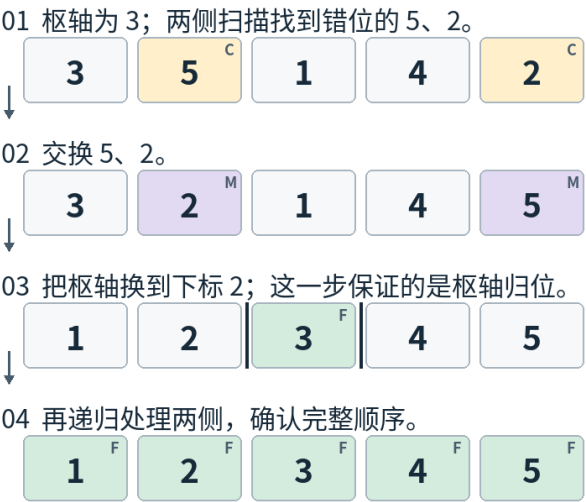
适合场景：适合不要求稳定性的通用数组排序；随机分析也是理解实用快速排序混合算法的基础。

容易误解：随机化不会消除数学上的平方级最坏情况；分区端点、相等键时的推进，以及递归排除枢轴，必须彼此一致。

依据：[C13]

过程图 C13

快速排序：分区并不等于已经全部排好



C 正在比较 M 正在移动 S 区域内有序 F 最终归位

C13 快速排序：分区并不等于已经全部排好。与本条目的示例对应；标记含义见“基础”。

C26 采用 Lomuto 分区的快速排序

名称／关系：使用 Lomuto 分区的快速排序

这种变体从左往右扫描，用一条边界把“不大于枢轴”的数字集中起来。动作容易理解，但大量相等的数字会让分区严重失衡。

具体做法

- 1. 对下标 lo 到 hi 的闭区间排序；lo ≥ hi 就停止。选一个枢轴，先把它交换到 A[hi]，记下其值 p，并令 i = lo。
- 2. 让 j 从 lo 扫到 hi-1。若 A[j] ≤ p，就交换 A[i] 与 A[j]，然后 i 加一。这样，i 前面全是不大于 p 的已检查数字；i 到 j 之间已检查的数字则大于 p。
- 3. 扫描结束后交换 A[i] 与 A[hi]，枢轴就到了最终位置 i。再分别排序 lo..i-1 和 i+1..hi，不能把已经归位的枢轴重新包含进去。
- 4. 可以随机选择枢轴，降低对原始排列的依赖。只递归处理较短的一侧，较长的一侧用循环继续，可控制递归栈深度。

跟着做：从 [4,1,3,2] 开始，选末尾的 2， $i=0$ 。4 大于 2，先不动。遇到 1 时，交换下标 0 与 1，得到 [1,4,3,2]，再令 $i=1$ 。

跟着做：3 也大于 2。扫描结束，把末尾的 2 与下标 1 交换，得到 [1|2|3,4]。左边只有一个数字；右边以 4 为枢轴，保持 [3,4]。最终得到 [1,2,3,4]。每次分区保证的是一个枢轴归位，并非整段已经排好。

效率与存储

最好	分区均衡时 $\Theta(n \log n)$ ，指完整递归版本
期望	互异键、每次独立均匀随机选枢轴时 $\Theta(n \log n)$
最坏	$\Theta(n^2)$ ；这里的 $\leq$ 规则遇到全相等输入也会退化
额外空间	只递归较短侧时 $O(\log n)$ ；直接递归两侧最坏 $O(n)$
稳定性	不稳定；严格原地：计入递归栈后不算

为什么快，为什么慢

一次分区扫描当前区间一遍。若不断均衡拆分，只有约 $\log n$ 层；若每次只分出一条，就会剩下 $n-1$ 、 $n-2$条，工作量累积成平方级。

适合：学习分区边界。单向扫描容易写清楚，但交换次数可能比双向扫描更多。

注意：随机选枢轴不能解决本版本的全相等问题，需要三路分区等办法。Lomuto 是快速排序的分区方案，不是另一个独立家族。

来源：[N01]

过程图 C26

Lomuto：单向扫描扩大左侧区域

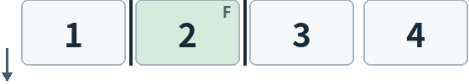
01 枢轴 2， $i=0$ ；4 不属于左侧的 ≤ 2 区域。



02 1 符合条件，交换到下标 0，令 $i=1$ 。



03 3 不符合条件，最后把枢轴交换到下标 1。



04 排两侧区域，递归范围不再包含枢轴。



C 正在比较 M 正在移动 S 区域内有序 F 最终归位

C26 Lomuto：单向扫描扩大左侧区域。与本条目的示例对应；标记含义见“基础”。

C27 采用 Hoare 分界式分区的快速排序

名称／关系：使用 Hoare 分界的快速排序

两根指针从两端相向扫描，交换站错一侧的数字。这里返回的是“两段之间的边界”，而不是枢轴的最终位置；这与 C13 的枢轴归位方案不同。

具体做法

1. 若 $lo < hi$ ，先保存枢轴值 $p=A[lo]$ ，令 $i=lo-1$ 、 $j=hi+1$ 。要保存值，因为原来的枢轴记录可能被交换到别处。
2. i 至少右移一次，遇到 $A[i] \geq p$ 停下； j 至少左移一次，遇到 $A[j] \leq p$ 停下。若 $i < j$ ，交换这两个数字，再重新移动两根指针。
3. 若 $i \geq j$ ，返回 j 。此时 $lo..j$ 内都不大于 p ， $j+1..hi$ 内都不小于 p 。下一层必须递归处理这两个完整区间。
4. 本写法从首位置取枢轴，保证返回的边界让问题缩小。要随机化，可先把随机选中的元素换到首位置。交换相等数字后也必须让两根指针继续移动。

跟着做：对 $[4,1,3,2]$ ，保存 $p=4$ 。 i 停在 4， j 停在 2，交换得 $[2,1,3,4]$ 。下一轮 i 到下标 3， j 停在下标 2，两者交错，返回 $j=2$ 。左区间是 $[2,1,3]$ ，右区间是 $[4]$ 。请注意：返回的下标 2 并不是枢轴 4 的位置。

跟着做：对 $[2,1,3]$ 再分区， $p=2$ ，先交换 2 与 1，得到 $[1,2,3]$ ；扫描交错后返回边界 0。继续处理 $[1]$ 与 $[2,3]$ ，整份输入就变成 $[1,2,3,4]$ 。

效率与存储

最好	完整递归版本为 $\Theta(n \log n)$
期望	互异键、随机枢轴时 $\Theta(n \log n)$
最坏	枢轴长期靠近极端位置时 $\Theta(n^2)$
额外空间	只递归较短侧为 $O(\log n)$ ，直接递归最坏 $O(n)$
稳定性	不稳定；计入栈后不是严格 $O(1)$ 额外空间

为什么快，为什么慢

一次分区扫描当前区间一遍。若不断均衡拆分，只有约 $\log n$ 层；若每次只分出一条，就会剩下 $n-1$ 、 $n-2$条，工作量累积成平方级。

适合：理解快速排序中两种不同的“分区结果”。相向扫描通常能少交换一些原本就在正确一侧的数字。

注意：不能把这里的边界与“跳过归位枢轴”的递归区间混用，否则可能漏排。相等值应让扫描停下，交换后再前进。

来源：[N01][N02]

过程图 C27

Hoare：返回分界，而非枢轴最终位置

01 保存枢轴值 4，比较两根指针停下的位置。



02 交换后， $i=3$ 、 $j=2$ 交错，返回边界 $j=2$ 。



03 递归处理 0..2 与 3..3；左侧先交换 2、1。



04 返回边界不是枢轴 4 的位置，最终全部排好。



C 正在比较 M 正在移动 S 区域内有序 F 最终归位

C27 Hoare：返回分界，而非枢轴最终位置。与本条目的示例对应；标记含义见“基础”。

C14 三路快速排序

名称／关系：荷兰国旗分区的三路快速排序

围绕一个枢轴分成小于、等于、大于三个区域，后续递归跳过整个相等区，因此特别适合重复键较多的数据。

具体怎么做

1. 选取枢轴键 p ，令 $lt=lo$ 、 $i=lo$ 、 $gt=hi$ ； $i..gt$ 是尚未分类的区间。
2. 若 $A[i]<p$ ，交换 $A[i]$ 与 $A[lt]$ ，再同时增加 lt 和 i ，扩大小于区。
3. 若 $A[i]>p$ ，交换 $A[i]$ 与 $A[gt]$ ，再将 gt 减一；此时不要增加 i ，因为新换来的元素还没分类。
4. 若 $A[i]=p$ ，只需增加 i 。直到 $i>gt$ ，得到小于区 $[lo,lt)$ 、相等区 $[lt,gt]$ 、大于区 $(gt,hi]$ 。
5. 只排序小于区和大于区。要获得期望时间保证，可随机选枢轴；递归较短区、循环处理较长区可限制栈深。

完整小例子：对 $[2,3,2,1,2]$ ，枢轴值取 2。第一个 2 进入等值区。遇到 3 时，把它与最后一个未知元素 2 交换，得到 $[2,2,2,1,3]$ 。扫描位置新换来的 2 仍需检查，它和后面的 2 随后都加入等值区。

接着遇到 1，将它与等值区最前面的 2 交换，得到 [1 | 2,2,2 | 3]。未知区为空，两边又都只有一个数字，所以已经完成。中间三个 2 不必再进入下一层递归。

效率与存储

最好	全相等时 $\Theta(n)$ ；互异且平衡时 $\Theta(n \log n)$
平均或期望	随机互异键期望 $\Theta(n \log n)$ ，重复键可改善
最坏	$\Theta(n^2)$ ，互异键持续坏枢轴
额外空间	较短侧递归栈 $O(\log n)$ ，朴素递归可达 $O(n)$
稳定性	不稳定
严格原地	在数组内部重排，递归仍占空间

为什么快，为什么慢

等于枢轴的整组记录一起退出递归，避免反复处理同一等值组；键全不相同时，主要成本仍由分区是否均衡决定。

适合场景：适合重复值很多的类别键、取值种类少的数据，以及需要避免反复处理相等键的快速排序。

容易误解：三路是相对于一个枢轴的三种比较结果；它不同于使用两个枢轴值的双枢轴快速排序。

依据：[C14]

过程图 C14

三路分区：等值组一起退出递归



C 正在比较 M 正在移动 S 区域内有序 F 最终归位

C14 三路分区：等值组一起退出递归。与本条目的示例对应；标记含义见“基础”。

C15 双枢轴快速排序

名称／关系：双枢轴快速排序

用两个有序枢轴把值域分成三段；以下说明清晰的教材版本，不等同于生产 JDK 中包含全部优化的实现。

具体怎么做

1. 先随机化输入或枢轴选择。对 $[lo,hi]$ ，调整两端使 $p=A[lo] \leq q=A[hi]$ ；不足两个元素时返回。
2. 令 $lt=lo+1$ 、 $gt=hi-1$ 、 $i=lt$ ，只要 $i \leq gt$ ，就按 p 和 q 对 $A[i]$ 分类。
3. 若 $A[i]<p$ ，交换它与 $A[lt]$ ，再同时增加 lt 、 i ；若 $A[i]>q$ ，交换它与 $A[gt]$ 、减少 gt ，并重新检查新来的 $A[i]$ ；其余情况只增加 i 。
4. 扫描后令 lt 减一、 gt 加一，把 p 交换到 lt 、 q 交换到 gt ；留下小于 p 、位于 p 与 q 之间、大于 q 的三个区间。
5. 递归排好左右外区间。仅当 $p<q$ 时递归中间区；若 $p=q$ ，中间所有元素都相等。递归不再包含两个已就位的枢轴。

小例子：枢轴取 2 和 5： $[2,6,4,1,3,5] \rightarrow [1|2|4,3|5|6]$ ；再排好中间的 $[4,3]$ 即完成。

效率与存储

最好	全相等且跳过中间区时 $\Theta(n)$ ；平衡互异键为 $\Theta(n \log n)$
平均或期望	互异键均匀随机排列时，期望 $\Theta(n \log n)$
最坏	$\Theta(n^2)$ ，枢轴持续产生失衡划分
额外空间	朴素递归最坏 $O(n)$ ；最大区间用循环可降至 $O(\log n)$
稳定性	不稳定
严格原地	在数组内部重排，递归仍占空间

为什么快，为什么慢

两个枢轴产生三个区域，但一条记录可能接受更多次判断。递归层数变少，不自动等于实测更快，比较顺序与搬动次数仍影响常数。

适合场景：用于理解多枢轴划分在比较、交换和内存访问上的取舍；实际速度高度依赖具体实现。

容易误解：两个枢轴不保证三等分，也不自动保证最坏 $O(n \log n)$ ；相等枢轴应跳过中间区，避免无意义递归。

依据：[C15] [C29]

C32 QuickMergesort：借用内部缓冲区的混合排序

名称／关系：QuickMergesort；以归并为子算法的 QuickXsort

先按枢轴分区，排其中一侧时，借另一侧的位置当临时存放区；随后再排被借用的一侧。缓冲区里原本就有真实数据，因此不能像普通空数组那样直接覆盖。

具体做法

1. 把当前区间分成左侧、枢轴、右侧。容易理解的一种选择是用归并排序处理较短侧，较长侧一定能提供足够的半段缓冲位置。
2. 使用能保留被挤出记录的交换式归并。安排好两条有序段与工作区，让输出位置原来的记录转移到已经读完的位置，并用一个暂存记录闭合移动环。不能照搬普通复制式归并。
3. 一侧排完后，它已处在枢轴正确的一边；借出去的位置仍保留另一侧的全部记录，只是内部顺序变了。再对那一侧继续 QuickMergesort，小区间可改用插入排序。
4. 随机抽样选枢轴有利于平均表现；若要简单明确的最坏 $O(n \log n)$ 保证，可另外设置深度预算，耗尽后改用堆排序。

跟着做：假设分区后为 [3,1,2|4|7,5,6]。先把左边的前两个数排成 [1,3,2|4|7,5,6]，再把有序段 [1,3] 交换到右侧两个位置，得到 [7,5,2|4|1,3,6]。左侧的 7、5 是被暂时挪进来的工作区记录，不能丢弃。

跟着做：现在合并借用位置里的 [1,3] 与原位置上的 [2]。把 1 交换到下标 0，得到 [1,5,2|4|7,3,6]；把 2 交换到下标 1，得到 [1,2,5|4|7,3,6]；再把 3 交换到下标 2，得到 [1,2,3|4|7,5,6]。最后把右侧 [7,5,6] 排好，便得到 [1,2,3,4,5,6,7]。

效率与存储

期望	合适的随机抽样与正确内部归并机制下为 $O(n \log n)$
最坏	不保护可能 $O(n^2)$ ；加入所述内省兜底后为 $O(n \log n)$
额外空间	$O(\log n)$ 栈；记录工作区借自输入数组
稳定性	不稳定；计入栈后不是严格 $O(1)$ 额外空间

为什么快，为什么慢

借用空间省掉的是大型外部记录缓冲，并没有消除合并工作。分区合理时，继续处理的一侧不断缩小；缩小得不好时仍需后备方案。

适合：希望结合归并的比较效率与较少外部存储时。更精细的实现会根据可借空间大小，调整先排序哪一侧。

注意：例子展示一个确实可行的小型移动过程，不代表任意归并代码都能把赋值改成交换。读写区域重叠必须专门处理；论文中的比较次数常数也依赖具体变体。

来源：[N06]

X01 非平衡二叉搜索树排序

名称／关系：二叉搜索树排序；此处不平衡

把记录逐个插入二叉搜索树，再按“左子树、节点、右子树”的顺序读出。

具体怎么做

1. 建立空树；每个键对应一个先进先出的记录队列，避免重复键覆盖记录。
2. 对每条输入记录，从根开始比较：较小走左边，较大走右边。
3. 遇到空位置就建节点；遇到相同键，就追加到该节点队列末尾。
4. 全部插入后递归遍历：先读左子树，再输出当前节点的全部记录，最后读右子树。
5. 将输出写入结果；本版本不通过旋转调整树的形状。

小例子：[3,1,2]：3 作根，1 在其左侧，2 在 1 的右侧；中序遍历得到 1,2,3。

效率与存储

最好	不同键且插入顺序形成平衡树时为 $\Theta(n \log n)$
平均或期望	不同键的输入顺序等概率随机时为 $\Theta(n \log n)$
最坏	$\Theta(n^2)$ ；已有序的不同键会形成一条链
额外空间	节点 $\Theta(n)$ ；遍历栈 $O(\text{树高})$
稳定性	本例同键记录按先进先出保存，因此稳定
严格原地	否

为什么快，为什么慢

树形合理时搜索路径短，退化成链时路径可长到 n 。按有序次序插入非平衡树，就是很典型的成链情况。

适合场景：适合练习树的插入与中序遍历，或排序后仍需使用搜索树的场景。

容易误解：普通字典可能覆盖重复键，造成数据丢失；对本算法而言，已有序输入反而可能最慢。

依据：[X-BST]

X06 图书馆排序

名称／关系：留空隙插入排序；图书馆排序

有意在已排序元素间保留空位，让新元素通常只需移动附近一小段。

具体怎么做

1. 随机打乱待插入顺序，分配 $(1+\epsilon)n$ 个槽位，其中 ϵ 是固定正数。

- 2. 先放入一个元素；分轮插入，每轮让已存元素数量大致翻倍。
 - 3. 每轮开始，将已有元素均匀分散到扩大的使用区域，记录作为搜索依据的支撑位置。
 - 4. 对新元素二分查找支撑位置，再在相邻小区间内确定准确插入位置。
 - 5. 只把连续的较大元素移动到附近空位，就地腾出位置插入，保持非空元素有序。
 - 6. 到轮次边界再次均匀留空，全部完成后删除空隙，输出普通有序序列。
- 完整小例子：考虑到达次序 3、1、5、4。先把 3 放在小书架 [3,空]；插入 1 时，把 3 移到空格，成为 [1,3]。下一轮扩容前，将已排记录重新分散到较大空间，例如 [1,空,3,空,空,空,空,空]。
- 在 3 后的空位插入 5，成为 [1,空,3,空,5,空,空,空]。在已占位置之间二分查找，确定 4 在 3、5 之间，邻近空格让它直接变成 [1,空,3,4,5,空,空,空]。最后压紧已占位置，输出 [1,3,4,5]。若附近没空格，就把一小段占用记录朝空格移动；达到扩容轮次时再整体分散。
- 这些书架状态展示移动规则，不冒充正式算法的精确轮次大小。期望／高概率 $O(n \log n)$ 依赖随机插入次序及合适的余量、重排计划，不是随便留几个洞就能保证。

效率与存储

最好	本二分搜索方案的最好情况上界为 $O(n \log n)$
平均或期望	随机到达顺序下，期望及高概率时间均为 $O(n \log n)$
最坏	不利插入使局部空隙耗尽时，上界 $O(n^2)$
额外空间	包括预留空位在内需要 $\Theta(n)$ 空间
稳定性	随机打乱后不稳定；附加原位置作为次关键字可恢复稳定性
严格原地	否，需要额外留空容量

为什么快，为什么慢

空格让插入不必每次移动整个紧密后缀；随机到达次序与定期重新分散，才使局部拥挤能以高概率保持可控。

适合场景：展示额外容量如何减少移动量，也有助于理解稀疏数组和紧凑动态数组结构。

容易误解：不打乱时不能把随机上界用于对抗性输入；二分搜索遇到空位的处理也必须明确。

依据：[X-LIBRARY]

D12 采样排序

用排好的样本键选取多个分界键，形成可独立排序的有序桶。

具体怎么做

- 1. 选择 b 个桶，抽取 $s \geq b$ 个有代表性的样本键并排序。

- 2. 按样本中等间隔的名次选出 $b-1$ 个分界键，并规定等于分界键的记录放哪侧。
 - 3. 对分界键二分查找，确定每个输入键的桶，并分配记录。
 - 4. 每个桶使用有 $O(t \log t)$ 保证的比较排序，例如对大小为 t 的桶使用归并排序。
 - 5. 按桶序连接结果；并行时把桶交给不同工作线程，全部完成后才算排序结束。
- 小例子：分界键 4、7 把 [8,2,6,1,9,5] 分成 [2,1 | 6,5 | 8,9]，分别排序后连接。

效率与存储

最好	桶均衡时最大局部排序任务最小
平均或期望	期望均衡需要足够随机样本或有证明的采样规则
最坏	工作量 $O(s \log s + n \log b + \sum n_i \log n_i)$ ，单桶可能有 n 项
额外空间	缓冲式分配为 $O(n+s+b)$
稳定性	须分配、桶内排序和等键处理都保持稳定
严格原地	该缓冲版本不是原地排序

为什么快，为什么慢

好的样本能划出大小接近的桶，方便独立排序；抽样与查分界也有开销，倒霉或偏斜的边界会把大部分工作压到一个桶里。

适合场景：适用于一般可比较键以及多核、分布式排序；运行时间还包含分界键构建和通信。

容易误解：样本过小或无代表性会破坏负载均衡；总工作量与 p 个线程上的耗时不是同一概念。

依据：[DIST17] [DIST18]

D 介于 $n \log n$ 与平方级之间

指定的二分旋转归并有 $O(n \log^2 n)$ 上界；采用指定 Knuth 增量的希尔排序有 $O(n^{3/2})$ 上界。这里只比较这些特定界的增长率，不代表所有增量序列或每份输入都一样。

编号只标识条目，不表示名次。

c31 稳定的二分旋转归并排序

名称／关系：二分旋转归并；递归旋转归并排序

先排好左右两半，再用二分查找确定应当交换位置的中间两块，通过旋转完成移动。它不需要线性大小的临时数组，但同一条记录可能在多次旋转中反复搬动。

具体做法

- 1. 递归排好左右两半。合并相邻有序段 A、B 时，任一为空就返回；总共只有两个元素时，只在逆序时交换。
- 2. 把较长的一段从中间分开。若枢轴来自 A，在 B 中找第一个不小于它的位置；若来自 B，在 A 中找第一个严格大于它的位置。两种不同的相等边界，是保持稳定性的关键。
- 3. 把四块写成 A1 A2 | B1 B2。将中间的 A2 B1 旋转为 B1 A2，保持每块内部顺序。具体可先反转 A2，再反转 B1，最后反转它们的整体。
- 4. 接着分别合并 A1 与 B1、A2 与 B2。二分找到的边界保证：左侧子问题的所有记录，都应排在右侧子问题之前。

跟着做：从无序输入 [4,1,5,2,3,6] 开始，先分别排好两半，得到 [1,4,5 | 2,3,6]。在右段的 3 前切开，再在左段找第一个大于 3 的位置，也就是 4 前面。因此 A1=[1]、A2=[4,5]、B1=[2]、B2=[3,6]。

跟着做：把 [4,5 | 2] 旋转成 [2 | 4,5]，得到 [1,2 | 4,5,3,6]。左侧 [1] 与 [2] 已完成合并；右侧的二分切块先把 6 留在末尾，再将 [5 | 3] 旋转为 [3 | 5]，得到 [1,2,4,3,5,6]；最后处理两条记录的合并，交换 4、3，成为 [1,2,3,4,5,6]。数字没有被丢掉，各块的最终内部次序也没有被颠倒。

效率与存储

最坏	这个简单整体排序方案为 $O(n \log^2 n)$
单次合并	m 条记录最多 $O(m \log m)$ 移动，不能当作线性合并
最好	若每次先检查两段边界是否已有序，可为 $\Theta(n)$
额外空间	$O(\log n)$ 递归栈、 $O(1)$ 临时记录
稳定性	稳定；严格原地：计入栈后不算

为什么快，为什么慢

一次合并中，同一记录可能在多个递归层被旋转搬动。把这种 $O(m \log m)$ 合并放入整体归并排序，就比线性缓冲归并多出一个 $\log n$ 因子。

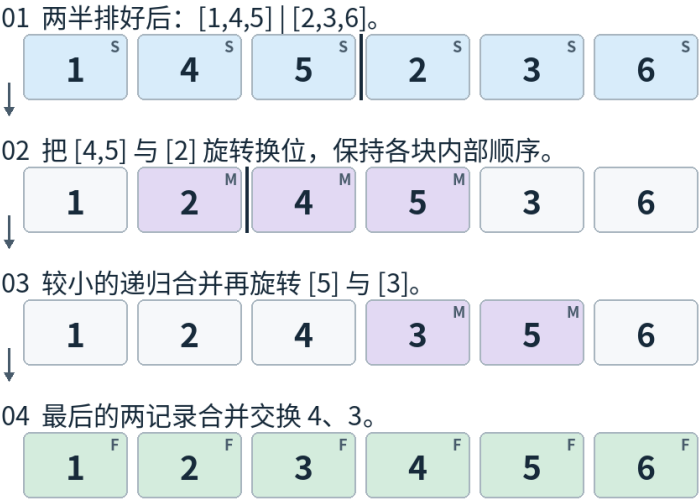
适合：不能申请线性缓冲区，又愿意接受额外搬动时。比普通归并慢的主要原因，是旋转会重复移动一些记录。

注意：这不是已经优化到 $O(n \log n)$ 的 WikiSort/GrailSort。实现时必须算清切块长度与新的中间位置，附录提供完整递归伪代码。

来源：[N05]

过程图 C31

旋转归并：交换块次序，再处理较小合并



C 正在比较 M 正在移动 S 区域内有序 F 最终归位

C31 旋转归并：交换块次序，再处理较小合并。与本条目的示例对应；标记含义见“基础”。

c08 希尔排序 Knuth 增量

名称／关系：Shellsort；缩小增量排序；本文指定 Knuth 增量

按逐渐缩小的间隔对交错子序列执行插入排序，先让元素大步移动，最后用间隔 1 完成排序。

具体怎么做

- 1. 按 $h=3h+1$ 生成 1、4、13、40.....，取小于 n 的合适最大间隔，再从大到小使用。
- 2. 对当前间隔 h ，让 i 从 h 到 $n-1$ 。保存 $A[i]$ ，沿着每次减 h 的位置向左检查。
- 3. 若 $A[j-h]$ 更大，就把它移到 $A[j]$ ，再令 j 减 h ；最后把保存的元素放到空位。
- 4. 一轮结束后，下标除以 h 余数相同的各个子序列都已有序。令 $h=floor(h/3)$ ，继续下一轮。
- 5. 必须完成 $h=1$ 的最后一轮。前面的大间隔能减少最后的插入工作，但某个间隔有序并不等于整个数组有序。

小例子：[8,7,6,5,4,3,2,1]，h=4 后为 [4,3,2,1,8,7,6,5]，h=1 后为 [1,2,3,4,5,6,7,8]；间隔 4 的子序列下标分别为 (0,4)、(1,5)、(2,6)、(3,7)。

效率与存储

最好	$\Theta(n \log n)$ ，Knuth 增量且输入已有序
平均或期望	不宣称 Knuth 增量的一般紧确平均界
最坏	所引 Knuth 增量实现为 $O(n^{3/2})$
额外空间	$\Theta(1)$
稳定性	不稳定
严格原地	是

为什么快，为什么慢

先用远距离插入减少大错位，再做最后的普通插入。增量序列会改变分析结果，本条上界针对 Knuth 增量，不能直接推广到所有希尔排序。

适合场景：适合需要代码简短、额外空间少的场景，但不提供通用 $O(n \log n)$ 的保证。

容易误解：希尔排序没有一张适用于所有增量的复杂度表；折半、Knuth、Pratt 等增量必须分别分析。

依据：[C08]

E 一般为平方级：小输入或特殊成本目标

这些方法在普通数组上可能产生平方级工作。例外必须看具体条件：接近有序时插入可以接近线性；Ford-Johnson 节省比较，但简单数组实现仍可能有平方级移动；循环排序优先节省写入。梳排序没有统一的 $n \log n$ 保证。

编号只标识条目，不表示名次。

C01 插入排序

名称／关系：直接插入排序

每次从未排序区取出一个元素，把它插入左侧有序区；需要消除的逆序对越少，工作量越小。

具体怎么做

1. 先把 $A[0]$ 看作有序区。依次令 i 从 1 走到 $n-1$ ，先把 $A[i]$ 保存为 key ，避免后面的移动覆盖它。
2. 从 $j=i-1$ 开始向左检查。只要 j 没越界且 $A[j]>key$ ，就把 $A[j]$ 复制到右边一格，再令 j 减一。
3. 遇到不大于 key 的元素，或已到数组开头，就把 key 写入 $A[j+1]$ 。此时从开头到 i 都已有序。
4. 重复上述过程，直到所有元素都加入有序区。相等元素不向右移动，才能保持原来的先后顺序。
5. 手算时在有序区后画一条分界线，每次插入结束就把线向右移一格。每移动一个较大元素，就消除了一个逆序对。

小例子： $[5,2,4,1] \rightarrow [2,5|4,1] \rightarrow [2,4,5|1] \rightarrow [1,2,4,5]$ ；插入 1 时，前面的三个元素都右移。

效率与存储

最好	$\Theta(n)$ ，输入已排序
平均或期望	$\Theta(n^2)$ ，互异键的均匀随机排列
最坏	$\Theta(n^2)$ ；更精确为 $\Theta(n+I)$ ， I 为逆序对数
额外空间	$\Theta(1)$
稳定性	稳定，只移动严格较大的元素
严格原地	是

为什么快，为什么慢

每次右移都消除一处“大数站在后面小数之前”的逆序。乱序少，移动就少；完全倒序会累积约 $1+2+\dots+(n-1)$ 次移动。

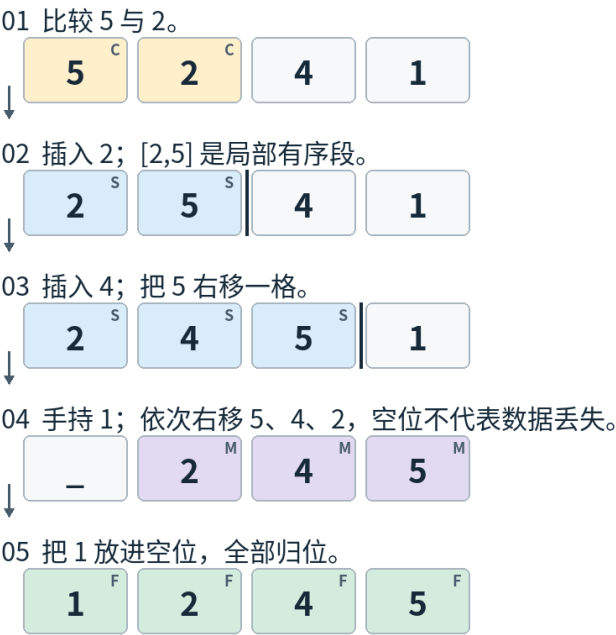
适合场景：适合小数组、接近有序的数据、向较小有序数组逐项加入元素，也常用作混合排序的小区间处理。

容易误解：先保存待插入元素，再移动其他元素。即使改用二分查找定位，数组中的搬移仍然存在。

依据：[C01]

过程图 C01

插入：有序前缀逐步增长



C 正在比较 M 正在移动 S 区域内有序 F 最终归位

C01 插入：有序前缀逐步增长。与本条目的示例对应；标记含义见“基础”。

C02 折半插入排序

名称／关系：二分插入排序；折半插入排序

用二分查找确定插入位置，再像普通插入排序一样移动元素；比较次数变少，不代表搬移次数也变少。

具体怎么做

- 1. 维持 A[0..i) 有序，并保存下一个元素 x=A[i]。二分查找只在这个左闭右开区间中进行。
- 2. 寻找第一个严格大于 x 的位置 p。检查中点 m：若 x<A[m]，右边界移到 m；否则左边界移到 m+1。
- 3. 两边界重合后得到 p，它位于已有相等元素之后。把 A[p..i) 从右向左逐个右移一格。
- 4. 将 x 写到 p，再处理下一个元素，直到整个数组有序。
- 5. 这里每轮都执行二分查找。若先检查 x 是否已经不小于前一个元素，就属于另一种优化，已排序输入的最好时间可降为线性。

完整小例子：从 [4,2 甲,3,2 乙] 开始，已排前缀先只有 [4]。二分查找确定 2 甲 应放在 4 前，把 4 右移，得到 [2 甲,4|3,2 乙]。接着为 3 找到 2 甲 与 4 之间的空隙，把 4 右移，得到 [2 甲,3,4|2 乙]。

处理 2 乙 时，要找第一个严格大于 2 的键，也就是 3。先右移 4，再右移 3，把 2 乙 放在 2 甲 后面，得到 [2 甲,2 乙,3,4]。二分减少的是找位置时的比较，并没有消除搬动记录的成本。

效率与存储

最好	$\Theta(n \log n)$ ，每轮无条件二分
平均或期望	$\Theta(n^2)$ 时间；随机互异键，比较为 $\Theta(n \log n)$
最坏	$\Theta(n^2)$ 时间；比较为 $\Theta(n \log n)$
额外空间	$\Theta(1)$
稳定性	稳定，插在相等元素之后
严格原地	是

为什么快，为什么慢

二分不断把搜索范围减半，因此找位置很快；但紧密数组中的插入仍可能搬动后面整段，比较次数减少不等于总工作量低于平方级。

适合场景：适合比较代价高但规模较小的数据，也常用于把较短的自然有序段扩展到指定长度。

容易误解：若插到第一个大于等于 x 的位置，稳定性会被破坏；不能只根据二分查找就宣称总时间是 $O(n \log n)$ 。

依据：[C02]

C03 选择排序

名称／关系：简单选择排序；直接选择排序

反复从剩余元素中找出最小值，放到下一个应输出的位置；原有顺序通常无法减少比较次数。

具体怎么做

1. 让 i 指向未排序区的第一个位置，初始为 0； i 左边都已放好。
2. 先记 $\text{min}=i$ ，再检查所有 $j>i$ 的位置。若 $A[j]<A[\text{min}]$ ，就把 min 更新为 j 。
3. 扫描完整个未排序区后，交换 $A[i]$ 和 $A[\text{min}]$ ；若本来就是同一位置，可省略交换。
4. i 加一，继续选择剩余元素的最小值。只剩最后一个元素时就结束。
5. 手算时既记录最小值，也记录被交换到后方的元素。普通选择排序不稳定，原因正是这种跨越多项的交换。

小例子：[3 甲,3 乙,1] → 交换 1 与 3 甲 → [1,3 乙,3 甲]；数值有序，但两个 3 的顺序反了。

效率与存储

最好	$\Theta(n^2)$
平均或期望	$\Theta(n^2)$ ，不依赖输入分布
最坏	$\Theta(n^2)$ ；键比较恰为 $n(n-1)/2$
额外空间	$\Theta(1)$
稳定性	普通交换版不稳定
严格原地	是

为什么快，为什么慢

第一次找最小值要看近 n 条，下一次近 $n-1$ 条，依次累积成平方级。每轮只交换一次，省的是写入，不是反复扫描中的比较。

适合场景：适合作为入门和对照算法；真正交换至多 $n-1$ 次，在写入远比较昂贵时有一定价值。

容易误解：必须扫描整个剩余区间后才交换；每发现一个较小元素就交换，会变成写入更多的另一种交换排序。

依据：[C03]

过程图 C03

选择：一次交换可能改变相等记录的先后

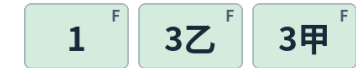
01 在剩余区间找最小值 1。



02 交换 3甲 与 1，首先确定第一个位置。



03 数值已排好，但 3甲 与 3乙 的原始顺序改变了。



C 正在比较 M 正在移动 S 区域内有序 F 最终归位

C03 选择：一次交换可能改变相等记录的先后。与本条目的示例对应；标记含义见“基础”。

C28 稳定选择排序：移动插入版

名称／关系：用搬移保持稳定的选择排序

仍然反复寻找剩余部分的最小值，但找到后不直接交换，而是像插入排序一样腾出位置。这多花了移动成本，却能保留相等记录的原始先后。

具体做法

- 1. 准备填下标 i 时，扫描右侧未排部分，记住最小键第一次出现的位置。只有遇到严格更小的键，才更新这个位置。
 - 2. 暂存选中的整条记录，把 i 到它原位置之前的记录依次右移一格，再把暂存记录放到 i 。
 - 3. i 向右移动，重复上述过程。每轮结束后，左侧前缀都是已按稳定顺序归位的最小记录。
- 跟着做：从 [3 甲, 3 乙, 1] 开始。最小值是 1，先拿在手里；把 3 乙 右移，再把 3 甲 右移，暂时成为 [空位, 3 甲, 3 乙]。最后将 1 填入，得到 [1, 3 甲, 3 乙]。
- 跟着做：下一轮选到的最小值是先出现的 3 甲，不必移动。最终仍是 [1, 3 甲, 3 乙]。普通交换式选择排序则会得到 [1, 3 乙, 3 甲]，两个 3 的顺序反了。

效率与存储

最好、平均、最坏比较次数	均为 $\Theta(n^2)$
移动量	有序输入 $O(n)$ ，最坏 $\Theta(n^2)$
额外空间	$O(1)$ ；稳定性：稳定；严格原地：是

为什么快，为什么慢

寻找最小值的扫描仍是平方级；用整段移动代替一对交换换来了稳定性，也可能让记录移动同样达到平方级。

适合：理解稳定性为何取决于具体动作，而不仅取决于算法名称。大规模数据通常仍应选更高效的稳定方法。

注意：必须取最小值的第一次出现；若取最后一个相等最小值，即使采用移动插入，也可能破坏稳定性。

来源：[C03]

C29 双向选择排序

名称／关系：双向选择；双端选择；最小最大选择

一次扫描同时找出最小值和最大值，分别送到未排区间的左右两端。每轮归位两个位置，但寻找它们仍然需要扫描剩余数据。

具体做法

- 1. 用 $L..R$ 表示尚未排好的区间。完整扫描一遍，记下最小值位置 $imin$ 和最大值位置 $imax$ 。
- 2. 先交换 $A[L]$ 与 $A[imin]$ 。若 $imax$ 原本等于 L ，必须把 $imax$ 改成 $imin$ ，因为最大值已经跟着这次交换移动了。

3. 再交换 $A[R]$ 与 $A[i_{\max}]$ 。L 加一，R 减一；当 $L \geq R$ 时结束。

跟着做：对 $[5, 2, 4, 1, 3]$ ，最小值 1 在下标 3，最大值 5 在下标 0。先交换最小值与左端，得到 $[1, 2, 4, 5, 3]$ 。最大值位置随之改为 3，再把它送到右端，得到 $[1 | 2, 4, 3 | 5]$ 。

跟着做：中间的 $[2, 4, 3]$ 中，2 已在左端；交换 4 与 3，得到 $[1, 2, 3, 4, 5]$ 。虽然一轮确定两个位置，剩余区间的扫描成本仍会累积成平方级。

效率与存储

最好、平均、最坏	完整扫描版均为 $\Theta(n^2)$ 比较
交换次数	$O(n)$ ；额外空间： $O(1)$
稳定性	不稳定；严格原地：是

为什么快，为什么慢

每轮固定两端，轮数确实减少，但扫描区间仍构成等差累加，改变的是常数，不是平方级这一增长类别。

适合：减少普通选择排序的完整扫描轮数。常数可能降低，但不会从平方级变成 $n \log n$ 级。

注意：第一次交换后忘记修正最大值下标，是很常见的错误。“双向选择”与双向冒泡不是同一方法。

来源：[C03]

C30 简单互换排序：依次比较所有后续位置

名称／关系：互换排序；exchange sort 一词有不同用法

固定一个位置，把它与后面每个位置比较；只要后面出现更小的数字，就立即交换。不同资料对“交换排序”的叫法并不一致，所以本条明确指这一种双循环操作。

具体做法

- 1. 让 i 从 0 走到 $n-2$ 。每轮让 j 从 $i+1$ 扫到 $n-1$ 。
- 2. 只要 $A[j] < A[i]$ ，就立即交换。因而在本轮任意时刻， $A[i]$ 都是已经检查过的这些数字中最小的。
- 3. 扫描完所有后续位置后， i 才真正归位，再处理 $i+1$ 。它与选择排序的区别在于：不等到扫描完再交换一次，而是途中可能交换很多次。

跟着做：从 $[3, 2, 1]$ 开始。 $i=0$ 时，先比较 3 与 2，交换得 $[2, 3, 1]$ ；再比较当前位置的 2 与 1，交换得 $[1, 3, 2]$ 。此时下标 0 已归位。

跟着做： $i=1$ 时，比较 3 与 2，交换得 $[1, 2, 3]$ 。这里比较的是一个位置与所有后续位置，不是冒泡排序的相邻位置。

效率与存储

最好、平均、最坏比较次数	均为 $\Theta(n^2)$
最坏交换次数	$\Theta(n^2)$ ；额外空间： $O(1)$
稳定性	不稳定；严格原地：是

为什么快，为什么慢

每一对先后位置都要比较，不管原本有多少顺序。立即交换可能增加许多写入，却不会减少这套固定的比较安排。

适合：分清“遇到更小的就交换”“找完最小值再交换”和“只交换相邻逆序”三种不同操作。

注意：NIST 把 exchange sort 作为冒泡排序的别名，某些代码库却用它指本方法。查资料时应看执行步骤，不能只凭名字认定算法相同。

来源：[N03] [N04]

C04 冒泡排序 提前结束版

名称／关系：下沉排序；NIST 中 exchange sort 可指冒泡排序

通过相邻交换把较大元素向右送；一整轮没有交换，就说明剩余区间已经有序。

具体怎么做

- 1. 令 $end=n-1$ 。每轮先把 swapped 设为假，再令 j 从 0 扫描到 $end-1$ 。
- 2. 比较 $A[j]$ 和 $A[j+1]$ ，只有左边严格较大时才交换，并把 swapped 设为真。
- 3. 一轮结束后，当前区间中的最大元素已到 $A[end]$ ，令 end 减一，不再检查这个位置。
- 4. 若整轮没有交换就直接结束；否则继续，直到 $end=0$ 。也可以记录最后交换位置，进一步缩短下一轮。

小例子： $[3,1,2] \rightarrow$ 比较 3 与 1 得 $[1,3,2] \rightarrow$ 比较 3 与 2 得 $[1,2,3]$ ；下一轮无交换，结束。

效率与存储

最好	$\Theta(n)$ ，已排序且启用提前结束
平均或期望	$\Theta(n^2)$ ，随机互异键
最坏	$\Theta(n^2)$
额外空间	$\Theta(1)$
稳定性	稳定，只交换严格逆序的相邻项

严格原地	是
------	---

为什么快，为什么慢

相邻交换一次只能让记录前进一格，远距离错位就需要许多次交换。已经有序时无需交换，提前停止才会明显省事。

适合场景：适合讲解相邻逆序对和提前终止；日常数组排序中，插入排序通常是更实用的简单基线。
容易误解：去掉无交换检测后，已排序输入仍需 $\Theta(n^2)$ ；位于右端的小元素每次正向扫描只能向左走一格。

依据：[C04]

过程图 C04

冒泡：比较邻居，将本轮最大值送到末尾



C 正在比较 M 正在移动 S 区域内有序 F 最终归位

C04 冒泡：比较邻居，将本轮最大值送到末尾。与本条目的示例对应；标记含义见“基础”。

C05 鸡尾酒排序 双向冒泡排序

名称／关系：摇摆排序；双向冒泡排序；鸡尾酒排序

交替向右和向左冒泡，让左侧大元素与右侧小元素都能在一轮中移动较远。

具体怎么做

- 1. 用 [left,right] 表示当前区间，初始覆盖整个数组。先从左向右扫描，交换逆序的相邻元素。
- 2. 若正向扫描没有交换，就结束；否则右端的最大值已就位，将 right 减一。
- 3. 再从 right 向 left+1 反向扫描，仍然只在左值大于右值时交换。
- 4. 若反向扫描没有交换，就结束；否则左端最小值已就位，将 left 加一，再处理缩小后的区间。

小例子：[2,3,4,1] → 正向得 [2,3,1,4] → 反向得 [1,2,3,4]；末尾的 1 在反向扫描中连续左移。

效率与存储

最好	$\Theta(n)$ ，已排序且提前结束
平均或期望	$\Theta(n^2)$ ，随机互异键
最坏	$\Theta(n^2)$
额外空间	$\Theta(1)$
稳定性	稳定，只交换严格逆序对
严格原地	是

为什么快，为什么慢

相邻交换一次只能让记录前进一格，远距离错位就需要许多次交换。已经有序时无需交换，提前停止才会明显省事。

适合场景：适合与单向冒泡比较，尤其能说明末尾小元素为什么受益于反向扫描。

容易误解：两个方向都使用同一个升序比较条件；扫描方向变了，并不意味着排序方向也要改变。

依据：[C05]

c06 侏儒排序

名称／关系：侏儒排序；有些资料也称 stupid sort

用一个游标检查相邻元素：有序就向右走，交换逆序对后向左退；它把插入排序的两层循环合成了一次往返行走。

具体怎么做

1. 从 $i=1$ 开始；不足两个元素时直接结束。
2. 若退到了 $i=0$ ，就回到 $i=1$ ；否则比较 $A[i-1]$ 与 $A[i]$ 。
3. 若前一个不大于后一个，就令 i 加一；若两者逆序，就交换，再令 i 减一。
4. 不断重复，直到 $i=n$ 。一个元素左移到正确位置后，游标会重新向右走，寻找下一处逆序。

小例子：[3,1,2]， $i=1 \rightarrow$ 交换得 [1,3,2]， $i=0 \rightarrow$ 向右走 $\rightarrow$ 交换得 [1,2,3] $\rightarrow$ 走到末尾。

效率与存储

最好	$\Theta(n)$ ，输入已有序
平均或期望	$\Theta(n^2)$ ，随机互异键
最坏	$\Theta(n^2)$ ；有 I 个逆序对时为 $\Theta(n+I)$

额外空间	$\Theta(1)$
稳定性	稳定，不交换相等元素
严格原地	是

为什么快，为什么慢

相邻交换一次只能让记录前进一格，远距离错位就需要许多次交换。已经有序时无需交换，提前停止才会明显省事。

适合场景：适合学习循环不变式，以及局部交换与插入排序的联系；通常不是生产环境的首选。

容易误解：读取 $A[i-1]$ 前必须先处理 $i=0$ 的情况；相等元素不应交换，否则增加工作并破坏稳定性。

依据：[C06]

C07 梳排序

名称／关系：梳排序；带间隔的交换方法

先比较相距较远的元素，再逐渐缩小间隔到 1，以减轻小元素在冒泡排序中缓慢向左移动的问题。

具体怎么做

- 1. 先令 $gap=n$ 、swapped 为真。只要 $gap>1$ ，或上一轮发生了交换，就继续。
- 2. 每轮开始先令 $gap=\max(1,\text{floor}(gap/1.3))$ ，并清除交换标记。
- 3. 依次比较 $A[i]$ 和 $A[i+gap]$ ，其中 i 从 0 到 $n-gap-1$ ；左值较大就交换并记录。
- 4. 间隔大于 1 时，每个间隔只扫描一轮。 $gap=1$ 后继续做相邻扫描，直到整轮没有交换。

小例子：[4,1,3,2]，间隔依次为 3、2、1：间隔 3 得 [2,1,3,4]，间隔 2 不变，间隔 1 得 [1,2,3,4]。

效率与存储

最好	$\Theta(n \log n)$ ，此几何间隔循环处理有序输入
平均或期望	不对该实现宣称统一的紧确平均界
最坏	$O(n^2)$ 上界，未宣称紧确 Θ 界
额外空间	$\Theta(1)$
稳定性	不稳定，远距离交换可能跨过相等项
严格原地	是

为什么快，为什么慢

大间隔让错位记录一次跨过更远距离，比冒泡逐格移动有利；但最后仍须做间隔为一的整理，不能因此保证一般输入都是 $n \log n$ 级。

适合场景：适合低内存的简单实验；大间隔常能改善冒泡表现，但不能获得归并排序那样的最坏时间保证。

容易误解： $gap > 1$ 时无交换不能说明整体有序。梳排序每个间隔只做一轮；希尔排序会把各个间隔子序列完整排好。

依据：[C07]

X07 串排序

反复从未处理序列抽取一条递增子序列，再并入已排序结果。

具体怎么做

- 1. 准备剩余元素链表，以及初始为空的有序结果链表。
- 2. 取走剩余链表的第一个元素，作为新串的起点。
- 3. 继续向右扫描；遇到不小于串末尾的元素就摘下，并追加到串末尾。
- 4. 暂时跳过的元素留在原处，保持彼此的相对顺序。
- 5. 将新串与结果归并，每次取较小的表头；相等时先取较早进入结果的记录。
- 6. 重复抽串和归并，直到剩余链表为空。

小例子：[4,1,3,2,5] 依次抽出 [4,5]、[1,3]、[2]，逐次归并得到 [1,2,3,4,5]。

效率与存储

最好	一次即可抽完时 $\Theta(n)$
平均或期望	未指定输入分布，给出 $O(n^2)$ 上界
最坏	不同键逆序输入时 $\Theta(n^2)$
额外空间	另建链表时 $O(n)$ ；已有节点可改指针复用
稳定性	遵循上述抽取及同键规则时稳定
严格原地	普通数组加输出表实现并非原地

为什么快，为什么慢

长递增串提供可利用的顺序；不利输入可能每次扫描只取出一条，导致剩余大部分记录一遍又一遍被扫描，累积成平方级。

适合场景：适合手算，展示利用已有递增结构的归并思想，尤其适用于链表。

容易误解：“串”不要求在输入中连续；若反复从紧凑数组中间删除元素，会产生额外移动。

依据：[X-STRAND]

X08 循环排序 比较版本

名称／关系：循环排序；此处靠比较计算目标位置

通过比较计算最终排名，沿置换环搬运元素，重点减少写入次数而非比较次数。

具体怎么做

1. 依次选择起点 s ，把该位置元素 x 暂存到一个变量中。
2. 统计 s 后方比 x 小的元素个数；目标位置为 s 加该数量。
3. 若尚未搬运且目标就是 s ，说明该位置已正确，直接换下一个起点。
4. 若目标槽已有相同键，就向后跳过这些槽；写入 x ，并把被挤出的值作为新的 x 。
5. 继续在同一后缀计算新 x 的排名并搬运，直到写回 s ，完成这一环。
6. 推进 s ，重复处理其他环；比较负责找位置，写入只在填入最终元素时发生。

小例子：[3,1,2]：暂存 3 写入位置 2；挤出的 2 写入位置 1；再把 1 写回位置 0，得到 [1,2,3]。

效率与存储

最好	即使已有序也为 $\Theta(n^2)$
平均或期望	$\Theta(n^2)$
最坏	$\Theta(n^2)$
额外空间	$O(1)$
稳定性	不稳定
严格原地	是

为什么快，为什么慢

沿循环把记录送到目的地，有利于减少指定条件下的写入；但通过统计较小记录来找位置，仍要扫描后缀。省写入与省总时间不是同一目标。

适合场景：主数组写入代价较高时有价值，能避免把同一条记录沿途反复交换。

容易误解：必须跳过相同键，否则可能在相同元素之间来回循环；它不同于直接寻址的线性版本。

依据：[X-CYCLE-CODE]

过程图 X08

循环排序：被挤出的记录继续拿在手中

01 3 前面应有两个更小值，目的地下标是 2。



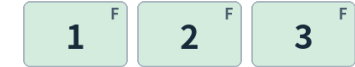
02 把 3 写到下标 2，手持被挤出的 2。



03 把 2 写到下标 1，手持被挤出的 1。



04 把 1 放回起始空位，这个环闭合。



C 正在比较 M 正在移动 S 区域内有序 F 最终归位

X08 循环排序：被挤出的记录继续拿在手中。与本条目的示例对应；标记含义见“基础”。

C25 Ford Johnson 归并插入排序

名称／关系：Ford-Johnson 排序；归并插入排序

先通过两两比较建立上界，再递归排好每对中的较大项，最后按 Jacobsthal 次序插入较小项；优化目标是最坏比较次数，而非数组搬移次数。

具体怎么做

1. 比较次数分析假设键互异。把元素两两配对，每对只比较一次，较小者记为 b ，较大者记为 a ；奇数个元素时，把未配对项暂存。
2. 递归用同一算法排列所有 a ，并始终保留它与 b 的配对关系。重新编号，使 $a_1 < a_2 < \dots < a_m$ 且 $b_i < a_i$ ，其中 $m = \text{floor}(n/2)$ ；不足两个元素时结束递归。
3. 建立已有序的主链 $[b_1, a_1, a_2, \dots, a_m]$ ，放入 b_1 不需额外比较。待插入项是 $b_2, \dots, b_m$ ；若有未配对项，将其记为 b_{m+1} 。
4. 按分组倒序插入：先 b_3 、 b_2 ，再 b_5 、 b_4 ，再 b_{11} 、 b_{10} 、 $\dots$ 、 b_6 ，再 b_{21} 、 $\dots$ 、 b_{12} ，依次继续，不存在的编号跳过。分组端点 1、3、5、11、21.....来自 Jacobsthal 数列： $J_0=0$ 、 $J_1=1$ 、 $J_k=J_{k-1}+2J_{k-2}$ 。
5. 插入 b_i 时，只在当前主链中严格位于其搭档 a_i 之前的前缀里二分，因为配对时已经证明 $b_i < a_i$ 。未配对项则搜索整条主链。插入后更新搭档的位置或引用。
6. 待插入项用完即结束。该次序让每次搜索区间不超过某个合适的 2^k-1 项上限，从而最多比较 k 次。后来插入的元素也算在后续搜索区间里，搬移这些元素的代价必须另计。

小例子：五项 [4,1,5,2,3]：两次配对比较得到 $(b_1,a_1)=(1,4)$ 、 $(b_2,a_2)=(2,5)$ ，剩余 $b_3=3$ 。再比较 4 与 5 一次，建立 [1,4,5]。先插 b_3 ：将 3 与 4、1 比较，共两次，得 [1,3,4,5]。再把 b_2 插在搭档 5 之前：将 2 与 3、1 比较，共两次，得 [1,2,3,4,5]。总共 $2+1+2+2=7$ 次，达到 $\text{ceil}(\log_2(5!))=7$ 的下界。

效率与存储

最好	实际时间取决于链的存储方式，不宣称统一的紧确最好时间界
平均或期望	互异键均匀随机排列时，比较为 $\Theta(n \log n)$ ；这不是总时间保证
最坏	最坏比较次数：令 k 从 1 到 n ，把 $\text{ceil}(\log_2(3k/4))$ 的结果相加；朴素连续数组插入 需 $\Theta(n^2)$ 搬移和时间
额外空间	常见主链和待插入链用 $O(n)$ 元素或引用，另加 $O(\log n)$ 递归栈
稳定性	标准形式不稳定；可把原始位置作为相等键的次级键来保证输出稳定
严格原地	常见的辅助链实现不是原地排序

为什么快，为什么慢

专门安排插入次序，让搜索窗口长度尽量适合二分，省的是比较次数；数组搬动与维护配对关系仍可能占据大部分实际时间。

适合场景：适合研究少比较排序，以及比较成本远高于其他操作的场景。五元素时比较次数最优，但不能推广为所有 n 都最优，也不代表实际运行最快。

容易误解：改成从左到右依次二分插入，就失去了 Ford-Johnson 的比较次数保证。必须保留配对关系、把已知搭档排除在搜索外，并单独计算数组搬移或链表遍历成本。

依据：[C32] [C33] [C34]

D28 秩排序／枚举排序 (Rank / enumeration sort)

名称／关系：秩排序；枚举排序；比较计数排序

直接计算每条记录前面应该有多少条记录，这个数量就是它的最终下标。每条记录的计算可以并行，但整体仍需大量两两比较。

具体做法

1. 给记录保留原始下标。对第 i 条记录，先令 $\text{rank}=0$ 。
2. 检查所有 $j \neq i$ ：若 $\text{key}[j] < \text{key}[i]$ ，计数加一；若键相等但 $j < i$ ，也加一。这样，相同键会按照原始先后获得不同的名次。
3. 把第 i 条记录写入 $\text{output}[\text{rank}]$ 。所有名次恰好覆盖 $0..n-1$ ，因此不会发生两条记录争用同一输出位置。

跟着做：对 [3 甲,1,3 乙,2]，3 甲 前面应有 1、2，名次为 2；1 的名次为 0；3 乙 前面应有 3 甲、1、2，名次为 3；2 的名次为 1。

跟着做：于是分别写入 output[2]=3 甲、output[0]=1、output[3]=3 乙、output[1]=2。按输出位置读取，就得到 [1,2,3 甲,3 乙]。它靠计算目的地形成顺序，而不是反复交换来调整顺序。

效率与存储

顺序执行的最好、平均、最坏	均为 $\Theta(n^2)$ 比较
额外空间	$\Theta(n)$ 输出／名次存储；使用上述相等规则时稳定
并行总工作量	$\Theta(n^2)$ ； $O(n^2)$ 个处理器配合树形求和可达 $O(\log n)$ 深度
严格原地	这个简单实现不是

为什么快，为什么慢

n 个名次各自检查 n-1 条其他记录。再多处理器也只是让它们同时做，不会把平方级比较总量变成线性。

适合：理解总工作量与并行深度的区别，或研究固定规模数据的高度并行名次计算。

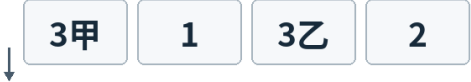
注意：若不使用原始下标打破平局，相等键会算出同一位置，相互覆盖。并行完成得快，并不代表总工作量只有线性。

来源：[N08]

过程图 D28

秩排序：直接计算每条记录的目的地

01 输入，原始下标为 0、1、2、3。



02 各条记录的名次；不是中间的数值数组。



03 写到各自名次位置，相等时用原始下标打破平局。



C 正在比较 M 正在移动 S 区域内有序 F 最终归位

D28 秩排序：直接计算每条记录的目的地。与本条目的示例对应；标记含义见“基础”。

F 额外结构：位移有限与字符串

元素离最终位置有距离上限时，问题已经改变。字符串还要计算查看了多少字符、重复检查了多少公共前缀，不能只数记录比较。本组另起比较条件，排在后面不表示比 E 组慢。

编号只标识条目，不表示名次。

D30 偏移不超过 k 的小堆排序

名称/关系：k 近有序 / k 位移数组的最小堆排序

若每条记录距离最终位置都不超过 k，那么下一个最小值一定藏在接下来的 k+1 个候选中。只需维护一个小堆，不必把全部数据同时装进堆。

具体做法

- 1. 先确认偏移上界 k 确实成立。把前 min(n,k+1) 条记录放入最小堆。
- 2. 取出堆中最小记录，追加到输出；再读入下一条尚未处理的记录，放进堆。不断重复，直到输入读完。
- 3. 最后依次取空堆。每取出一条，就确定一个输出位置。需要稳定性时，把“键、原始下标”共同作为堆的比较依据。

跟着做：对 [2,1,4,3,5]，已知 k=1。先放入 [2,1]，输出 1，再加入 4，候选为 {2,4}；输出 2，加入 3，候选为 {3,4}；输出 3，加入 5，候选为 {4,5}。

跟着做：输入已读完，依次输出 4、5，得到 [1,2,3,4,5]。为什么还没读到末尾就敢输出 1？因为 k=1 保证真正排第一的记录，不可能一开始位于下标 1 之后。

效率与存储

时间	$O(n \log(k+2))$ ，这样写也涵盖 $k=0$
额外工作空间	$O(k+1)$ ，不含另行存放的输出
稳定性	加入原始下标可稳定；k 随输入增长时不是严格常数空间

为什么快，为什么慢

堆中只有 k+1 个候选，修复路径取决于 $\log(k+1)$ ，而不是 $\log n$ 。能够在读完输入前确认输出，完全依赖那个已知的偏移保证。

适合：已知乱序幅度有上界的数据，例如只可能迟到少数位置的记录流。k 很小时，小堆远比完整大堆便宜。

注意：不能只因数据“看起来差不多有序”就使用。对 [2,3,1] 错把 k 当成 1，会过早输出 2，导致失败。

来源：[N10]

过程图 D30

有界偏移：k=1 时只需两个候选

01 已知每条记录离最终位置至多一格。



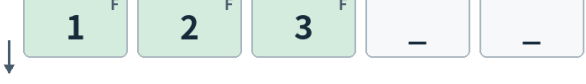
02 堆 {1,2} 输出 1，再读 4；候选变为 {2,4}。



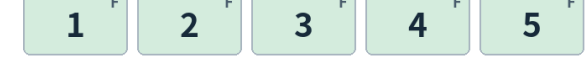
03 输出 2，再读 3；候选为 {3,4}。



04 输出 3，再读 5；候选为 {4,5}。



05 输入读完，依次取空堆。



C 正在比较 M 正在移动 S 区域内有序 F 最终归位

D30 有界偏移：k=1 时只需两个候选。与本条目的示例对应；标记含义见“基础”。

D10 爆发式字典树排序

把字符串保存在较小的字典树桶中；桶过大时，把它展开为下一层字符节点。

具体怎么做

- 1. 选择桶大小阈值，规定字符顺序，并设置明确的结束标记。
- 2. 根据字符串的前缀沿字典树前进，把字符串引用追加到到达的桶中。
- 3. 桶超过阈值时，把桶替换为树节点，按下一字符把其中引用重新分配到子桶。
- 4. 继续展开过大的非终止桶；在节点结束的字符串不再检查后续字符。
- 5. 用明确指定的字符串排序方法整理各桶尚未检查的后缀，再按字符顺序遍历树并输出各桶。

完整小例子：使用定长三字符数字键 [213,211,212,304]，教学上把桶容量设为 2。先装入 213、211，加入 212 后，公共前缀桶超过容量，于是“爆开”为按下一字符分流的结构。三者共用前缀 21，剩余后缀 3、1、2 再按顺序分到 1、2、3。

304 从根的 3 分支前进，不走 2 分支。按分支序与局部排好的后缀桶读取，得到 [211,212,213,304]。实际爆发阈值通常大得多，这里只是为了让桶变成字典树节点的过程可见；普通字符串 cat、car、can 共用 ca，也是相同道理。

效率与存储

最好	至少 $\Omega(n)$ ，取决于读取字符数和桶内收尾
平均或期望	取决于分布、前缀与桶内算法，不能笼统写 $\Theta(n)$
最坏	遍历、重新分配与桶内排序成本之和，不能只用 n 概括
额外空间	需要字符串引用、树节点及桶；通常为 $O(n)$ 引用加字典树存储
稳定性	取决于桶插入与收尾算法，不保证稳定
严格原地	标准字典树加桶版本不是原地排序

为什么快，为什么慢

小后缀桶让相关键在内存中靠近；只拆开拥挤的桶，避免为每一个稀疏前缀都提前建立很大的字典树。

适合场景：适用于大型字符串集合，尤其在缓存局部性与重复前缀检查开销较大时。

容易误解：Burstsort 包含指针型、复制型等变体；论文的实测加速比不是普遍的复杂度保证。

依据：[DIST13] [DIST14]

D11 多键快速排序

名称／关系：三路字符串快速排序；三路基数快速排序

根据一个字符做三路划分，只在等于枢轴字符的组内推进字符位置。

具体怎么做

1. 在位置 d 从某个字符串选取枢轴字符；字符串结束标记小于所有真实字符。
2. 做三路划分，形成当前字符小于、等于、大于枢轴的三个组。
3. 较小组和较大组仍在同一字符位置 d 递归排序。
4. 相等组在 $d+1$ 位置递归；若枢轴是结束标记则不再递归。
5. 组内只有零个或一个字符串时停止；打乱输入或随机选枢轴可减少连续坏划分。

完整小例子：对定长键 [213,211,102]，取 213 的首字符 2 为字符枢轴。第一次分区为 [102 | 213, 211 | 空]：一组当前字符更小，一组当前字符相等，没有更大组。

单独的 102 已完成。只让等值组前进到下一位：两者第二位同为 1，再前进；第三位分别为 3、1，分区后得到 [211,213]。完整输出为 [102,211,213]。小于组与大于组仍停留在当前字符深度，只有等值组前进一位。

效率与存储

最好	相同空串或有界长度相同键可为 $\Theta(n)$
----	----------------------------

平均或期望	随机枢轴下保守字符操作上界 $O(n \log n + nw)$
最坏	字符操作上界 $O(n^2 + nw)$, w 为最大键长
额外空间	栈期望 $O(\log n + w)$, 最坏 $O(n + w)$
稳定性	交换式划分不稳定
严格原地	引用原地移动, 仍需递归栈

为什么快，为什么慢

同一等值组的公共前缀一起跳过，不必在许多整串比较中从头重读；但某个字符深度上的坏分区仍可能很昂贵。

适合场景：适用于有共同前缀的变长字符串，避免每次完整比较时重复检查已知前缀。

容易误解：只有相等组推进字符位置，其他两组不推进；这与直接比较整个字符串的三路快排不同。

依据：[DIST15] [DIST16]

G 外部存储与分布式数据

这里往往最该节省的是磁盘块传输或网络流量。顺序读写、归并路数、分区均衡和缓冲布局比单次比较更重要。置换选择只是生成有序段的部件，完整外部排序还需要归并。

编号只标识条目，不表示名次。

D14 外部多路归并排序

先生成能放入内存的有序段，再利用块缓冲区一次归并多个有序段。

具体怎么做

1. 为缓冲区留出空间；每次读取最多 M 条记录，在内存中排序并写成有序段，直至读完输入。
2. 选择归并路数 f ，使 f 个输入块和一个输出块可放入内存；每块含 B 条记录。
3. 打开下一组至多 f 个有序段，把每段尚未消费的最小记录放入最小堆。
4. 取出堆顶写入输出缓冲区，再用同一段的下一条记录替换。
5. 顺序补充输入缓冲区、写出已满输出缓冲区；重复归并轮次，直到只剩一段。

完整小例子：假设内存一次只能排序两条记录，文件是 $[7,1,6,2,5,3]$ 。先读 $[7,1]$ ，排成 $[1,7]$ ，写为有序段 R_1 ；再处理 $[6,2]$ ，写 $R_2=[2,6]$ ；最后处理 $[5,3]$ ，写 $R_3=[3,5]$ 。每段内部有序，不表示把三段直接连接就已全局有序。

若可为三路输入分配缓冲，比较各段段首：先取 1，从原段补入 7；取 2，补入 6；取 3，补入 5；再依次取 5、6、7。输出块顺序写入文件，最终为 $[1,2,3,5,6,7]$ 。若缓冲不足，就每轮少合并几路，多做几轮。

效率与存储

最好	固定初始 r 段时，块 I/O 为 $\Theta(\text{ceil}(n/B)(1 + \text{ceil}(\log_f r)))$
平均或期望	块 I/O 为 $\Theta(\text{ceil}(n/B)(1 + \text{ceil}(\log_f r)))$ ；分块生成时 $r = \text{ceil}(n/M)$
最坏	块 I/O 为 $\Theta(\text{ceil}(n/B)(1 + \text{ceil}(\log_f r)))$ ；比较排序 CPU 工作量 $O(n \log n)$
额外空间	$O(M)$ 内存及 $O(n)$ 临时外存
稳定性	段内稳定排序且归并等键顺序全局一致时稳定
严格原地	不是，需要外部目标空间

为什么快，为什么慢

顺序读写整块数据，比零散地访问磁盘更合适；每轮合并的路数越多，整份文件被完整读写的次数通常越少。

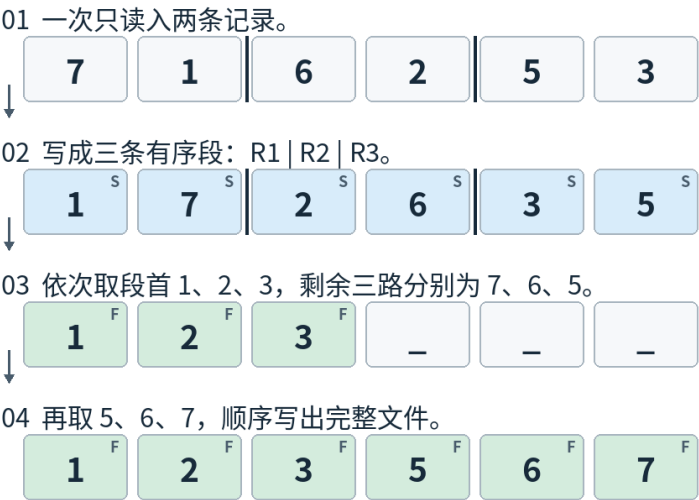
适合场景：适用于大于内存的文件、数据库 ORDER BY 和批量处理；假设 $f \geq 2$ ，块传输与 CPU 操作分开计算。

容易误解：不缓冲、逐条读磁盘会严重损害性能；内存中排好的小块仅是初始有序段。

依据：[DIST21] [DIST22]

过程图 D14

外部归并：各段有序不等于整份文件有序



C 正在比较 M 正在移动 S 区域内有序 F 最终归位

D14 外部归并：各段有序不等于整份文件有序。与本条目的示例对应；标记含义见“基础”。

D15 置换选择 有序段生成

名称／关系：置换选择是部件，不是已经完成的文件排序

用堆延长当前输出有序段，并暂存那些会破坏当前顺序的新记录。

具体怎么做

- 1. 先用最多 M 条输入记录建立最小堆，并开始一个新的输出有序段。
 - 2. 取出并输出最小的活动记录，把其键记为 last。
 - 3. 读取下一条输入；若键不小于 last，就放进活动堆。
 - 4. 否则将其冻结，留给下一段；活动堆缩小时释放的空间可存冻结记录。
 - 5. 活动堆为空时结束本段，激活冻结记录、重新建堆并开始下一段；最后还须归并所有段。
- 小例子：M=2、输入 [2,4,3,1] 时，输出 2 后接纳 3，输出 3 后冻结 1，再输出 4；得到 [2,3,4] 与 [1]。

效率与存储

最好	递增输入可能只生成一段，长度 n
----	------------------

平均或期望	标准随机输入假设下，期望段长约为 2M
最坏	递减输入可仅有 M 长的段；堆工作量 $O(n \log M)$
额外空间	$O(M)$ 内存、I/O 缓冲区及输出段
稳定性	不自动稳定；需要时用原输入位置处理等键
严格原地	作为外部段生成流水线不是原地排序

为什么快，为什么慢

新到的键只要不破坏当前输出顺序，就能留在同一条段里。初始段更长，之后归并轮数可能减少，但生成有序段本身还没有完成全局排序。

适合场景：适用于减少外部归并前的初始段数量；生成阶段的缓冲块 I/O 为 $\Theta(n/B)$ 。

容易误解：它本身不是完整排序；“平均 2M”描述段长，不是运行时间保证，也不是每段都如此。

依据：[DIST23]

D16 多阶段归并排序

故意让文件中的有序段数量不均等，使某个输入文件耗尽后变成下一阶段的输出文件。

具体怎么做

1. 先生成有序段并明确记录边界；三个文件的练习可在两个文件中放相邻斐波那契数量的段，第三个留空。
2. 每次从两个非空输入文件各取一段，归并到空输出文件。
3. 持续归并，直到一个输入文件的段耗尽；另一文件可能仍有未归并的段。
4. 把刚耗尽的文件改为输出，上一输出改为输入；不能越过已记录的段边界。
5. 直到只剩一个真实段；初始数量不匹配时，可补逻辑上的空虚拟段。

完整小例子：使用 A、B、C 三条带。A 放五条单记录段 [8]、[2]、[6]、[1]、[5]，B 放三条段 [7]、[3]、[4]，C 为空。这里的方括号表示各条独立有序段，不能把同一条带上的内容误当作已经整体有序。

第一阶段逐对合并 A、B 的前三条段，写入 C：[7,8]、[2,3]、[4,6]。A 剩 [1]、[5]，B 变空，段数为 (2, 0, 3)。第二阶段把 A 的两段分别与 C 的前两段合并，写到 B：[1,7,8]、[2,3,5]；C 剩 [4,6]，段数为 (0, 2, 1)。

第三阶段把 B 的第一段 [1,7,8] 与 C 的 [4,6] 合并到 A，得 [1,4,6,7,8]。B 仍有 [2,3,5]，段数为 (1, 1, 0)。第四阶段把 A、B 合并到 C，得 [1,2,3,4,5,6,7,8]。每阶段由刚刚变空的带接任输出，避免普通轮次之间再完整分配一遍。

效率与存储

最好	已有一段则无须归并；读写为 $\Theta(n/B)$
平均或期望	取决于初始段长及多阶段调度
最坏	固定路数与正确调度下，归并 I/O 为 $O((n/B) \log r)$
额外空间	内存中使用块缓冲区；外部工作空间 $O(n)$
稳定性	需要明确追踪等键先后，例如原始位置标签
严格原地	不是原地排序

为什么快，为什么慢

不均匀的初始段数让读空的输入带接任输出，主要省的是带之间的重新分配，而不是普通内存模型中的比较次数。

适合场景：适用于理解历史磁带排序，以及有限输入输出流如何影响归并调度。

容易误解：多阶段指有序段调度，并非新的比较规则；包含多个有序段的文件未必整体有序。

依据：[DIST24] [DIST25]

D17 分布式范围排序／TeraSort

名称／关系：TeraSort；MapReduce 全序范围排序

把采样范围边界、分布式重新分发和局部排序组合起来，生成全局有序的输出分区。

具体怎么做

1. 主任务开始前先采样输入键并把样本排序。
2. 选择 $p-1$ 个有序分界键，为归约器或工作节点定义 p 个互不重叠的键范围。
3. 对每条输入记录计算所属范围，通过 shuffle 把完整记录送到相应节点。
4. 各节点按键排序或归并收到的全部记录；内存不足时溢写磁盘并外部归并。
5. 各节点写出有序分区；按范围顺序读取，并检查分区内部及相邻分区边界是否有序。

完整小例子：节点 A 起初有 [60,10,40]，B 有 [20,70,50]。假设抽样分界规定第一份输出收键 <50 ，第二份收键 ≥ 50 。A 分别发送 [10,40] 与 [60]；B 分别发送 [20] 与 [70,50]。

第一个目的节点收到 [10,40,20]，局部排成 [10,20,40]；第二个收到 [60,70,50]，排成 [50,60,70]。按约定范围顺序读取两份文件，才得到 [10,20,40,50,60,70]。随意颠倒文件读取次序不能保证全局有序，附带信息也必须始终随键一起传输。

效率与存储

最好	范围均衡时最大节点负载最小
平均或期望	依赖采样、网络、磁盘及局部排序，不能给统一实际耗时 Θ 界
最坏	坏样本可能让一个节点收到近 n 条记录，磁盘溢写会增加轮次
额外空间	分布式记录存储、shuffle 缓冲区，以及可能 $O(n)$ 的溢写文件
稳定性	Hadoop TeraSort 的键排序不保证稳定
严格原地	不是原地排序

为什么快，为什么慢

按范围分区把一个大排序变成独立的局部排序；抽样、网络传输以及负载过重的慢节点，都可能决定总体耗时。

适合场景：适用于跨多台机器的数据集；TeraSort 是系统级采样排序应用，不是新的渐近复杂度基本算法。

容易误解：只有各文件局部有序还不够，还必须有序且不重叠地划分范围；网络重新分发也有成本。

依据：[DIST26]

D13 带采样的直方图排序

名称／关系：HSS；带采样的直方图排序

先用全局名次统计反复改进候选分界键，再移动整个分布式数据集。

具体怎么做

1. 先对各处理器本地的键排序或建立索引，目标全局名次设在 n/p 、 $2n/p$ 等附近。
2. 从仍未确定的名次区间中采样候选键，并把候选键发给所有处理器。
3. 各处理器统计小于候选键的本地键数，再求和获得全局名次，形成直方图。
4. 接受落入允许名次窗口的候选键；缩小其余区间，继续采样和统计。
5. 分界键合格后才重新分配完整记录，再对收到的分区排序或归并，完成全局排序。

完整小例子：两个工作节点持有 $A=[8,1,6,3]$ 与 $B=[7,2,5,4]$ ，希望分成各四条记录的两个全局范围。假设抽样先提出上边界 6，统计不大于 6 的数量，A 有三条、B 有三条，总共六条，说明低值范围会超载。

一次可能的修正是试边界 4：A 提供 1、3，B 提供 2、4，总数恰为四条，分界均衡。把 [1,3]、[2,4] 发往低值节点，把 [8,6]、[7,5] 发往高值节点。各自排好后得到 [1,2,3,4] 与 [5,6,7,8]，按范围顺序连接即为答案。

直方图提供的是候选边界的名次信息，并没有直接排好桶内数据。这里的精确小例子只说明“提议—统计—修正”的反馈过程；大规模 HSS 使用抽样名次区间和分布式通信，不会逐个穷试所有可能键。

效率与存储

最好	一轮即成功，另加本地排序与数据交换
平均或期望	依赖 p 、均衡容差 ϵ 、样本量及 BSP 通信成本
最坏	无单一仅含 n 的界，不能省略细化与通信项
额外空间	本地记录加样本、直方图和交换缓冲区
稳定性	若要求稳定输出，须明确用原始位置打破等键关系
严格原地	不保证普通单数组意义上的原地排序

为什么快，为什么慢

名次反馈能在昂贵的数据重分发之前修正抽样边界；额外统计与通信是否值得，取决于它避免了多严重的负载失衡。

适合场景：适用于分区质量和数据移动开销占主导的大型分布式数据；这是独立的分布式直方图分支。

容易误解：直方图只是数量摘要，并非有序数据；不要把 HSS 与计数排序或线性数值桶排序混为一谈。

依据：[DIST19] [DIST20]

D25 漏斗排序 惰性缓冲分支

名称／关系：惰性漏斗排序；缓存无关的漏斗排序家族

阅读层级：高级构造或实现家族；具体展开范围见下文。

通过不同大小缓冲区组成的递归归并层次，在不知道缓存参数时减少缓存传输。

具体怎么做

1. 输入较大时，分成约 $n^{1/3}$ 个连续块，每块约 $n^{2/3}$ 项，再递归排序各块。
2. 把 k 个有序流接到平衡二叉归并树的叶子；基本操作是普通双路归并。
3. 递归设置缓冲区大小：在树高约一半处切分，边界边分配约 $k^{3/2}$ 个槽，再在上、下子树内重复。
4. 每个递归定义的子树及缓冲区连续存储，并保存流位置及耗尽标记。
5. 填充节点输出时比较两输入前端并输出较小者；输入缓冲区空了，就递归填充相应子节点。
6. 持续填充根节点直到全部输出；耗尽流不再提供记录，小规模基本情况用普通排序。

完整小例子：先用四个无序小块 [7,1]、[8,2]、[6,3]、[5,4] 观察惰性合并。基础排序后得到 [1,7]、[2,8]、[3,6]、[4,5]；前两段接到一个下层合并器，后两段接到另一个。

仅为演示，令两个中间缓冲区各放两条记录。第一个下层合并器填出 [1,2]，第二个填出 [3,4]。顶层先输出 1、2；第一个缓冲区空了，才向下请求补充，得到 [7,8]。另一个缓冲区仍有 [3,4]，所以接着输出 3、4。

第二个缓冲区再补入 [5,6]，顶层继续输出 5、6、7、8。下层不是随意运行，而是上层缺数据时才补充。完整惰性漏斗使用递归选定、大小不同的缓冲；这里的两格缓冲只解释补充机制，不作为缓存最优大小定理的演示。

效率与存储

最好	CPU 工作量上界 $O(n \log n)$ ，不假设已有序捷径
平均或期望	比较工作量 $O(n \log n)$ ；缓存传输另算
最坏	CPU 工作 $O(n \log n)$ ；理想缓存传输上界为 $O(1 + (n/B) \times [1 + \text{以 } M/B \text{ 为底的 } \log(n/B)])$ ，对数若为负则按 0 计；要求 $M=\Omega(B^2)$ 且 $M/B>1$ 。
额外空间	包含输出与归并缓冲区的总辅助空间 $O(n)$
稳定性	基本排序稳定、等键归并先取左侧时可以稳定
严格原地	不是原地排序

为什么快，为什么慢

分层缓冲把邻近合并工作集中起来，值得做更大传输时再推进。精心选择的递归大小能同时照顾多层缓存，而不必事先写死每层容量。

适合场景：适用于缓存无关排序与存储层次理论；M 是缓存容量，B 是块大小，I/O 公式中的对数至少取 1。

容易误解：随意换成堆或统一小缓冲区不能保留该定理；高缓存假设不可忽略。

依据：[DIST39] [DIST40]

H 并行执行与排序网络

并行总工作量与并行深度必须分别比较。AKS 给出理论上的对数深度档；常见网络往往为对数平方深度；网络或只能交换邻居的规则又增加移动约束。Merge Path 与 IPS4o 是处理器和内存上的算法，并非固定网络，要另写机器条件。

编号只标识条目，不表示名次。

D26 AKS 排序网络 高级理论构造

名称／关系：Ajtai-Komlós-Szemerédi 排序网络

阅读层级：高级构造或实现家族；具体展开范围见下文。

这是一种高级、与输入数据无关的网络，通过基于扩展图的近似对半分离器和精密安排的纠错阶段，实现对数级并行深度。

具体怎么做

- 1. 先理解基本操作：比较交换把两条记录中的较小者送到指定的一条线上，较大者送到另一条线上。
- 2. ϵ 分离器使用合适的有界度二部扩展图：这是一种稀疏但跨组连接性很强的图。图的边定义比较对，再安排成各对互不重叠的并行层。
- 3. 分离器把较小、较大的名次近似分开。对于不超过输入规模一半的任意 k ，最大的 k 项中留在左侧的至多为 ϵk 项；最小项留在右侧的限制对称。
- 4. AKS 系列构造在多个层次安排分离器及更强的分隔组件，反复修正放错组的元素；仅对近似分成的两半递归，并不足以证明正确。
- 5. 完整论文构造会确定图、常数、组容量及每一层比较。执行完整调度后才保证精确有序；本概述未给出这些复杂的构造细节。

小例子：仅演示基本操作：一个比较器把 $[9,2]$ 变为 $[2,9]$ 。近似分离器仍可能留下错位项，它本身不是完整排序网络。

效率与存储

最好	深度 $O(\log n)$ ，比较器工作量／网络规模 $O(n \log n)$ ；网络固定，不提前退出
平均或期望	任何输入分布下，深度 $O(\log n)$ ，比较器数量 $O(n \log n)$
最坏	每层至多 $n/2$ 个互不重叠的常数成本比较器并行执行时，深度 $O(\log n)$ ；总比较数 $O(n \log n)$
额外空间	保留 n 个活动记录值；显式保存的比较器调度表有 $O(n \log n)$ 项。电路规模不等于普通辅助数组空间。
稳定性	不天然稳定；可把原输入位置作为第二关键字

严格原地	寄存器执行可覆盖原来的 n 个值，但布线／调度构造另需存储，不能声称严格 $O(1)$ 辅助空间
------	--

为什么快，为什么慢

构造依靠大量可同时执行的稀疏比较交换及反复纠错，在理论上达到最优深度阶；隐藏常数与网络生成却很复杂，不能当作实用小网络看待。

适合场景：用于证明排序网络深度与规模的渐近最优性，以及构造理论并行算法。

容易误解：巨大隐藏常数与复杂的扩展图／分离器构造，使经典 AKS 不适合作为普通实际排序的推荐方案；对数深度不是对数串行时间。

依据：[DIST_AKS_01] [DIST_AKS_02] [DIST_AKS_03]

D21 采用 Merge Path 的并行归并排序

名称／关系：并行归并排序中的 Merge Path／共同秩分割

按输出名次切分归并任务，让各线程把不同输入片段归并到互不重叠的输出区间。

具体怎么做

- 1. 先独立排好 p 个连续输入块，再把相邻有序段记为 A 、 B ；把可用线程分配给同时进行的归并对。
 - 2. 对每个归并对，把合并长度 N 分成 q 个近乎等长的输出名次区间， q 为分给该对的线程数。
 - 3. 对每个边界名次 r ，二分查找 $i+j=r$ ，满足 $A[i-1] \leq B[j]$ 且 $B[j-1] < A[i]$ ；越界比较忽略。
 - 4. 每个线程取相邻两边界之间的 A 、 B 片段，普通归并到自己负责的输出区间。
 - 5. 等键时先取 A ；每层完成后同步、交换输入输出缓冲区，并把段长翻倍，直到只剩一段。
- 小例子： $A=[1,4,7]$ 、 $B=[2,3,8]$ ， $r=3$ 的边界为 $(1,2)$ ；两线程分别输出 $[1,2,3]$ 与 $[4,7,8]$ 。

效率与存储

最好	单次 N 项归并分配 q 个线程时，时间 $O(N/q + \log N)$ ；完整排序上界见后项
平均或期望	上述共享内存调度中，完整排序的并行时间上界为 $O((n/p)\log n + \log n \log p)$
最坏	时间 $O((n/p)\log n + \log n \log p)$ ；工作量 $O(n \log n + p \log n \log p)$ ， $2 \leq p \leq n$
额外空间	$O(n)$ 输出缓冲区加分区边界
稳定性	局部排序稳定、等键始终先取左段时稳定
严格原地	该缓冲方法不是原地排序

为什么快，为什么慢

共同名次把输出分成互不冲突的小块，同时保留同一个稳定归并顺序。加速来自同时做这些小块，搜索、同步和带宽成本仍不能省略。

适合场景：适用于并行归并及多核归并排序的后期，此时只并行执行不同归并任务已不足以利用所有线程。

容易误解：把两输入各自等长切块会出错；Merge Path 是归并组件，单次归并界不是整个排序界。

依据：[DIST32] [DIST33]

D18 双调排序网络

名称／关系：双调归并排序；双调排序网络

先构造先升后降的双调序列，再反复比较相隔半段的元素，把它变成有序序列。

具体怎么做

- 1. 先使用 2 的幂个记录；其他长度可补带标记的正无穷哨兵，以便最终升序排列。
- 2. 递归把前半段排成升序、后半段排成降序，两者连接即为双调序列。
- 3. 对长度 2h 的升序双调归并，同时比较 i 与 i+h (i<h) ，较小值留在前半段。
- 4. 分别在两半内部重复同样的升序归并；长度为一时不做操作。
- 5. 每个递归排序均用此归并收尾，最终删去补入的哨兵。

小例子：[3,1,4,2] → [1,3 | 4,2] → 跨半段比较得 [1,2 | 4,3] → [1,2,3,4]。

效率与存储

最好	工作量 $\Theta(n \log^2 n)$ ，比较器深度 $\Theta(\log^2 n)$
平均或期望	固定比较器调度：工作量 $\Theta(n \log^2 n)$ ，深度 $\Theta(\log^2 n)$
最坏	工作量 $\Theta(n \log^2 n)$ ；足够比较器并行执行时深度 $\Theta(\log^2 n)$
额外空间	每条线额外 $O(1)$ ；迭代软件可用 $O(1)$ 辅助字
稳定性	不稳定，除非添加原始位置作为第二关键字
严格原地	数组模拟可原地执行，不计补位空间

为什么快，为什么慢

同一层中的比较对互不冲突，因此可以并行执行；但固定网络在有序输入上也照常比较，平方对数深度不代表总工作量也那么少。

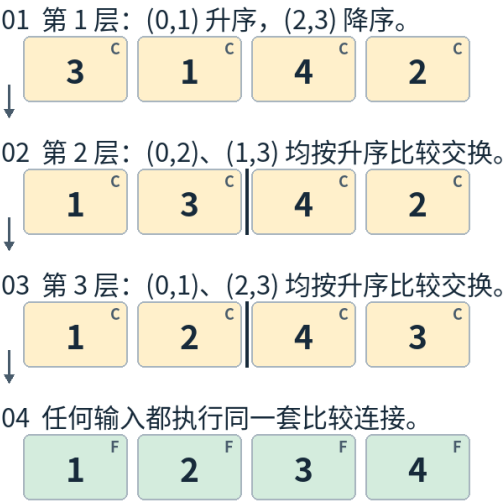
适合场景：适用于 GPU 小块、SIMD、小型硬件电路及比较计划与数据无关的场景。

容易误解：平方对数深度是硬件足够时的并行时间，不是串行时间；归并的输入必须已具双调结构。

依据：[DIST27] [DIST28]

过程图 D18

双调网络：四个键的三层固定调度



C 正在比较 M 正在移动 S 区域内有序 F 最终归位

D18 双调网络：四个键的三层固定调度。与本条目的示例对应；标记含义见“基础”。

D19 Batcher 奇偶归并排序

名称／关系：Batcher 奇偶归并网络

递归归并两半中的偶数位置与奇数位置，再用固定的相邻比较层收尾。

具体怎么做

- 1. 使用 2 的幂长度和从零开始的下标，先递归把两半都排成升序。
- 2. 归并时分别取两半的偶数下标元素，再递归归并这两个子序列。
- 3. 另行归并两半的奇数下标子序列；最小情形是比较两个单元素。
- 4. 把偶数归并结果和奇数归并结果交错放置，先放偶数结果。
- 5. 比较交换 (1,2)、(3,4) 等相邻位置，首尾不动；每一排序层均使用这种归并。

完整小例子：从 [4,1,3,2] 开始，先递归排好两对，得到 [1,4 | 2,3]。合并时，先把两段的偶数下标子序列 [1] 与 [2] 合成 [1,2]，再把奇数下标子序列 [4] 与 [3] 合成 [3,4]。

把两个结果交错放回偶数线与奇数线，得到 [1,3,2,4]；最后比较交换下标 1、2，成为 [1,2,3,4]。这些连接是预先固定的。它与奇偶换位排序不同：更大网络靠递归合并奇偶子序列来构造，而不是只反复做邻居交换。

效率与存储

最好	比较器工作量 $\Theta(n \log^2 n)$ ，深度 $\Theta(\log^2 n)$
平均或期望	固定比较器调度：工作量 $\Theta(n \log^2 n)$ ，深度 $\Theta(\log^2 n)$
最坏	工作量 $\Theta(n \log^2 n)$ ；足够比较器并行执行时深度 $\Theta(\log^2 n)$
额外空间	网络保留 n 个值；递归软件栈 $O(\log n)$
稳定性	未添加原始位置第二关键字时不稳定
严格原地	使用步长下标可原地执行；教学中的子序列若复制则另需空间

为什么快，为什么慢

同一层中的比较对互不冲突，因此可以并行执行；但固定网络在有序输入上也照常比较，平方对数深度不代表总工作量也那么少。

适合场景：适用于硬件排序，以及在读入数值前就确定比较模式的规则并行内核。

容易误解：奇偶归并不是奇偶换位排序；输入的两半必须已经有序。

依据：[DIST29] [DIST30]

D22 列排序

交替执行列排序、特定重排和半列移位，最终得到按列读取的全局有序结果。

具体怎么做

1. 按列优先把 $n=rs$ 个记录放进 r 行 s 列；经典版本要求 r 为偶数、 s 整除 r ，且 $r \geq 2s^2$ 。
2. 把每列从上到下排序。
3. 按列读取矩阵，再按行写回同样形状，然后再次排序所有列。
4. 精确撤销这次重排，再第三次从上到下排序所有列。
5. 在 $r \times (s+1)$ 矩阵中，把列优先序列右移 $r/2$ 位：前补 $r/2$ 个负无穷、后补 $r/2$ 个正无穷哨兵，再排序各列。
6. 撤销半列移位，去掉哨兵，最后从左到右逐列由上到下读取。

完整小例子：取 $r=8$ 、 $s=2$ ，满足本版本的限制。每个列表表示一列从上到下的值。输入两列为 [7,11,15,10,4,8,1,2] 与 [12,9,3,16,6,5,13,14]。第一次列内排序后，为 [1,2,4,7,8,10,11,15] 与 [3,5,6,9,12,13,14,16]。

重排后两列为 [1,4,8,11,3,6,12,14] 与 [2,7,10,15,5,9,13,16]。分别排好，得到 [1,3,4,6,8,11,12,14] 与 [2,5,7,9,10,13,15,16]。撤销重排，成为 [1,2,3,5,4,7,6,9] 与 [8,10,11,13,12,15,14,16]。

第三次列内排序后为 [1,2,3,4,5,6,7,9] 与 [8,10,11,12,13,14,15,16]。跨列边界仍有 9 在 8 前面的逆序。把列优先序列右移四个位置，前加四个 $-\infty$ ，后加四个 $+\infty$ 。移位后的中间一列正好是 [5,6,7,9,8,10,11,12]。

把该列排成 [5,6,7,8,9,10,11,12]，其余移位列也分别排序；最后撤销移位并移除哨兵，真实数据便是顺序的 1 到 16。半列移位的作用，是把原本跨边界的错误放进同一列中处理。

效率与存储

最好	各列采用固定调度归并排序时，工作量 $\Theta(n \log r)$
平均或期望	采用所述列排序时工作量 $\Theta(n \log r)$ ；无须随机输入假设
最坏	工作量 $\Theta(n \log r)$ ；按列并行时间与重排通信依赖机器模型
额外空间	直接使用重排缓冲区和列排序时 $O(n)$
稳定性	不天然稳定，重排可能改变等键记录顺序
严格原地	直接缓冲版本不是原地排序

为什么快，为什么慢

固定次数的列内排序与重排代替了任意全局比较。每列约花 $r \log r$ ，重排时的通信成本也可能盖过表面上很简单的并行结构。

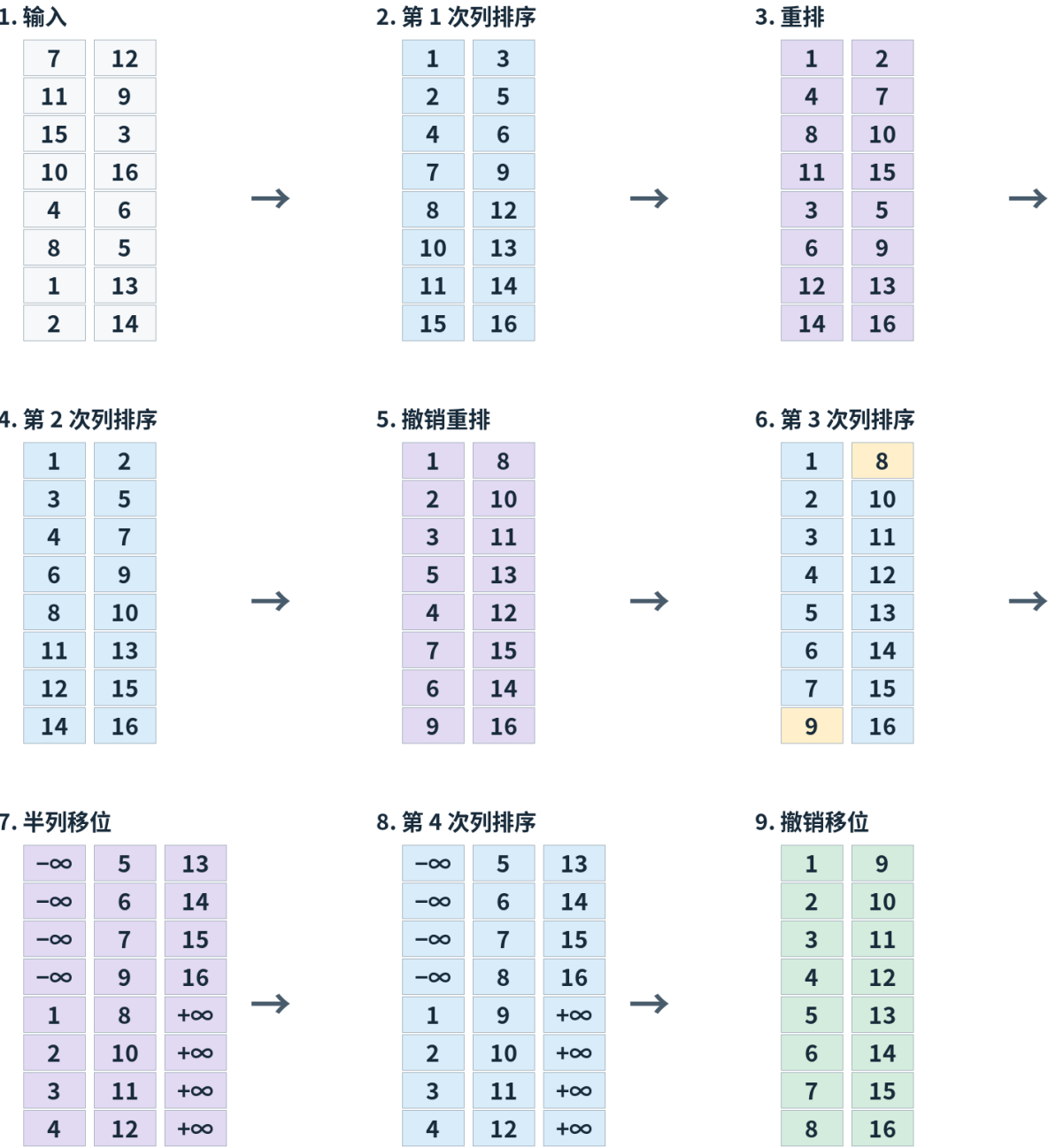
适合场景：适用于学习结构化数据移动，以及主要阶段数较少的并行和外存排序。

容易误解：矩阵尺寸限制是正确性的组成部分；随意交替行列排序并不等于 Columnsort。

依据：[DIST34] [DIST35]

过程图 D22

Columnsort：逐步核对每次重排与列排序



每列从上往下读，再移到右列；金色标出 9、8 的跨列逆序。

D22 Columnsort 完整过程。与本条目的示例对应；标记含义见“基础”。

D24 剪切排序

在正方形处理器网络上，交替进行蛇形方向行排序和向下列排序。

具体怎么做

1. 把 $n=m^2$ 条记录放入 $m \times m$ 网格，每个节点一条，并规定蛇形读取顺序。

- 2. 第 0 行从左到右排序，第 1 行从右到左排序，此后交替方向。
 - 3. 所有列同时从上到下排序。
 - 4. 重复“行排序、列排序”共 $\text{ceil}(\log_2 m)$ 轮，最后再做一次蛇形方向行排序。
 - 5. 沿蛇形读取：偶数行从左到右，奇数行从右到左；每次行列排序可用 m 阶段的奇偶换位排序。
- 小例子：两行 [4,1]、[3,2] → 行排序 [1,4]、[3,2] → 列排序 [1,2]、[3,4] → 最终 [1,2]、[4,3]，蛇形读取为 1,2,3,4。

效率与存储

最好	固定网格调度：深度 $\Theta(\sqrt{n} \log n)$
平均或期望	深度 $\Theta(\sqrt{n} \log n)$ ；总比较交换工作量 $\Theta(n^{3/2} \log n)$
最坏	在所述 n 节点网格上，深度 $\Theta(\sqrt{n} \log n)$ ，工作量 $\Theta(n^{3/2} \log n)$
额外空间	每节点除记录外需 $O(1)$ 局部空间
稳定性	不天然稳定
严格原地	网格上原地进行；转换蛇形输出顺序可能还需一趟

为什么快，为什么慢

交替整理蛇形行与纵列，会逐渐缩小可能混乱的带状区域；记录移动受到网格限制，因此成本分析必须计入这种通信结构。

适合场景：适合讲解二维通信约束下的排序，以及存储布局顺序与数值顺序的区别。

容易误解：最终网格按蛇形有序，并非普通行优先有序；一次行或列阶段本身就是完整的局部排序。

依据：[DIST38] [DIST41]

D20 奇偶换位排序

名称／关系：砖排序；奇偶排序；不是 Batchers 奇偶归并

交替对互不重叠的相邻对比较交换，使元素经过足够轮次到达最终位置。

具体怎么做

- 1. 位置从零编号，先采用固定执行 n 个阶段的版本。
 - 2. 第 0 阶段比较 (0,1)、(2,3) 等，仅在左键严格大于右键时交换。
 - 3. 第 1 阶段比较 (1,2)、(3,4) 等，规则相同。
 - 4. 交替执行两种模式；同阶段中的位置互不重叠，因此可以同时比较。
 - 5. n 个阶段后数组有序；自适应版本可在完整偶、奇两阶段都无交换时提前停止。
- 小例子：[3,2,1] → 偶阶段 [2,3,1] → 奇阶段 [2,1,3] → 偶阶段 [1,2,3]。

效率与存储

最好	固定网络：工作量 $\Theta(n^2)$ ，深度 $\Theta(n)$
平均或期望	固定网络：工作量 $\Theta(n^2)$ ，深度 $\Theta(n)$
最坏	工作量 $\Theta(n^2)$ ； $\Theta(n)$ 个线程时深度 $\Theta(n)$
额外空间	串行辅助空间 $O(1)$ ，每线程局部状态 $O(1)$
稳定性	不交换相邻相等键时稳定
严格原地	是

为什么快，为什么慢

只有相邻位置交换信息，站在错误一端的记录要走过许多阶段；虽然多对能同时工作，总比较交换量仍是平方级。

适合场景：适合讲解同步，以及一维处理器链上的最近邻排序。

容易误解：单个阶段无交换不代表有序；固定网络与自适应版本的最好情况不能混用。

依据：[DIST31]

D31 IPS4o：原地并行超标量抽样排序

名称／关系：In-place Parallel Super Scalar Samplesort；IPS4o

阅读层级：高级构造或实现家族；具体展开范围见下文。

先按抽样得到的分界值分类，把同一范围的记录收集成整块，再将整块送往所属区域。这样能让判断和搬动更规律，减少处理器反复猜错分支的代价。

具体做法

1. 从较大的样本中选出有序分界值。每个工作线程扫描自己的条带，把记录放进对应桶的小缓冲区；缓冲区满后，写回条带中已经读完的空间。
2. 汇总各桶数量，算出整个数组里的精确桶边界。借助交换缓冲区，把完整数据块搬到正确区域；被挤出的块也要继续保存并送回其目的地。
3. 处理桶边界处不足一块的数据和缓冲区剩余记录，确保每条真实记录都回到输入数组。与分界值相等的键必须一致处理，采用等值桶时也要单独安排。
4. 递归排各个非平凡桶，将独立任务分配给工作线程，小桶用局部基础排序。并行实现还必须协调数据块的读写归属，防止一个线程覆盖另一个仍在读取的数据。

跟着做：用单线程、小块大小 2 做教学演示，对 [7,1,6,2,8,3,9,4] 以 5 分界。小值缓冲区依次收集 [1, 2]、[3,4]，大值缓冲区收集 [7,6]、[8,9]。每个完整块内的记录都属于同一侧。

跟着做：假设写回块的次序为 [7,6|1,2|8,9|3,4]。计数表明前四个位置应属于小值桶，于是交换第 0 块与第 3 块，得到 [3,4|1,2|8,9|7,6]。块已分区正确，再分别排好两侧，得到 [1,2,3,4|6,7,8,9]。真实输入常有不足整块的边角，必须执行清理步骤，不能只考虑例子中恰好整齐的情况。

效率与存储

总工作量	论文抽样条件下，以高概率为 $O(n \log n)$
并行耗时	取决于线程数、负载均衡、内存带宽与同步成本
额外空间	每线程桶／块缓冲、计数及任务或递归状态，不能笼统写 $O(1)$
稳定性	通常不稳定

为什么快，为什么慢

桶内成块让内存传输更规律，减少难预测的分支。加速仍依赖任务均衡和硬件资源，单数比较次数会漏掉相当一部分工程收益。

适合：多核机器上的大数组排序。此时减少分支预测失败和杂乱的内存访问，可能与减少比较次数同样重要。

注意：它是一套工程化方法，不是脱离条件的效率冠军。名称里的“原地”不表示所有缓冲都消失了，也不能把并行时间当成总工作量。

来源：[N11] [N12]

I 受限移动与物理模型

一次允许的操作可能是翻转整个前缀、让许多珠子下落，或者移动实体细棒。把这些动作都当成一次普通数组比较，会得到误导性的速度结论。每个条目分别说明它使用的模拟或物理模型。

编号只标识条目，不表示名次。

X10 煎饼排序

像翻转一摞煎饼上部一样，通过前缀反转把剩余最大值送到末尾。

具体怎么做

- 1. 令 m 表示尚未固定的前缀长度，初始为 n 。
- 2. 扫描位置 0 到 $m-1$ ，找到最大元素所在位置 k 。
- 3. 若 $k=m-1$ ，它已经就位；否则反转 0 到 k ，使最大值来到最前。
- 4. 再反转 0 到 $m-1$ ，把最大值送到当前前缀末尾；已就位时跳过此步。
- 5. m 减一，重复到 $m \leq 1$ ；每次反转都从前缀两端向内交换。

小例子：[3,1,2]：翻前 3 个得到 [2,1,3]，再翻前 2 个得到 [1,2,3]；只能翻前缀，不能任意翻中段。

效率与存储

最好	每轮完整扫描找最大值时 $\Theta(n^2)$
平均或期望	$\Theta(n^2)$
最坏	元素操作 $\Theta(n^2)$ ；前缀翻转次数 $O(n)$
额外空间	迭代反转时 $O(1)$
稳定性	不稳定
严格原地	是

为什么快，为什么慢

每个最大值只需少数几次翻转就能归位，但寻找它和反转整个前缀都可能访问许多记录，所以线性翻转次数仍可对应平方级数组工作。

适合场景：适理解操作受限的排序，并区分“翻转次数”和实际搬运元素的成本。

容易误解：只有抽象煎饼模型才把一次翻转算作常数成本；反转 m 个数组元素需 $\Theta(m)$ ，且贪心法未必翻转最少。

依据：[X-PANCAKE]

过程图 X10

煎饼排序：只允许翻转前缀



C 正在比较 M 正在移动 S 区域内有序 F 最终归位

X10 煎饼排序：只允许翻转前缀。与本条目的示例对应；标记含义见“基础”。

X11 焦面煎饼排序

名称／关系：焦面煎饼排序；带方向的前缀翻转

既排大小也调朝向：每次翻转前缀时，其中所有煎饼的正反面都会倒转。

具体怎么做

- 1. 用不同绝对值 1...n 表示大小；正号表示最终要求的朝向。
 - 2. 对长度为 m 的未处理前缀，找到绝对值 m；若末尾已是 +m，直接缩短前缀。
 - 3. 否则翻转到该元素为止的前缀，同时逆序并改变所有符号，让它来到最前。
 - 4. 若最前是 +m，就翻转这个单元素前缀，使其变成 -m。
 - 5. 翻转全部 m 个元素，-m 就会以 +m 的正确朝向落在末尾。
 - 6. m 减一后重复；最后一个元素若为负，就单独翻正。
- 小例子：[-2,+1] 翻前 2 个得到 [-1,+2]，再翻前 1 个得到 [+1,+2]；位置与符号都会改变。

效率与存储

最好	上述每轮完整扫描版本为 $\Theta(n^2)$
平均或期望	$\Theta(n^2)$
最坏	数组操作 $\Theta(n^2)$ ；这种简单策略至多 $3n$ 次翻转
额外空间	$O(1)$
稳定性	规定大小唯一；一般化后不保证稳定

严格原地	是
------	---

为什么快，为什么慢

每个最大值只需少数几次翻转就能归位，但寻找它和反转整个前缀都可能访问许多记录，所以线性翻转次数仍可对应平方级数组工作。

适合场景：适合学习带符号置换，以及同时受位置和朝向约束的重排问题。

容易误解：反转时不能忘记变号；这里的次数上界仅属于这一简单构造，并非最优翻转次数。

依据：[X-BURNT]

X12 珠排序／重力排序

名称／关系：重力排序；珠排序

把非负整数 x 表示为一行 x 颗珠子，再让每列珠子向下落到底。

具体怎么做

- 1. 找到最大值 M ；本版本不接受负数或非整数。
- 2. 建立 n 行 M 列的布尔网格；值为 x 的行把前 x 格置为有珠。
- 3. 逐列统计共有多少颗珠子。
- 4. 清空该列，再从底部填入同样数量的珠子，不必逐颗反复模拟下落。
- 5. 从上到下统计每行珠数，所得数字就是升序结果。

小例子：[3,1,2] 表示为 111／100／110；各列落到底后为 100／110／111，对应 [1,2,3]。

效率与存储

最好	显式网格模拟为 $\Theta(n(M+1))$
平均或期望	$\Theta(n(M+1))$ ，取决于数值大小 M
最坏	$\Theta(n(M+1))$ ，不能只用 n 衡量
额外空间	网格 $\Theta(nM)$ 格，另需 $O(n)$ 输出
稳定性	不是记录级稳定排序：珠子表示会丢失个体身份
严格原地	否

为什么快，为什么慢

数值 x 被展开为 x 颗珠子。表面上的简单隐藏了这种一元表示的大小，处理宽至最大值的网格，可能远比处理二进制整数昂贵。

适合场景：展示一元编码和非常规计算；适合用小非负整数做网格演示。

容易误解：重力“瞬间完成”或专用硬件线性时间的说法，忽略了软件需要承担的成本；M 可能远大于其二进制位数。

依据：[X-BEAD]

X13 意大利面排序

把数字转化为面条长度，再反复取出最高的一根，是一个物理计算思想实验。

具体怎么做

1. 选择正数，并确定一个能让长度差异清楚可辨的比例。
2. 为每个值准备一根直杆，标记身份，并保留长度与数值的对应关系。
3. 让整束直杆立在同一平面上，底端对齐。
4. 从上方降下平板，先碰到的就是最长杆；取出它，把对应值放进输出的最后一个空位。
5. 重复接触和取出操作，直到所有位置填满；相同长度的处理次序必须另行规定。

小例子：[2,5,3] 对应的杆按 5、3、2 取出；从右向左填入输出，得到 [2,3,5]。

效率与存储

最好	仅在接触、取出均为常数成本的理想模型下 $\Theta(n)$
平均或期望	现实中没有通用的平均时间界
最坏	理想模型 $\Theta(n)$ ；物理精度及准备成本另计
额外空间	n 根物理杆，另有 $\Theta(n)$ 输出表示
稳定性	等长杆不保证稳定顺序
严格原地	不是 RAM 模型的原地算法

为什么快，为什么慢

物理摆放允许一次动作影响很多根杆，但测量、制作和读取都是真实工作。普通程序模拟时，不能把物理选出最长杆当作免费基本操作。

适合场景：适合比较不同计算模型，识别隐藏的并行能力、精度要求与编码成本。

容易误解：平板实际上在并行寻找最大值；这并没有突破普通顺序计算机的比较排序下界。

依据：[X-SPAGHETTI]

J 刻意低效但可定义的过程

确定性的完整过程按增长趋势从 Stooge、Slowsort 走向排列穷举。随机反复尝试另成一组：期望成本并不能提供任何有限的最坏截止时间。这些方法用于理解怎样才算取得进展，不是实用推荐。

编号只标识条目，不表示名次。

X17 Stooge 排序 三段递归排序

对重叠的三分之二区间递归三次：前段、后段、再前段。

具体怎么做

- 1. 对于下标 l 到 r 的区间，少于两个元素时直接返回。
- 2. 比较首尾元素；如果首项更大，就交换首尾。
- 3. 若只有两个元素，此时已排好；否则令 $t = \text{floor}(\text{区间长度}/3)$ 。
- 4. 递归排序 l 到 $r-t$ ，也就是前面约三分之二。
- 5. 递归排序 $l+t$ 到 r，也就是后面约三分之二。
- 6. 再次递归排序 l 到 $r-t$ ，修复重叠部分在前一次操作中造成的失序。

小例子：[3,1,2] 先交换首尾得 [2,1,3]；排前两个得 [1,2,3]，再排后两个及前两个，结果不变。

效率与存储

最好	$\Theta(n^{\log_3 2.7095})$ ，指数约为 2.7095
平均或期望	$\Theta(n^{\log_3 2.7095})$ ；递归规模不随已有顺序改变
最坏	$\Theta(n^{\log_3 2.7095})$
额外空间	递归栈 $O(\log n)$
稳定性	不稳定
严格原地	覆盖原数组，但不满足严格 $O(1)$ 额外空间

为什么快，为什么慢

同样规模的三分之二区间被重叠地递归处理三次。每次局部动作看似很少，整体递推仍会比平方级增长得更快。

适合场景：适合练习递推式：三个约 $2n/3$ 大小的子问题，比两个一半大小的子问题昂贵得多。

容易误解：第三次递归不能省略；已有序时即使不交换，仍要执行全部递归，最好时间并非线性。

依据：[X-STOOGES]

X18 慢速排序

为了找到两段的最大值，先把两段都排好；只固定一个最大值后，又把几乎全部元素重排一次。

具体怎么做

- 1. 对区间 $l \dots r$ ，若 $l \geq r$ 就返回；否则取中点 $m = \text{floor}((l+r)/2)$ 。
- 2. 递归慢速排序 $l \dots m$ ，其最大值会来到 m 。
- 3. 递归慢速排序 $m+1 \dots r$ ，其最大值会来到 r 。
- 4. 比较 $A[m]$ 、 $A[r]$ ，按需交换，使整个区间的最大值位于 r 。
- 5. 固定位置 r ，再递归慢速排序 $l \dots r-1$ ，即使其中大部分刚处理过。

小例子：[3,1,2] 先排前半段得 [1,3,2]，比较 3 与 2 并交换得 [1,2,3]，然后还要再排一次前两个。

效率与存储

最好	$n^{(\Theta(\log n))}$ ，超多项式时间
平均或期望	$n^{(\Theta(\log n))}$ ；递归调用结构相同
最坏	对该固定递推式为 $n^{(\Theta(\log n))}$
额外空间	最大递归深度 $O(n)$
稳定性	不稳定
严格原地	覆盖原数组，但 $O(n)$ 栈使其不属于严格原地排序

为什么快，为什么慢

排完两半后，只把一个最大值归位，又把缩短后的前缀从头排一遍。已完成工作被如此少地利用，导致准多项式级增长。

适合场景：虽然是戏仿作品，却是能终止并正确排序的算法；可说明重复工作如何拖垮分治策略。

容易误解：递推式是 $T(n) = T(\text{floor}(n/2)) + T(\text{ceil}(n/2)) + T(n-1) + \Theta(1)$ ，不是归并排序的递推式；已有序输入也走同样的调用。

依据：[X-SLOW]

X22 排列穷举排序

名称／关系：排列穷举排序；不是随机洗牌

不重复地枚举全部候选排列，直到找到一个有序排列。

具体怎么做

- 1. 把每个输入位置视为不同标签，即使键相同；先检查当前顺序。

- 2. 用深度优先交换生成排列：填位置 i 时，依次尝试每个 $j \geq i$ 的元素。
 - 3. 交换 i, j ，递归处理 $i+1$ ；若未找到答案，回退交换，再试下一个候选。
 - 4. 每生成一个完整候选，检查全部 $n-1$ 对相邻元素，判断是否有序。
 - 5. 一旦发现有序候选就停止并保留该排列，不能再撤销成功路径上的交换。
- 小例子：[2,1,3] 的一种枚举顺序是 [2,1,3]、[2,3,1]、[1,2,3]，到第三个候选时结束。

效率与存储

最好	第一个候选已排好时 $\Theta(n)$
平均或期望	全扫描、不同键且初始排列均匀随机时 $\Theta(n \cdot n!)$
最坏	本全检查枚举版本为 $\Theta(n \cdot n!)$
额外空间	递归生成器栈 $O(n)$ ，无须保存全部排列
稳定性	未明确要求原位置次序时不稳定
严格原地	在原数组交换，但需要 $O(n)$ 栈

为什么快，为什么慢

枚举能避免重复访问同一个带身份的排列，但仍有 $n!$ 种排列；若每个候选都完整检查顺序，还要再乘一个 n 。

适合场景：适合学习有限穷举；与随机 Bogo 不同，正确枚举保证在有限候选内终止。

容易误解：若先保存全部排列，会消耗 $\Theta(n \cdot n!)$ 空间；是否提前停止检查会影响时间界，本条明确按完整扫描计费。

依据：[X-PERMUTATION] [X-PERMUTATION-CODE]

X19 Bogo 排序／猴子排序

名称／关系：猴子排序；随机排列排序；Bogo 排序

先检查是否有序；若无序，就把全部元素等概率洗牌，再重新检查。

具体怎么做

- 1. 扫描相邻元素；若每个左键都不大于右键，就返回结果。
- 2. 否则执行原地 Fisher-Yates 洗牌： i 从 $n-1$ 递减到 1，每次等概率选取 $0 \dots i$ 中的 j 。
- 3. 交换位置 i, j ；每次使用新的独立随机选择，也允许 $i=j$ 。
- 4. 完整洗牌结束后重新检查，直到某次检查成功。
- 5. 实际演示应设置尝试上限，达到上限就报告未完成；超时不能被当作排序成功。

小例子：[2,1,3] 可能洗成 [3,2,1]，再洗成 [1,2,3]；这只是一次幸运轨迹，并非保证次数。

效率与存储

最好	初始已有序时 $\Theta(n)$
平均或期望	不同键、独立均匀且每次成本 $\Theta(n)$ 的洗牌下，期望 $\Theta(n \cdot n!)$
最坏	无有限确定性上界，可能运行任意久
额外空间	迭代原地洗牌时 $O(1)$
稳定性	不稳定
严格原地	是

为什么快，为什么慢

均匀洗牌把先前进展全部忘掉。互异键共有 $n!$ 种排列，只有一种有序，因此期望洗牌次数达到阶乘级，且没有有限的最坏完成期限。

适合场景：适合学习几何分布：不同键的一次均匀洗牌成功概率为 $1/n!$ ，无序起点的期望洗牌次数为 $n!$ 。

容易误解：有限期望不代表有限最坏时间；重复键会改变成功概率，非均匀洗牌也不能套用该计算。

依据：[X-BOGO] [X-RANDOM-SLOW]

X20 Bozo 排序 随机异位交换

名称／关系：Bozo-sort+；本文采用两个不同位置的版本

每次不做完整洗牌，只随机交换两个不同位置，再检查是否有序。

具体怎么做

- 1. 本条采用明确版本：检查全部 $n-1$ 对相邻元素，记录是否存在逆序，不提前结束检查。
- 2. 若没有逆序就返回；否则在全部不同位置组成的无序对中等概率选一对。
- 3. 交换这两个位置的记录，即使交换后看起来更乱也照做。
- 4. 重复完整检查和随机交换，直到有序；演示时应设上限，超过就如实报告未完成。

小例子：[3,1,2] 交换位置 0、1 得 [1,3,2]，再交换 1、2 得 [1,2,3]；别的选择也可能把进展抵消。

效率与存储

最好	初始已有序时 $\Theta(n)$
平均或期望	每次完整检查、初始不同键排列均匀随机时，期望 $\Theta(n \cdot n!)$
最坏	$n \geq 3$ 时随机运行没有有限最坏时间上界

额外空间	$O(1)$
稳定性	不稳定
严格原地	是

为什么快，为什么慢

随机交换可能撤销前一次有用的交换，过程在各种排列间游走，没有持续改善的进度量；本版本每轮还要花 $\Theta(n)$ 检查整个数组。

适合场景：可展示在排列空间中的随机游走；文献对随机初始顺序的异位交换版本给出 $\Theta(n!)$ 的期望交换次数。

容易误解：相邻状态有关联，不能直接照搬 Bogo 的“每次成功概率 $1/n!$ ”推理；提前结束检查也会改变时间分析。

依据：[X-RANDOM-SLOW]

X21 Bogobogo 排序

把“检查有序”本身也写成极慢的递归排序，是对 Bogo 的进一步戏仿。

具体怎么做

- 1. 这里采用 David Morgan-Mar 的复制后比较版本；长度为 0 或 1 时直接返回。
- 2. 检查 A 时先复制为 B，再对 B 的前 $n-1$ 个元素递归执行 Bogobogo。
- 3. 比较 B 的最后一项与已排序前缀的最大值；若最后一项更小，就打乱整个 B，再重排其前缀。
- 4. B 最终有序后，与 A 逐项比较；只有顺序完全一致，才算 A 通过检查。
- 5. 若检查失败，就打乱 A 并重新执行整套递归检查；成功则返回 A。

完整小例子：对 $A=[2,1]$ ，先复制为 B，递归处理 B 的单元元素前缀。这个前缀本来有序，但末尾 1 小于前缀最大值 2，所以要打乱 B 后再做内层检验。假设某次 B 变为 $[1,2]$ ，才通过这一步。

通过检验的 $B=[1,2]$ 仍不等于原始 $A=[2,1]$ 。按原始“复制后比较”定义，这表示外层检验失败，还要打乱 A 本身再试。若之后某次 A 变成 $[1,2]$ ，检验通过的副本就可能与它相同，最终返回 $[1,2]$ 。这些只是可能发生的随机结果，不是保证两轮完成。

效率与存储

最好	本复制版本在已有序输入上的最好时间为 $\Theta(n^2)$
平均或期望	本条不宣称该变体有可靠的简洁紧确期望界
最坏	随机运行时间无界
额外空间	同时存在的副本共 $O(n^2)$ ，递归深度 $O(n)$

稳定性	不稳定
严格原地	否

为什么快，为什么慢

候选检验本身又调用随机排序，之后还可能把整个副本否决。嵌套解释了极端浪费，但不能据此凭空猜一个精确的阶乘公式。

适合场景：适合用极小输入展示验证成本与嵌套随机重试；若终止，确实会给出排序结果。

容易误解：这个名称对应多个不同定义；原网页把夸张的阶乘表达式明确写成猜测，不能把它当作已证明的界。

依据：[X-BOGObOGO]

K 不是真正排序的名称与相关任务

等待、删掉记录、不可靠的计时输出，以及想象中的量子捷径，都不能等同于正确排序。拓扑排序是真实但不同的任务：它遵守依赖关系，而不是数值大小。本组不做快慢排行。

编号只标识条目，不表示名次。

X23 奇迹排序

反复检查一个从未修改的列表，等待它“奇迹般”变得有序；这是等待型笑话。

具体怎么做

- 1. 读取列表，不改动任何记录或位置。
 - 2. 检查相邻元素，判断它是否已经非递减。
 - 3. 若检查成功，原样返回列表。
 - 4. 若检查失败，稍等片刻，再做同样检查；不交换、不洗牌、也不插入。
 - 5. 演示时检查几次就停止，并报告“仍然无序”；这一限制只是展示失败，并未修复算法。
- 小例子：[2,1] 检查失败后仍为 [2,1]，再次失败后也仍为 [2,1]；若外界不改数据，就会一直如此。

效率与存储

最好	不可变输入本就有序时 $\Theta(n)$
平均或期望	若无序不可变输入有正概率，则没有有限期望完成时间
最坏	不可变且无序的输入永不终止
额外空间	重复检查需要 $O(1)$ 额外空间
稳定性	成功时保持原样，但不是通用排序算法
严格原地	根本没有重排，用“原地排序”描述会误导

为什么快，为什么慢

反复检查并不会改变一个保持不变的无序输入。这是根本没有进展，不是一个上界很大但终究有限的慢算法。

适合场景：清楚展示“检查某性质”与“让该性质成立”是两回事。

容易误解：不能编造宇宙射线改变位元的等待时间界；硬件损坏既不是算法步骤，也不保证保留原记录。

依据：[X-MIRACLE]

X24 Stalin “排序” 过滤器

名称／关系：Stalin “排序”；有损的单调筛选

只保留不小于上一个保留值的记录；留下的序列有序，但数据被丢弃了。

具体怎么做

- 1. 输入为空就返回空结果；否则保留第一条记录，并记住它的键。
 - 2. 从左至右，每条后续记录只检查一次。
 - 3. 若其键不小于记住的键，就保留并更新记忆；相同键也保留。
 - 4. 若更小，就丢弃；数组可使用独立读写指针，高效压缩保留项。
 - 5. 只返回保留的前缀，并明确说明它是过滤后的子序列，不是原数据的排序结果。
- 小例子：[3,1,4,2,4] 保留 3、跳过 1、保留 4、跳过 2、保留 4，得到 [3,4,4]；1 与 2 已丢失。

效率与存储

最好	单次扫描压缩为 $\Theta(n)$
平均或期望	$\Theta(n)$
最坏	$\Theta(n)$ ；若反复在数组中间删除则可能退化为平方时间
额外空间	原地压缩为 $O(1)$ ，另建结果表则为 $O(n)$
稳定性	保留项相对次序不变，但排序稳定性并不适用
严格原地	可原地过滤，但不是合法的通用排序

为什么快，为什么慢

一遍扫描很便宜，是因为不合顺序的记录被丢掉了。删除数据绕过了排序要求中的难点，并没有解决它。

适合场景：只有确实需要这种单调子序列过滤器时才有实际用途，也可用来强调排序必须保留全部元素。

容易误解：仅“输出有序”还不够，排序必须保留输入的多重集；初值应取首项而非 0，否则负数也会被错误丢弃。

依据：[X-STALIN]

X25 睡眠排序

让每个数字等待与其值成比例的时间后输出，用时间先后代替显式比较。

具体怎么做

- 1. 选择非负数字与正延时比例 s ，并说明计时精度会限制可区分的数值。
 - 2. 为每个值 x 建立计时器或工作任务，目标释放时间设为共同起点加 $s \cdot x$ 。
 - 3. 到时而任务请求运行；调度器真正执行它时，再通过同步机制追加输出 x 。
 - 4. 等待所有任务结束，再收集输出序列。
 - 5. 另行验证最终序列；若计时器执行顺序错乱，应报告失败，不能假装排好了。
- 小例子：对 $[3,1,2]$ ，若 $s=1$ 秒，理想情况下 1 在第 1 秒、2 在第 2 秒、3 在第 3 秒输出；实际调度仍可能打乱次序。

效率与存储

最好	不能只用 n 给出有意义的界，数值与计时模型同样重要
平均或期望	理想经过时间 $O(n+sM)$ ， M 为最大值；不构成调度保证
最坏	任意调度下没有通用完成时限或顺序正确性保证
额外空间	$O(n)$ 个任务或计时器及输出，线程栈可能很昂贵
稳定性	同一目标时间的记录不保证稳定顺序
严格原地	否

为什么快，为什么慢

看似省掉的排序工作被交给了计时器与调度器。等待长度、计时精度、同时到期与调度延迟，都必须算进真实过程。

适合场景：适合讲解时钟、调度、同步，以及计时器内部隐藏的管理成本。

容易误解：sleep 只要求等待，并不保证精确时刻运行；大数导致长等待，微小差值、启动时间偏差和相等值都会暴露竞争问题。

依据：[X-SLEEP] [X-SLEEP-DOC]

X26 量子 Bogo 排序 笑话

这个多世界笑话幻想抛弃所有未排好的结果，只留下排好的一条宇宙分支。

具体怎么做

- 1. 只做纸面演示：列出极小输入的可能排列，把它们当作想象中的分支，而非可控制的真实宇宙。
- 2. 在每个想象分支中检查相邻元素，标出该排列是否有序。
- 3. 接下来笑话要求从现实中删除所有失败分支；必须把此步标记为“量子计算并未提供这种操作”。
- 4. 在纸上圈出成功排列即可；若写程序搜索这些候选，仍须支付真实搜索成本。

小例子：[2,1] 的想象分支是 [2,1] 与 [1,2]；圈出后者，并不等于解释了计算机如何免费得到它。

效率与存储

最好	对可实现的排序算法而言未定义
平均或期望	未定义；笑话把挑选成功结果当作免费能力
最坏	未定义，没有可执行的完成保证
额外空间	虚构步骤无法导出有意义的硬件空间界
稳定性	不适用
严格原地	不适用

为什么快，为什么慢

“丢弃失败宇宙”的故事没有给出可执行操作，真实测量规则并不会免费提供只选择已排好分支的能力。

适合场景：可说明谈论量子分支并不等于设计量子算法，测量也不能随意挑出想要的答案。

容易误解：没有建立任何 $O(1)$ 或 $O(n)$ 的加速结论；量子计算机没有“删除所有失败宇宙”的指令。

依据：[X-QUANTUM-JOKE] [X-QUANTUM-DOC]

X27 拓扑排序：另一种“排序”任务

名称／关系：拓扑次序；本文说明 Kahn 与 DFS 两种做法

它按依赖关系安排任务，不按数字大小排列。若任务 3 必须先于任务 1，就应把 3 放在 1 前面。正确输出要包含每个节点一次，并让所有依赖箭头都指向后面的节点。

具体做法

1. Kahn 版本：统计每个节点有多少条进入它的箭头，这叫入度。把所有入度为零的节点放进队列。
2. 从队列取出一个节点并输出，再删除它发出的箭头，即把后继节点的入度逐一减一；谁减到零，谁就入队。若队列空了却没输出全部节点，剩余图中存在有向环。
3. 深度优先版本：给节点标记“未见过、正在访问、已经结束”。遇到未访问节点时标成正在访问，递归处理它指向的所有未访问邻居；全部处理完后，标成结束并加入结束列表。
4. 若遇到指向“正在访问”节点的边，就检测到了环，应停止。没有环时，把结束列表反转即得答案。还要从每个未访问节点重新出发，以免漏掉不连通的部分。

跟着做：节点为 {1,2,3,4}，箭头是 $3 \rightarrow 1$ 、 $3 \rightarrow 2$ 、 $1 \rightarrow 4$ 、 $2 \rightarrow 4$ 。入度分别为 1:1、2:1、3:0、4:2。Kahn 队列先是 [3]；输出 3 后，1、2 都可入队；输出 1 后，4 的入度降为 1；输出 2 后，4 降为 0，最后输出 4。得到 [3,1,2,4]。

跟着做：深度优先则可从 3 走到 1，再走到 4；先结束 4，再结束 1。随后访问 2，它指向的 4 已结束，于是结束 2，最后结束 3。结束列表为 [4,1,2,3]，反转得到 [3,2,1,4]，也是合法答案。两种结果都不是数值升序。

效率与存储

时间	邻接表表示时 $\Theta(V+E)$ ，V 为节点数，E 为箭头数
额外工作空间	$O(V)$ ，不含 $O(V+E)$ 的图存储
稳定性与普通数值排序的比较下界	不适用

为什么快，为什么慢

每个节点只进入、结束一次，每条依赖边只检查常数次。这个线性上界针对图的规模，不是任意数值键的比较排序问题。

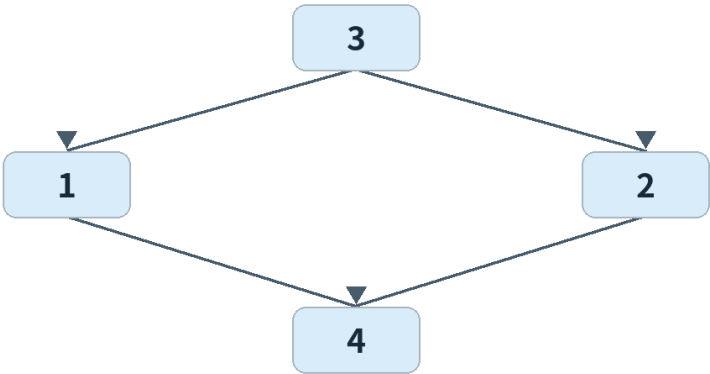
适合：课程先修条件、构建步骤、带依赖的工作安排。队列或邻居访问次序不同，可能得到多个同样正确的答案。

注意：若有 $1 \rightarrow 2 \rightarrow 1$ 这样的环，就不存在完整拓扑序。本条明确作为相关任务介绍，不将它冒充为任意数值数组的另一种排序方法。

来源：[N13] [N14]

过程图 X27

依赖关系：拓扑序不是数值大小顺序



Kahn: [3,1,2,4] DFS: [3,2,1,4]

两者都合法：每条箭头都从先执行的任务指向后执行的任务。

X27 依赖关系与拓扑次序。与本条目的示例对应；标记含义见“基础”。

关键伪代码

这些配方把边界规则写明确，不必先读代码才读前面的解释。缩进表示属于该循环或条件内部； $\leftarrow$ 表示赋值。比较看键，移动带上完整记录。数组下标从零开始。

插入排序

```

对 i = 1 到 A 的长度 - 1:
  暂存  $\leftarrow$  A[i]
  j  $\leftarrow$  i - 1
  当 j  $\geq$  0 且 键(A[j]) > 键(暂存) 时重复:
    A[j + 1]  $\leftarrow$  A[j]
    j  $\leftarrow$  j - 1
  A[j + 1]  $\leftarrow$  暂存
  
```

稳定的二路归并

```

归并(左队, 右队):
  输出  $\leftarrow$  空列表
  当两队都还有未取出的记录时重复:
    如果 键(左队头)  $\leq$  键(右队头):
      把左队头接到输出 并从左队取走
    否则:
      把右队头接到输出 并从右队取走
  把两队剩余记录依次接到输出
  返回 输出
  
```

```

归并排序(A):
  如果 A 的长度  $\leq$  1: 返回 A
  把 A 分为左半和右半
  返回 归并(归并排序(左半), 归并排序(右半))
  
```

三个容器版本的快速排序

```

快速排序(A):
  如果 A 的长度  $\leq$  1: 返回 A
  枢轴  $\leftarrow$  从 A 中均匀随机选一条记录
  小于组, 等于组, 大于组  $\leftarrow$  空列表
  按原顺序读取 A 中的每条记录 x:
    键(x) < 键(枢轴) 时将 x 接到小于组
    键(x) = 键(枢轴) 时将 x 接到等于组
    键(x) > 键(枢轴) 时将 x 接到大于组
  返回 快速排序(小于组) + 等于组 + 快速排序(大于组)
  
```

这个便于理解的版本使用额外列表；按输入顺序追加时可以稳定。它不是条目中按原地交换分析空间与稳定性的那个快速排序实现。

从零编号的堆排序

```

下沉(A, 根, 结束位置):
  当  $2 * \text{根} + 1 < \text{结束位置}$  时重复:
    孩子  $\leftarrow 2 * \text{根} + 1$ 
    如果右孩子存在 且右孩子更大:
      孩子  $\leftarrow \text{孩子} + 1$ 
    如果  $A[\text{根}] \geq A[\text{孩子}]$ : 退出循环
    交换  $A[\text{根}]$  与  $A[\text{孩子}]$ 
    根  $\leftarrow \text{孩子}$ 

```

```

从最后一个父节点到根 依次执行下沉
让结束位置从 A 的长度 - 1 递减到 1:
  交换  $A[0]$  与  $A[\text{结束位置}]$ 
  下沉(A, 0, 结束位置)

```

键范围为零到 k 减一的稳定计数排序

```

计数  $\leftarrow k$  个零
对 A 中每条记录 x:
  计数[键(x)]  $\leftarrow$  计数[键(x)] + 1
对 v = 1 到 k - 1:
  计数[v]  $\leftarrow$  计数[v] + 计数[v - 1]
输出  $\leftarrow$  与 A 等长的数组
对 i 从 A 的长度 - 1 递减到 0:
  v  $\leftarrow$  键( $A[i]$ )
  计数[v]  $\leftarrow$  计数[v] - 1
  输出[计数[v]]  $\leftarrow A[i]$ 
返回 输出

```

此过程移动完整记录。它要求键为指定范围内的整数，并且能够分配对应大小的计数数组。本附录用累计终点配合反向扫描；D01 用起点配合正向扫描。两者都可稳定，但不能混用边界与扫描方向。

定长非负整数的最低位优先基数排序

```

从最低位到最高位 逐位执行:
  A  $\leftarrow$  根据当前位对 A 做稳定计数排序
返回 A

```

Lomuto 与 Hoare：不要混用返回值

下面两种版本都使用包含端点的 lo..hi。Lomuto 返回某条枢轴记录的最终下标；Hoare 返回两个未完成区域之间的分界，不能像前者那样把分界元素从两侧递归中都排除。lo $\geq$ hi 的停止条件不能少。

[N01] [N02]

Lomuto

```

partition_lomuto(A, lo, hi):
    pivot ← 键(A[hi]); i ← lo
    对 j 从 lo 到 hi-1:
        若 键(A[j]) ≤ pivot:
            交换 A[i], A[j]; i ← i+1
    交换 A[i], A[hi]
    返回 i

quick_lomuto(A, lo, hi):
    若 lo ≥ hi: 返回
    q ← partition_lomuto(A, lo, hi)
    quick_lomuto(A, lo, q-1)
    quick_lomuto(A, q+1, hi)

```

Hoare

```

partition_hoare(A, lo, hi):
    pivot ← 键(A[lo]); i ← lo-1; j ← hi+1
    重复:
        重复: i ← i+1 直到 键(A[i]) ≥ pivot
        重复: j ← j-1 直到 键(A[j]) ≤ pivot
        若 i ≥ j: 返回 j
    交换 A[i], A[j]

quick_hoare(A, lo, hi):
    若 lo ≥ hi: 返回
    q ← partition_hoare(A, lo, hi)
    quick_hoare(A, lo, q)
    quick_hoare(A, q+1, hi)

```

C31 的完整二分旋转归并

这里改用半开区间: $[lo, hi)$ 不包含 hi 。lower_bound 找第一个 $\geq x$ 的键，upper_bound 找第一个 $> x$ 的键。reverse(a,b) 通过两端交换并向内收拢，原地反转 $[a,b)$ 。连续三次反转能交换相邻块的位置，同时恢复每块内部原来的顺序。[N05]

```

lower_bound(A, lo, hi, x):
    当 lo < hi:
        m ← 向下取整((lo+hi)/2)
        若 键(A[m]) < x: lo ← m+1
        否则: hi ← m
    返回 lo
upper_bound(A, lo, hi, x):
    当 lo < hi:
        m ← 向下取整((lo+hi)/2)
        若 键(A[m]) ≤ x: lo ← m+1
        否则: hi ← m
    返回 lo

```

```
merge_rotate(A, lo, mid, hi):  
  若 lo == mid 或 mid == hi: 返回  
  若 键(A[mid-1]) <= 键(A[mid]): 返回  
  若 hi-lo == 2:  
    交换 A[lo], A[mid]; 返回  
  若 mid-lo > hi-mid:  
    a ← lo + 向下取整((mid-lo)/2)  
    b ← lower_bound(A, mid, hi, 键(A[a]))  
  否则:  
    b ← mid + 向下取整((hi-mid)/2)  
    a ← upper_bound(A, lo, mid, 键(A[b]))  
  reverse(A, a, mid)  
  reverse(A, mid, b)  
  reverse(A, a, b)  
  new_mid ← a + (b-mid)  
  merge_rotate(A, lo, a, new_mid)  
  merge_rotate(A, new_mid, b, hi)  
sort_rotate(A, lo, hi):  
  若 hi-lo < 2: 返回  
  mid ← 向下取整((lo+hi)/2)  
  sort_rotate(A, lo, mid)  
  sort_rotate(A, mid, hi)  
  merge_rotate(A, lo, mid, hi)
```

按任务选择

先看约束，不要先选一个喜欢的名字。实际开发通常应优先使用正确且经过测试的库实现，而不是手写巧妙变体；学习这些原理，是为了看懂保证，以及识别自己的数据是否值得换一种模型。

任务／约束	适合的方向	理由
一般记录，没有特殊键结构	B：归并或带保护的快速排序家族	先保证最坏增长，再决定稳定性和内存。
少量错位、小批数据	C01/C02；确有 k 位移上限时可看 D30	开销小，或工作量随乱序程度变化。
大量重复键	C14 或稳定自然段方法；小范围时用 D01	避免对等值区域反复递归。
要求稳定，内存足够	C09/C10/C18/C19	额外缓冲让稳定归并更直接。
要求稳定，不能开线性缓冲	C31 逻辑较短；经核验的 C23 实现有更强界	要接受更多移动，或更复杂的构造。
受限整数或定长键	A：满足条件时计数、基数或直接寻址	利用表示结构，不只做比较。
长字符串且公共前缀多	D10/D11/D27	少重复检查相同开头的字符。
文件大于内存	D14；有帮助时配合 D15	生成段、按块顺序传输、用有限缓冲合并。
多处理器或数据无关电路	H，并代入实际机器约束	总工作、深度、通信不是同一成本。
任务依赖	X27，而不是数值排序	所需顺序由依赖箭头定义。

测速时，各方法应处理同一批记录、同样的键函数、稳定性要求与内存额度。随机、已排好、倒序、重复值很多、接近有序等输入要分开测试。不要一边逐步打印一边测速，显示耗时可能淹没算法本身。

练习与参考答案

先自己做

1. 对 [4,2,3,1] 做插入排序，写出每次插入后的数组。
2. 为什么 [1,2,3] 不是 [3,2,1,2] 的正确排序结果？
3. 对 [3 甲,3 乙,1] 做选择排序的第一次交换，它稳定吗？
4. 合并 [1,4,7] 与 [2,3,8]，每次取哪一侧的段首？
5. Hoare 在 [4,1,3,2] 上返回 $q=2$ ，递归处理哪些范围？
6. 对 [21,12,13] 做 LSD 的十位轮次时，若把同十位记录反转会怎样？
7. 计数为 $O(n+k)$ ，对 $[0,10^{12}]$ 是否就一定合适？
8. 每个元素距离最终位置至多 $k=2$ 时，最小堆最多要放几条？
9. $O(\log^2 n)$ 个并行轮次是否就比 $O(n \log n)$ 个串行操作更省工作？
10. 要求稳定，又不能申请线性缓冲，普通堆排序能同时满足吗？
11. 依赖为 $3 \rightarrow 1$ 、 $3 \rightarrow 2$ 、 $1 \rightarrow 4$ 、 $2 \rightarrow 4$ 时，[1,2,3,4] 是否为合法执行顺序？
12. 为什么随机猜测排序可能有有限期望成本，却没有有限最坏截止时间？

参考答案

1. $[2,4|3,1] \rightarrow [2,3,4|1] \rightarrow [1,2,3,4]$ 。最后一次先保存 1，依次右移 4、3、2，再写入 1。
2. 少了一条 2。数值有序和保留全部记录，是两个独立要求。
3. 最小值 1 与 3 甲 交换，得到 $[1,3 \text{ 乙}, 3 \text{ 甲}]$ 。相等键反序，所以该实现不稳定；改为整体搬移前面的记录，可以保留原顺序。
4. 依次取左 1、右 2、右 3、左 4、左 7；左侧已空，再接上右 8，得到 $[1,2,3,4,7,8]$ 。
5. 处理包含端点的 0..2、3..3。枢轴值 4 此时在下标 3，不在 q；不能把 q 当成已归位枢轴而从两侧递归中都去掉。
6. 可能得到 $[13,12,21]$ 。十位没错，但个位建立的顺序被破坏；每轮稳定性是关键。
7. 不一定。密集计数表大约需要一万亿个槽，但 n 只有 2；取值范围反而主导成本。
8. 三条，即 $k+1$ 。输出下一个最小值前，所有可能占据该位置的记录都必须已入堆；前提不能省略。
9. 它们衡量的不是同一件事。双调网络深度为对数平方，但总比较量为 $n \log^2 n$ 。比较前先写处理器数及每轮工作量。
10. 不能。普通堆排序节省辅助空间，却不稳定。稳定块归并更复杂；C31 是较短的稳定旋转方案，但有 $O(\log n)$ 栈和更多搬动。
11. 不合法。1、2 不能在 3 之前执行。 $[3,1,2,4]$ 、 $[3,2,1,4]$ 都可以；单纯按数字大小排错了问题。
12. 对随机尝试求平均可能收敛，但任意长的连续失败仍有可能。期望时间不是到那个时刻一定完成的承诺。

术语表

English	中文	解释
Algorithm	算法	完成任务的一组明确步骤
Array	数组	可按位置编号访问的一串元素
Key	键	用来决定先后顺序的值
Record	记录	必须随键一起移动的完整数据
Comparison	比较	判断两个键的大小或先后
Swap	交换	把两个位置上的元素互换
Shift	移动	为腾位置而搬动一段元素
Stable	稳定	相等键的记录保留原先后
In place	原地	严格定义下只用常数额外存储
Auxiliary space	额外空间	输入输出之外的工作存储
Adaptive	自适应	利用输入中已有的某种顺序
Inversion	逆序对	前面元素大于后面元素的一对位置
Run	有序段	连续的一段已经排好序的元素
Pivot	枢轴	用来划分左右或多个区域的值
English	中文	解释
Partition	划分	按枢轴等规则把元素分区
Merge	归并	把多个有序序列合成一个有序序列
Heap	堆	父子之间满足规定大小关系的树
Recursion	递归	用规模更小的同类问题解决原问题
Bucket	桶	存放某一类键的容器
Radix	基数	每一位可能取值的数量
Distribution	分布	键在取值范围中的疏密情况
Expected cost	期望成本	在明确概率模型下加权平均的成本
Worst case	最坏情况	允许的输入或选择中最大的成本
Work	总工作量	所有处理器完成的操作总数
Depth or span	深度或跨度	并行计算中最长的依赖步骤链
I O complexity	输入输出复杂度	不同存储层之间的数据块传输次数
Permutation	排列	不增不减地重排原来的全部元素
Multiset	多重集合	既考虑元素也考虑出现次数的集合

术语说明的是性质或操作，不自动保证速度。“稳定”“原地”“自适应”“并行”“缓存无关”回答的是不同问题。

别名与重要区别

算法数量不能靠名字来数。目录明确保留以下区别；相应条目也逐项列出名称关系。

名称或变体	位置与真正改变的动作
快速排序、Lomuto、Hoare、三路、双枢轴	C13—C15、C26—C27。分区保证和递归端点不同；只换枢轴取法，不算一项全新独立发明。
Introsort、pdqsort、BlockQuicksort	C17、C21、C22。分别给快速排序家族加入后备策略、模式处理或分块分区。
归并家族与调度策略	C09—C12、C18—C20、C24、C31。拆分方式、发现自然段、合并次序、具体合并动作，是不同层面的设计。
TimSort 与 Powersort	C18 是经典实现策略；C19 是可放入 TimSort 式框架的归并调度。既不能说完全无关，也不能简单当成同义词。
WikiSort 与 GrailSort	C23 讲共同的稳定块归并家族，并标明具体构造差别。两个名称不代表恢复规则相同。
exchange sort／互换排序	C04、C30。NIST 把 exchange sort 作为冒泡别名，一些代码却用它表示与后面所有位置逐项比较的过程。应看循环，不只看名称。 [N03] [N04]
奇偶归并与奇偶换位	D19 与 D20。前者递归归并分离的子序列，后者轮流比较相邻的奇偶位置对。
计数、鸽巢、桶、基数与位图	D01—D07、D29。都利用分布，但槽的含义、重复值存法，以及桶内是否还要排序，都不同。
循环排序与直接寻址循环放置	X08 与 X09。前者靠统计较小记录数定位；后者只有在规定排列前提下才能直接算出位置。
猴子／Bogo 与排列穷举	X19、X22。随机尝试可能一直重复失败；穷举按规则遍历，不漏候选，具有有限的枚举上界。

增量序列、三数取中或更大枢轴样本、哨兵、插入阈值、二叉或 d 叉堆、特定 SIMD 内核，都可能重要，但不在这里每个单列为完整条目。本书解释所选版本，不把一个版本的精确界移给所有变体。

核验、依据与局限

两版使用相同的 90 个条目标识、25 幅过程图与 130 条参考记录。别名说明和范围页区分完整过程、变体、家族、部件、物理模型、理论概述与非排序。参考记录的数量，不代表同样数量的互不相关研究论文。

独立教学实现对 29 个过程运行了 32,611 个测试用例。核心测试包括由 $\{0,1,2\}$ 构成、长度从零到六的全部数组，额外随机数组，600 例二的幂长度的双调网络测试，以及 500 例 $r=8$ 、 $s=2$ 的 Columnsort 测试。已测试结果均与另外计算的正确排序结果一致，包括重复值次数。测试实现用于验证逻辑，Python 中用了多少临时存储，不能作为原地空间界的证明。

重点核对了 Hoare 的返回分界、稳定旋转的切点、LSD 稳定轮次、相等键处理、Columnsort 完整的重排与移位过程、后备算法前提，以及期望和最坏情况的区别。复杂度说明对照了选定原始论文、作者实现、官方文档，以及 NIST、Princeton、MIT 的教学资料。

这不是每种实现的形式化证明，也不声称已把 90 个条目全部作为工业程序执行。少量测试无法证明渐近复杂度。高级构造概述有意保留范围，不代替所引代码与证明。文档经过渲染检查排版和图像；引用网页与主分支实现仍可能在本版之后变化。

参考来源与延伸阅读

正文方括号编号可跳到这里；标题和地址均可点击。某项资料可能只支持特定变体或成本模型，不代表同家族所有版本。数字示例和过程图为本手册整理。参考资料按 2026 年 10 月 5 日版本选取，并核对关键结论。

[F1] MIT 6 006 Lecture 5 Linear Sorting

<https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/pages/lecture-notes/>

[C01] Sedgewick & Wayne: Elementary Sorts

<https://algs4.cs.princeton.edu/21elementary/>

[C02] Princeton BinaryInsertion.java

<https://algs4.cs.princeton.edu/code/edu/princeton/cs/algs4/BinaryInsertion.java.html>

[C03] Princeton Selection.java

<https://algs4.cs.princeton.edu/21elementary/Selection.java.html>

[C04] NIST: Bubble sort

<https://xlinux.nist.gov/dads/HTML/bubblesort.html>

[C05] NIST: Bidirectional bubble sort

<https://xlinux.nist.gov/dads/HTML/bidirectionalBubbleSort.html>

[C06] NIST: Gnome sort

<https://xlinux.nist.gov/dads/HTML/gnomeSort.html>

[C07] NIST: Comb sort

<https://xlinux.nist.gov/dads/HTML/combSort.html>

[C08] Princeton Shell.java

<https://algs4.cs.princeton.edu/21elementary/Shell.java.html>

[C09] Princeton Merge.java

<https://algs4.cs.princeton.edu/22mergesort/Merge.java.html>

[C10] Princeton MergeBU.java

<https://algs4.cs.princeton.edu/22mergesort/MergeBU.java.html>

[C11] Sedgewick & Wayne: Mergesort

<https://algs4.cs.princeton.edu/22mergesort/>

[C12] Liang Huang, Oregon State: Applications of Heap

https://web.engr.oregonstate.edu/~huanlian/algorithms_course/1-datastructures/heap_apps.html

[C13] Sedgewick & Wayne: Quicksort

<https://algs4.cs.princeton.edu/23quicksort/>

[C14] Princeton Quick3way.java

<https://algs4.cs.princeton.edu/23quicksort/Quick3way.java.html>

[C15] Princeton QuickDualPivot.java

<https://algs4.cs.princeton.edu/23quicksort/QuickDualPivot.java.html>

[C16] Princeton Heap.java

<https://algs4.cs.princeton.edu/24pq/Heap.java.html>

[C17] Musser: Introspective Sorting and Selection Algorithms

<https://webpages.charlotte.edu/rbunescu/courses/ou/cs4040/introsort.pdf>

[C18] Tim Peters: CPython 3.10 listsort design

<https://github.com/python/cpython/blob/v3.10.0/Objects/listsort.txt>

[C19] CPython: Current listsort design

<https://github.com/python/cpython/blob/main/Objects/listsort.txt>

[C20] Munro & Wild: Nearly-Optimal Mergesorts

<https://arxiv.org/pdf/1805.04154>

[C21] Sebastian Wild: Nearly-Optimal Mergesorts author page

<https://www.wild-inter.net/publications/munro-wild-2018>

[C22] Orson Peters: pdqsort implementation and design

<https://github.com/orlp/pdqsort>

[C23] Edelkamp & Weiß: BlockQuicksort

<https://arxiv.org/pdf/1604.06697>

[C24] Mike McFadden: WikiSort

<https://github.com/BonzaiThePenguin/WikiSort>

[C25] WikiSort, Chapter 3: In-Place

<https://github.com/BonzaiThePenguin/WikiSort/blob/master/Chapter%203.%20In-Place.md>

[C26] Andrey Astrelin: GrailSort

<https://github.com/Mrrl/GrailSort>

[C27] Vincent Jugé: Adaptive Shivers Sort (author manuscript, 2024)

<https://monge.univ-eiffel.fr/~juge/papers/talg2024.pdf>

[C28] Sebastian Wild: Powersort implementation repository

<https://github.com/sebawild/powersort>

[C29] Wild & Nebel: Analysis of Pivot Sampling in Dual-Pivot Quicksort

<https://arxiv.org/abs/1412.0193>

[C30] CPython 3.10 listobject.c, merge_collapse and minrun

<https://raw.githubusercontent.com/python/cpython/v3.10.0/Objects/listobject.c>

[C31] Orson Peters: pdqsort.h

<https://raw.githubusercontent.com/orlp/pdqsort/master/pdqsort.h>

[C32] Stober & Weiß: On the Average Case of MergeInsertion

<https://arxiv.org/pdf/1905.09656>

[C33] Morwenn: cpp-sort merge_insertion_sort.h

https://github.com/Morwenn/cpp-sort/blob/1.x.y-stable/include/cpp-sort/detail/merge_insertion_sort.h

[C34] Sedgewick & Wayne: Mergesort, optimal nonoblivious sorting exercise

<https://algs4.cs.princeton.edu/22mergesort/index.php>

[DIST01] MIT 6.006: Sorting III, Counting Sort and Radix Sort

https://courses.csail.mit.edu/6.006/fall09/lecture_notes/lecture10.pdf

[DIST02] NIST Dictionary of Algorithms and Data Structures: Pigeonhole sort

<https://xlinux.nist.gov/dads/HTML/pigeonholeSort.html>

[DIST03] Sedgewick and Wayne: LSD.java

<https://algs4.cs.princeton.edu/51radix/LSD.java.html>

[DIST04] Sedgewick and Wayne: MSD.java

<https://algs4.cs.princeton.edu/51radix/MSD.java.html>

[DIST05] Sedgewick, Wayne and Pesin: AmericanFlag API

<https://algs4.cs.princeton.edu/code/javadoc/edu/princeton/cs/algs4/AmericanFlag.html>

[DIST06] Princeton COS 226: String Sorts lecture

<https://www.cs.princeton.edu/courses/archive/spring19/cos226/lectures/51StringSorts.pdf>

[DIST07] Robert Sedgewick: Algorithms in Java, Chapter 10, Radix Sorting

<https://www.cs.princeton.edu/courses/archive/spr09/cos226/handouts/Algs3Ch10.pdf>

[DIST08] Sedgewick and Wayne: String Sorts, uniformly distributed data exercise

<https://algs4.cs.princeton.edu/51radix/>

[DIST09] Karl-Dietrich Neubert: The FlashSort Algorithm

<https://www.neubert.net/FSOIntro.html>

[DIST10] M. S. Rahman: thesis, section 7.2 Flashsort

<https://www.mathematics-and-computer-science.leicester.ac.uk/people/rraman/rahman-thesis.pdf>

[DIST11] Norman Jacobson, UC Irvine: A Synopsis of ProxmapSort and ProxmapSearch

<https://ics.uci.edu/~jacobson/ics23/ProxmapHandout.pdf>

[DIST12] Thomas A. Standish: ProxmapSort and ProxmapSearch, Part I

<https://ics.uci.edu/~standish/ProxmapSortSearchPartI.pdf>

[DIST13] Ranjan Sinha and Justin Zobel: Cache-Conscious Sorting of Large Sets of Strings with Dynamic Tries

<https://people.eng.unimelb.edu.au/jzobel/fulltext/acmjea04.pdf>

[DIST14] Ranjan Sinha, Justin Zobel and David Ring: Cache-Efficient String Sorting Using Copying

<https://people.eng.unimelb.edu.au/jzobel/fulltext/acmjea06.pdf>

[DIST15] Jon Bentley and Robert Sedgewick: Fast Algorithms for Sorting and Searching Strings

<https://www.cs.tufts.edu/comp/150FP/archive/bob-sedgewick/fast-strings.pdf>

[DIST16] Sedgewick and Wayne: Quick3string API

<https://algs4.cs.princeton.edu/code/javadoc/edu/princeton/cs/algs4/Quick3string.html>

[DIST17] CUNY: Sample Sort lecture

<https://www.cs.csi.cuny.edu/~yumei/csc744/Samplesorting.pdf>

[DIST18] Kurt Mehlhorn and Peter Sanders: Algorithms and Data Structures, The Basic Toolbox, external sorting chapter

<https://users.dcc.uchile.cl/~nbaloian/cc3001-02/Libros/Mehlhorn-Sanders-Toolbox.pdf>

[DIST19] Vipul Harsh, Laxmikant Kale and Edgar Solomonik: Histogram Sort with Sampling

<https://arxiv.org/html/1803.01237>

[DIST20] University of Illinois: Histogram sort with sampling publication record

<https://experts.illinois.edu/en/publications/histogram-sort-with-sampling/>

[DIST21] STXXL: Parallel Disk Sorting

https://stxxl.org/tags/1.4.1/design_algo_sorting.html

[DIST22] STXXL: stxxl::sort, Sorting Comparison-Based

https://stxxl.org/tags/1.4.1/design_algo_sort.html

[DIST23] OpenDSA: External Sorting, Replacement Selection

<https://opensa.org/OpenDSA/Books/Catalog/html/ExternalSort.html>

[DIST24] Gordon College CS321: External Sorting

https://www.math-cs.gordon.edu/courses/cs321/lectures/external_sorting.html

[DIST25] Thomas Niemann, University of Michigan course materials: External Sorting

https://www.eecs.umich.edu/courses/eecs380/ALG/niemann/s_ext.htm

[DIST26] Apache Hadoop: [org.apache.hadoop.examples.terasort](https://hadoop.apache.org/docs/current/api/org/apache/hadoop/examples/terasort/package-summary.html) package

<https://hadoop.apache.org/docs/current/api/org/apache/hadoop/examples/terasort/package-summary.html>

[DIST27] Pavel Tvrdik, University of Wisconsin course: Sorting on hypercubic networks

<https://pages.cs.wisc.edu/~tvrdik/17/html/Section17.html>

[DIST28] University of Oregon: Batcher bitonic sorting case study

<https://www.cs.uoregon.edu/research/paraducks/papers/psc94.d/node2.html>

[DIST29] Western Michigan course notes: Odd-even mergesort basics

<https://cs.wmich.edu/gupta/teaching/cs5260/5260Sp15web/lectureNotes/Odd-even%20mergesort%20basics.pdf>

[DIST30] MIT 18.310: Non-Adaptive Sorting, Batcher's Algorithm

https://math.mit.edu/~djik/18.310/Lecture-Notes/non_adaptive_sorting-2007.html

[DIST31] Pavel Tvrdik, University of Wisconsin course: Sorting on mesh-based topologies

<https://pages.cs.wisc.edu/~tvrdik/15/html/Section15.html>

[DIST32] Oded Green, Saher Odeh and Yitzhak Birk: Merge Path, A Visually Intuitive Approach to Parallel Merging

<https://arxiv.org/pdf/1406.2628>

[DIST33] Saher Odeh et al.: Merge Path, Cache-Efficient Parallel Merge and Sort, CCIT Report 802

https://ece.technion.ac.il/wp-content/uploads/2021/01/publication_abstract_802-1.pdf

[DIST34] Murshed: The Reconfigurable Mesh, thesis, section 4.3.3

<https://maths-people.anu.edu.au/brent/pd/Murshed-thesis.pdf>

[DIST35] Geeta Chaudhry and Thomas H. Cormen: Stupid Columnsort Tricks

https://digitalcommons.dartmouth.edu/cs_tr/210/

[DIST36] Boost.Sort: Spreadsort documentation

https://www.boost.org/doc/libs/latest/libs/sort/doc/html/sort/single_thread/spreadsort.html

[DIST37] Steven Ross: The Spreadsort High-Performance General-Case Sorting Algorithm

https://www.boost.org/doc/libs/1_61_0/libs/sort/doc/papers/original_spreadsort06_2002.pdf

[DIST38] Behrooz Parhami, UCSB: Introduction to Parallel Processing, instructor materials

https://web.ece.ucsb.edu/~parhami/pubs_folder/f12_im_par_v2_w02.pdf

[DIST39] Gerth Stølting Brodal and Rolf Fagerberg: Cache Oblivious Distribution Sweeping, section 2 Lazy Funnelsort

<https://cs.au.dk/~gerth/papers/icalp02cache.pdf>

[DIST40] Matteo Frigo et al.: Cache-Oblivious Algorithms

<https://cs.uwaterloo.ca/~imunro/cs840/frigo.pdf>

[DIST41] Behrooz Parhami, UCSB: Parallel Processing, Fundamental Concepts slides

https://web.ece.ucsb.edu/~parhami/pres_folder/f32-book-parallel-pres-pt1.pdf

[DIST_AKS_01] Miklós Ajtai, János Komlós and Endre Szemerédi: An $O(n \log n)$ Sorting Network, STOC 1983, pp. 1–9

<https://doi.org/10.1145/800061.808726>

[DIST_AKS_02] Joel Seiferas: Sorting Networks of Logarithmic Depth, Further Simplified, author technical report, revised 2007

<https://urresearch.rochester.edu/fileDownloadForInstitutionalItem.action?itemFileId=3258&itemId=2353>

[DIST_AKS_03] Emil Jeřábek: A Sorting Network in Bounded Arithmetic, author manuscript, 2010, section 3

<https://users.math.cas.cz/~jerabek/papers/aks.pdf>

[X-BST] Princeton: Symbol Tables and inorder BST traversal

<https://introcs.cs.princeton.edu/java/44st/>

[X-BALANCED] Princeton: Red-Black Trees (Verified Functional Algorithms)

<https://www.cs.princeton.edu/courses/archive/spring21/cos510/sf/vfa/Redblack.html>

[X-AVL] Princeton Algorithms: AVLTreeST

<https://algs4.cs.princeton.edu/99misc/AVLTreeST.java.html>

[X-TOURNAMENT] Stepanov and Kershnerbaum: Using Tournament Trees to Sort

<https://stepanovpapers.com/TournamentTrees.pdf>

[X-SMOOTH] Edsger W. Dijkstra: Smoothsort, an alternative for sorting in situ (EWD796a)

<https://www.cs.utexas.edu/~EWD/transcriptions/EWD07xx/EWD796a.html>

[X-SMOOTH-IMPL] Keith Schwarz: Smoothsort Demystified

<https://www.keithschwarz.com/smoothsort/>

[X-PATIENCE] Aldous and Diaconis: Patience Sorting, Longest Increasing Subsequences and a Continuous Space Analog

<https://www.stat.berkeley.edu/users/aldous/Research/OP/patience.pdf>

[X-PATIENCE-CODE] TheAlgorithms/Python: patience_sort.py

https://github.com/TheAlgorithms/Python/blob/master/sorts/patience_sort.py

[X-LIBRARY] Bender, Farach-Colton and Mosteiro: Insertion Sort is $O(n \log n)$

<https://arxiv.org/abs/cs/0407003>

[X-STRAND] NIST Dictionary of Algorithms and Data Structures: strand sort

<https://xlinux.nist.gov/dads/HTML/strandSort.html>

[X-CYCLE-CODE] TheAlgorithms/Python: cycle_sort.py

https://github.com/TheAlgorithms/Python/blob/master/sorts/cycle_sort.py

[X-CYCLE-PAPER] B. K. Haddon: Cycle-Sort: A Linear Sorting Method

https://www.researchgate.net/publication/220459798_Cycle-Sort_A_Linear_Sorting_Method

[X-PANCAKE] Carnegie Mellon 15-251: Sorting by prefix reversal

<https://www.cs.cmu.edu/afs/cs.cmu.edu/academic/class/15251-f10/Site/Materials/Lectures/Lecture01/lecture01.pdf>

[X-BURNT] Anthony Labarre and Josef Cibulka: Polynomial-time sortable stacks of burnt pancakes

<https://arxiv.org/abs/1010.0219>

[X-BEAD] Arulanandham, Calude and Dinneen: A fast natural algorithm for searching

<https://www.cs.auckland.ac.nz/~cristian/FastNaturalAlgorithm.pdf>

[X-SPAGHETTI] Erik Koch: Programming a Quantum Computer (spaghetti-computer model)

<https://www.cond-mat.de/teaching/QC/QCintro.pdf>

[X-WEAK] R. D. Dutton: Weak-Heap Sort (University of Central Florida repository)

<https://stars.library.ucf.edu/facultybib1990/692/>

[X-WEAK-CODE] guitarrapc/SortAlgorithms: WeakHeapSort.cs

<https://github.com/guitarrapc/SortAlgorithms/blob/main/src/SortAlgorithm/Algorithms/Heap/WeakHeapSort.cs>

[X-CARTESIAN] David Eppstein: Splaysort versus Cartesian tree sort

<https://11011110.github.io/blog/2014/01/21/splaysort-versus-cartesian.html>

[X-CARTESIAN-CODE] Morwenn/cpp-sort: Sorters documentation

<https://github.com/Morwenn/cpp-sort/wiki/Sorters>

[X-SPLAY] Moffat, Eddy and Petersson: Splaysort: Fast, Versatile, Practical

[https://doi.org/10.1002/\(SICI\)1097-024X\(199607\)26:7<781::AID-SPE35>3.0.CO;2-B](https://doi.org/10.1002/(SICI)1097-024X(199607)26:7<781::AID-SPE35>3.0.CO;2-B)

[X-SPLAY-BASE] Sleator and Tarjan: Self-Adjusting Binary Search Trees

<https://www.cs.cmu.edu/~sleator/papers/self-adjusting.pdf>

[X-STOOGES] NIST Dictionary of Algorithms and Data Structures: stooge sort

<https://xlinux.nist.gov/dads/HTML/stoogesort.html>

[X-SLOW] Broder and Stolfi: Pessimal Algorithms and Simplexity Analysis

<https://graphics.stanford.edu/courses/cs161-16-winter/Handouts/pessimal-algorithms.pdf>

[X-BOGO] NIST Dictionary of Algorithms and Data Structures: bogosort

<https://xlinux.nist.gov/dads/HTML/bogosort.html>

[X-RANDOM-SLOW] Gruber, Holzer and Ruepp: Sorting the Slow Way

<https://archive.air.in.tum.de/Main/Publications/Ruepp2007.pdf>

[X-BOGBOGO] David Morgan-Mar: Bogobogosort

<https://dangermouse.net/esoteric/bogobogosort.html>

[X-PERMUTATION] NIST Dictionary of Algorithms and Data Structures: permutation sort

<https://xlinux.nist.gov/dads/HTML/permutationSort.html>

[X-PERMUTATION-CODE] Michael Guerzhoy, University of Toronto: Derandomized Bogosort

<https://www.cs.toronto.edu/~guerzhoy/bogosort/DerandomizedBogosort.html>

[X-MIRACLE] chandraseta: Implementation of Miracle Sort in Python

<https://gist.github.com/chandraseta/c39689fca3f03eacac4b8005084c89dc>

[X-STALIN] gustavo-depaula/stalin-sort: project README

<https://github.com/gustavo-depaula/stalin-sort>

[X-SLEEP] Kevlin Henney: Need Something Sorted? Sleep On It!

<https://accu.org/journals/overload/31/175/henney/>

[X-SLEEP-DOC] Python documentation: time.sleep

<https://docs.python.org/3/library/time.html#time.sleep>

[X-QUANTUM-JOKE] mathNEWS 110.6: Quantum Bogosort (2009)

<https://mathnews.uwaterloo.ca/wp-content/uploads/2009/07/mathNEWS-110-6.pdf>

[X-QUANTUM-DOC] IBM Quantum Learning: General measurements

<https://learning.quantum.ibm.com/course/general-formulation-of-quantum-information/general-measurements>

[N01] Cormen et al., Introduction to Algorithms, 4th ed. — Quicksort

<https://mitpress.mit.edu/9780262046305/introduction-to-algorithms/>

[N02] Hoare, Quicksort (1962)

<https://doi.org/10.1093/comjnl/5.1.10>

[N03] NIST DADS — exchange sort

<https://xlinux.nist.gov/dads/HTML/exchangeSort.html>

[N04] TheAlgorithms/Python — exchange_sort.py

https://raw.githubusercontent.com/TheAlgorithms/Python/master/sorts/exchange_sort.py

[N05] GCC libstdc++ — merge without buffer

https://raw.githubusercontent.com/gcc-mirror/gcc/master/libstdc%2B%2B-v3/include/bits/stl_algo.h

[N06] Edelkamp & Weiß, QuickMergesort (2018)

<https://arxiv.org/html/1804.10062>

[N07] Sedgewick & Wayne — TrieST.java

<https://algs4.cs.princeton.edu/52trie/TrieST.java.html>

[N08] Sedgewick & Wayne — rank definition in BinarySearchST

<https://algs4.cs.princeton.edu/31elementary/BinarySearchST.java.html>

[N09] Bentley, Programming Pearls, 2nd ed.

<https://www.informit.com/store/programming-pearls-9780201657883>

[N10] Sedgewick & Wayne — Priority Queues

<https://algs4.cs.princeton.edu/24pq/>

[N11] Axtmann et al., In-place Parallel Super Scalar Samplesort (2017)

<https://arxiv.org/html/1705.02257>

[N12] Axtmann et al., Engineering In-place (Shared-memory) Sorting Algorithms

<https://arxiv.org/abs/2009.13569>

[N13] Sedgewick & Wayne — TopologicalX.java

<https://algs4.cs.princeton.edu/42digraph/TopologicalX.java.html>

[N14] Sedgewick & Wayne — Directed Graphs

<https://algs4.cs.princeton.edu/42digraph/>

[F2] NIST Dictionary of Algorithms and Data Structures — sort

<https://xlinux.nist.gov/dads/HTML/sort.html>

[F3] Sedgewick & Wayne — Algorithms and Data Structures Cheatsheet

<https://algs4.cs.princeton.edu/cheatsheet/>

中文版结束。